MMKP Resource Allocation in IoT-EC-CC Networks

1.0.0

Generated by Doxygen 1.9.8

1 MMKP Resource Allocation in IoT-EC-CC Networks 1
1.1 Project Structure . . . 1
1.2 Prerequisites . . . 1
1.3 How to Compile and Run . . . 1
1.3.1 Option 1: Using <tt>build.sh</tt> (Recommended) . . . 1
1.3.2 Option 2: Using CMake . . . 2
1.4 Implemented Algorithms . . . 2
1.5 Simulation Parameters . . . 2
1.6 Simulation Results (<tt>results/</tt>) . . . 2
1.7 Documentation (Doxygen) . . . 3

2 Namespace Index 3
2.1 Namespace List . . . 3

3 Hierarchical Index 4
3.1 Class Hierarchy . . . 4

4 Class Index 4
4.1 Class List . . . 4

5 File Index 5
5.1 File List . . . 5

6 Namespace Documentation 5
6.1 convert Namespace Reference . . . 5
6.1.1 Detailed Description . . . 6
6.1.2 Function Documentation . . . 6
6.2 core Namespace Reference . . . 8
6.2.1 Detailed Description . . . 8
6.2.2 Typedef Documentation . . . 8
6.3 data Namespace Reference . . . 8
6.4 data::generate Namespace Reference . . . 8
6.4.1 Detailed Description . . . 9
6.4.2 Function Documentation . . . 9
6.4.3 Variable Documentation . . . 10
6.5 data::load Namespace Reference . . . 10
6.5.1 Detailed Description . . . 10
6.5.2 Function Documentation . . . 11
6.6 display Namespace Reference . . . 11
6.6.1 Detailed Description . . . 12
6.6.2 Function Documentation . . . 12
6.7 display::details Namespace Reference . . . 14
6.7.1 Detailed Description . . . 14
6.7.2 Function Documentation . . . 14

6.7.3 Variable Documentation . . . 15
6.8 exact Namespace Reference . . . 15
6.8.1 Detailed Description . . . 16
6.8.2 Typedef Documentation . . . 16
6.8.3 Function Documentation . . . 16
6.9 exact::anonymous_namespace{exact.hpp} Namespace Reference . . . 16
6.9.1 Function Documentation . . . 16
6.10 file_manager Namespace Reference . . . 17
6.11 file_manager::csv Namespace Reference . . . 17
6.11.1 Detailed Description . . . 17
6.11.2 Function Documentation . . . 17
6.12 heuristic Namespace Reference . . . 19
6.12.1 Detailed Description . . . 19
6.12.2 Function Documentation . . . 19
6.13 heuristic::anonymous_namespace{heuristic.hpp} Namespace Reference . . . 19
6.13.1 Function Documentation . . . 19
6.14 metaheuristic Namespace Reference . . . 20
6.14.1 Detailed Description . . . 21
6.14.2 Function Documentation . . . 21
6.15 metaheuristic::anonymous_namespace{metaheuristic.hpp} Namespace Reference . . . 21
6.15.1 Function Documentation . . . 21
6.15.2 Variable Documentation . . . 23
6.16 metrics Namespace Reference . . . 23
6.16.1 Detailed Description . . . 24
6.16.2 Enumeration Type Documentation . . . 24
6.16.3 Function Documentation . . . 24
6.16.4 Variable Documentation . . . 25
6.17 randomness Namespace Reference . . . 26
6.17.1 Detailed Description . . . 26
6.17.2 Function Documentation . . . 26
6.18 sorting Namespace Reference . . . 28
6.18.1 Detailed Description . . . 28
6.18.2 Function Documentation . . . 28
6.19 sorting::details Namespace Reference . . . 29
6.19.1 Detailed Description . . . 30
6.19.2 Function Documentation . . . 30

7 Class Documentation **30**

7.1 core::Device Struct Reference . . . 30
7.1.1 Detailed Description . . . 31
7.1.2 Constructor & Destructor Documentation . . . 31
7.1.3 Member Data Documentation . . . 31

7.2 metrics::ExactMetrics Struct Reference . . . 33
7.2.1 Detailed Description . . . 34
7.2.2 Constructor & Destructor Documentation . . . 34
7.2.3 Member Function Documentation . . . 35
7.2.4 Member Data Documentation . . . 36
7.3 metrics::HeuristicMetrics Struct Reference . . . 37
7.3.1 Detailed Description . . . 39
7.3.2 Constructor & Destructor Documentation . . . 39
7.3.3 Member Function Documentation . . . 40
7.3.4 Member Data Documentation . . . 41
7.4 metrics::Metrics::Inputs Struct Reference . . . 42
7.4.1 Detailed Description . . . 42
7.4.2 Member Function Documentation . . . 42
7.4.3 Member Data Documentation . . . 43
7.5 metrics::MetaHeuristicMetrics Struct Reference . . . 43
7.5.1 Detailed Description . . . 45
7.5.2 Constructor & Destructor Documentation . . . 45
7.5.3 Member Function Documentation . . . 46
7.5.4 Member Data Documentation . . . 47
7.6 metrics::MetricEntry Struct Reference . . . 48
7.6.1 Detailed Description . . . 49
7.6.2 Member Data Documentation . . . 49
7.7 metrics::Metrics Struct Reference . . . 50
7.7.1 Detailed Description . . . 51
7.7.2 Constructor & Destructor Documentation . . . 51
7.7.3 Member Function Documentation . . . 53
7.7.4 Member Data Documentation . . . 54
7.8 metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move Struct Reference . . . 55
7.8.1 Detailed Description . . . 55
7.8.2 Member Data Documentation . . . 55
7.9 metrics::Metrics::Outputs Struct Reference . . . 56
7.9.1 Detailed Description . . . 57
7.9.2 Member Function Documentation . . . 57
7.9.3 Member Data Documentation . . . 57
7.10 metrics::Result Struct Reference . . . 58
7.10.1 Detailed Description . . . 59
7.10.2 Constructor & Destructor Documentation . . . 59
7.10.3 Member Function Documentation . . . 60
7.10.4 Member Data Documentation . . . 60
7.11 core::Server Struct Reference . . . 60
7.11.1 Detailed Description . . . 62
7.11.2 Constructor & Destructor Documentation . . . 62

7.11.3 Member Function Documentation . . . 62
7.11.4 Member Data Documentation . . . 63
7.12 core::Supply Struct Reference . . . 64
7.12.1 Detailed Description . . . 65
7.12.2 Constructor & Destructor Documentation . . . 66
7.12.3 Member Data Documentation . . . 66

8 File Documentation **67**
8.1 include/exact.hpp File Reference . . . 67
8.1.1 Detailed Description . . . 68
8.2 exact.hpp . . . 68
8.3 include/heuristic.hpp File Reference . . . 70
8.3.1 Detailed Description . . . 71
8.4 heuristic.hpp . . . 71
8.5 include/metaheuristic.hpp File Reference . . . 73
8.5.1 Detailed Description . . . 75
8.6 metaheuristic.hpp . . . 75
8.7 include/mmkp.hpp File Reference . . . 78
8.7.1 Detailed Description . . . 81
8.7.2 Typedef Documentation . . . 81
8.8 mmkp.hpp . . . 82
8.9 include/tools.hpp File Reference . . . 98
8.9.1 Detailed Description . . . 100
8.9.2 Variable Documentation . . . 100
8.10 tools.hpp . . . 101
8.11 README.md File Reference . . . 106
8.12 src/main.cpp File Reference . . . 106
8.12.1 Detailed Description . . . 107
8.12.2 Function Documentation . . . 107
8.12.3 Variable Documentation . . . 108
8.13 main.cpp . . . 109

Index **113**

1 MMKP Resource Allocation in IoT-EC-CC Networks

Language / Idioma: English | Português (Brasil)

A C++17 simulator for solving the **Multiple Multidimensional Knapsack Problem (MMKP)** applied to resource allocation across **Internet of Things (IoT)**, **Edge Computing (EC)**, and **Cloud Computing (CC)** infrastructures. The simulator evaluates Exact (ILP via IBM ILOG CPLEX), Constructive Heuristic, and Meta-heuristic (Simulated Annealing) paradigms.

1.1 Project Structure

```
MMKP_IoT_EC_CC/
├── .github/workflows/
│   └── docs.yml            # GitHub Actions workflow for Doxygen (HTML + PDF)
├── data/                   # Input datasets (auto-generated if absent)
│   ├── devices/            # IoT device demands (Devices_<N>.txt)
│   ├── servers/            # EC and CC server capacities (servers_<EC>_<CC>.txt)
│   └── Services.txt        # Service templates (COST, CPU, MEM, STO, TSK)
├── docs/
│   ├── html/               # Generated Doxygen HTML documentation (git-ignored)
│   ├── latex/              # Generated Doxygen LaTeX/PDF documentation (git-ignored)
│   └── Doxyfile            # Doxygen documentation configuration
├── include/                # Header-only C++17 implementation
│   ├── exact.hpp           # CPLEX ILP mathematical formulation
│   ├── heuristic.hpp       # Constructive heuristics (GR, GRF, GCB)
│   ├── metaheuristic.hpp   # Simulated Annealing and neighborhood operators
│   ├── mmkp.hpp            # Core data structures, I/O, and metrics
│   └── tools.hpp           # CSV handler, RNG, formatting, and sorting utilities
├── results/                # Simulation outputs (CSV tables and ASCII summaries)
│   ├── EXACT/
│   ├── HEURISTIC/
│   └── METAHEURISTIC/
├── src/
│   └── main.cpp            # Benchmark entry point and simulation parameters
├── build.sh                # Automated build and execution script
├── CMakeLists.txt          # CMake build configuration
├── README.md               # Project documentation (English)
└── README.pt-BR.md         # Project documentation (Portuguese)
```

1.2 Prerequisites

- **OS:** Linux or Windows Subsystem for Linux (WSL2 - Ubuntu 24.04 recommended)
- **Compiler:** `g++` with C++17 support
- **Build System:** Bash (`build.sh`) or `CMake` (>= 3.10)
- **Solver:** IBM ILOG CPLEX Optimization Studio 22.2 installed at: `/opt/ibm/ILOG/CPLEX_Studio222`

  Note: If your CPLEX installation is in a different directory, update the `CPLEX_INCLUDE`, `CONCERT_INCLUDE`, `CPLEX_LIB`, and `CONCERT_LIB` paths in `build.sh` and `CMakeLists.txt`.

1.3 How to Compile and Run

1.3.1 Option 1: Using <tt>build.sh</tt> (Recommended)

Grant execution permission and run the script from the project root:

```
chmod +x build.sh
./build.sh
```

The script supports optional flags before compiling and running:

- `./build.sh -b` or `--build`: Cleans the `build/` directory.
- `./build.sh -d` or `--data`: Deletes existing datasets in `data/` (forces regeneration).
- `./build.sh -r` or `--results`: Clears previous outputs in `results/`.
- `./build.sh -a` or `--all`: Cleans `build/`, `data/`, and `results/`.
- `./build.sh -doc` or `--docs`: Regenerates Doxygen HTML and LaTeX PDF documentation.

1.3.2 Option 2: Using CMake

```
mkdir -p build && cd build
cmake ..
make
cd ..
./build/MMKP_IoT-EC-CC
```

∗(Always run the binary from the project root directory so relative paths to `data/` and `results/` resolve properly).∗

1.4 Implemented Algorithms

Option (main.cpp)	Paradigm	Algorithm	Description
0	EXACT	SOLVER_MAXIMUM_COST	Exact ILP solved with CPLEX maximizing avoided SLA penalties.
-1	HEURISTIC	GREEDY_CB	Deterministic greedy allocation sorted by surrogate cost-benefit utility.
-2	HEURISTIC	GREEDY_RANDOM	Stochastic baseline assigning each device to a random server.
-3	HEURISTIC	GREEDY_RANDOM_↩ FALLBACK	Two-stage stochastic allocation prioritizing Edge (EC) with Cloud (CC) fallback.
1	METAHEURISTIC	SA + GREEDY_CB	Simulated Annealing initialized with GREEDY_CB.
2	METAHEURISTIC	SA + GREEDY_RANDOM	Simulated Annealing initialized with GREEDY_RANDOM.
3	METAHEURISTIC	SA + GREEDY_RANDOM_↩ FALLBACK	Simulated Annealing initialized with GREEDY_↩ RANDOM_FALLBACK.

Note: When executing options 1, 2, and 3, the initial heuristic solution is automatically evaluated and saved under results/HEURISTIC/ prior to running Simulated Annealing.

1.5 Simulation Parameters

Simulation constants can be customized directly in src/main.cpp:

- SERVICES = 100: Number of service profiles.
- DEVICES_START = 10000, DEVICES_STEP = 1000, DEVICES_END = 20000: Device scaling range.
- EC_LENGTH = 499, CC_LENGTH = 1: Number of Edge and Cloud servers.
- MAX_RUNS = 50: Number of independent runs per instance.
- CPLEX_TIME_LIMIT = 3600: Maximum solver runtime in seconds (1 hour).
- TEMPERATURE = 50.0, ALPHA = 0.9999: Simulated Annealing cooling schedule.
- FIXED_PENALTY = false: Controls whether device SLA penalty costs are constant (1.0) or task-proportional.

1.6 Simulation Results (<tt>results/</tt>)

In the output files stored inside the results/ directory (EXACT/, HEURISTIC/, and METAHEURISTIC/):

- **First batch of execution rows (10,000 to 20,000 devices):** Executed using a **constant penalty cost (1.0)** for all devices (FIXED_PENALTY = true).
- **Subsequent execution rows (10,000 to 20,000 devices):** Executed using a **variable penalty cost** (FIXED_PENALTY = false), where each device's penalty cost is determined by the random draw of tasks generated for its assigned service profile in data/Services.txt.

1.7 Documentation (Doxygen)

To generate or update the HTML (docs/html/index.html) and PDF (docs/latex/refman.pdf) documentation locally, run from the project root:

```
./build.sh --docs
```

2 Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

convert
Short namespace alias for `file_manager::csv` 5

core
Contains the fundamental domain entities for the MMKP resource allocation problem 8

data 8

data::generate
Generates or updates synthetic datasets for services, IoT devices, and EC/CC servers 8

data::load
Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors 10

display
Utility namespace for formatting and printing entities and ASCII metric tables to console or files 11

display::details
Internal helper routines for rendering ASCII box-drawing tables 14

exact
Contains mathematical programming models for exact optimization 15

exact::anonymous_namespace{exact.hpp} 16

file_manager 17

file_manager::csv
Provides file I/O operations for reading, writing, and appending delimited CSV/text files 17

heuristic
Contains constructive heuristic algorithms for network resource allocation 19

heuristic::anonymous_namespace{heuristic.hpp} 19

metaheuristic
Encapsulates meta-heuristic algorithms for network resource allocation 20

metaheuristic::anonymous_namespace{metaheuristic.hpp} 21

metrics
Provides structures and routines for tracking, formatting, and persisting simulation metrics 23

randomness
Mersenne Twister (`std::mt19937`) random number generation and permutation utilities 26

sorting
Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes 28

sorting::details
Internal implementation helpers for pointer-to-member index sorting 29

3 Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

core::Device 30

metrics::Metrics::Inputs 42

metrics::MetricEntry 48

metrics::Metrics 50

- **metrics::ExactMetrics** 33
- **metrics::HeuristicMetrics** 37
- **metrics::MetaHeuristicMetrics** 43

metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move 55

metrics::Metrics::Outputs 56

metrics::Result 58

core::Server 60

core::Supply 64

4 Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

core::Device
Represents an IoT device requesting computational resources for a specific service 30

metrics::ExactMetrics
Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap 33

metrics::HeuristicMetrics
Specialization of `Metrics` for constructive heuristic runs 37

metrics::Metrics::Inputs
Captures the static input instance dimensions and maximum possible penalty cost 42

metrics::MetaHeuristicMetrics
Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection 43

metrics::MetricEntry
Represents a formatted key-value entry for displaying or exporting simulation metrics 48

metrics::Metrics
Polymorphic base structure for tracking, calculating, and saving simulation metrics 50

metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move
Represents a reversible state transition within the neighborhood search 55

metrics::Metrics::Outputs
Captures the resulting performance and resource utilization metrics after allocation 56

metrics::Result
Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run 58

core::Server
Represents an Edge Computing (EC) or Cloud Computing (CC) server node 60

core::Supply
Tracks the cumulative resource consumption and allocated device IDs on a single server 64

5 File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/exact.hpp
Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX 67

include/heuristic.hpp
Constructive heuristic algorithms for fast MMKP resource allocation 70

include/metaheuristic.hpp
Meta-heuristic optimization algorithms and neighborhood operators for MMKP 73

include/mmkp.hpp
Core data structures, synthetic dataset generators, metric trackers, and console/file display routines 78

include/tools.hpp
General-purpose utilities for CSV I/O, string formatting, random number generation, and indirect sorting 98

src/main.cpp
Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator 106

6 Namespace Documentation

6.1 convert Namespace Reference

Short namespace alias for `file_manager::csv`.

Functions

- template<typename T >
  std::string to_string (const T &value, int precision=DECIMAL_PLACES)
  Converts a value of any streamable type to its string representation.
- std::string to_percentage_string (double numerator, double denominator, int precision=DECIMAL_PLACES)
  Calculates a percentage and converts it to a formatted string.
- template<typename T >
  std::string vector_to_string (const std::vector< T > &vec)
  Converts a vector of primitive types to a bracketed array string representation.
- template<typename T >
  std::string set_to_string (const std::set< T > &s)
  Converts a `std::set` of primitive types to a bracketed array string representation.

6.1.1 Detailed Description

Short namespace alias for `file_manager::csv`.
String conversion and formatting utilities for numbers, percentages, vectors, and sets.

6.1.2 Function Documentation

set_to_string()

```
template<typename T >
std::string convert::set_to_string (
            const std::set< T > & s )  [inline]
```

Converts a `std::set` of primitive types to a bracketed array string representation.

Template Parameters

T	The type of elements in the set.

Parameters

s	The set to convert.

Returns

A string formatted as "`[e1,e2,...]`".

Definition at line 243 of file tools.hpp.
Referenced by display::show_server().

to_percentage_string()

```
std::string convert::to_percentage_string (
            double numerator,
            double denominator,
            int precision = DECIMAL_PLACES )  [inline]
```

Calculates a percentage and converts it to a formatted string.
Computes `(numerator / denominator) * 100.0` and formats it to `precision` decimal places. Safely returns `0.0` when `denominator == 0.0`.

Parameters

numerator	The value representing the part of the total.
denominator	The value representing the total.
precision	The number of decimal places for the resulting percentage string.

Returns

A string representation of the calculated percentage.

Definition at line 210 of file tools.hpp.
References to_string().
Referenced by metrics::Metrics::build_metrics(), metrics::ExactMetrics::build_metrics(), and display::show_servers_usage().

to_string()

```
template<typename T >
std::string convert::to_string (
            const T & value,
            int precision = DECIMAL_PLACES )  [inline]
```

Converts a value of any streamable type to its string representation.
Uses `std::ostringstream` for conversion, applying fixed-point notation with the specified precision for floating-point types.

Template Parameters

T	The data type of the value to be converted.

Parameters

value	The value to convert.
precision	The number of decimal places for floating-point types.

Returns

A formatted string representation of `value`.

Definition at line 193 of file tools.hpp.
Referenced by metrics::Metrics::build_metrics(), metrics::ExactMetrics::build_metrics(), metrics::MetaHeuristicMetrics::build_metrics(), metrics::Metrics::data(), metrics::ExactMetrics::data(), metrics::MetaHeuristicMetrics::data(), data::generate::devices(), data::generate::servers(), data::generate::services(), and to_percentage_string().

vector_to_string()

```
template<typename T >
std::string convert::vector_to_string (
            const std::vector< T > & vec )  [inline]
```

Converts a vector of primitive types to a bracketed array string representation.

Template Parameters

T	The type of elements in the vector.

Parameters

vec	The vector to convert.

Returns

A string formatted as `"[e1,e2,...]"`.

Definition at line 225 of file tools.hpp.

6.2 core Namespace Reference

Contains the fundamental domain entities for the MMKP resource allocation problem.

Classes

- struct Device

  Represents an IoT device requesting computational resources for a specific service.
- struct Server

  Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
- struct Supply

  Tracks the cumulative resource consumption and allocated device IDs on a single server.

Typedefs

- using Devices = std::vector< Device >

  Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).
- using Servers = std::vector< Server >

  Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).

6.2.1 Detailed Description

Contains the fundamental domain entities for the MMKP resource allocation problem.

6.2.2 Typedef Documentation

Devices

```
using core::Devices = typedef std::vector<Device>
```

Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).
Definition at line 67 of file mmkp.hpp.

Servers

```
using core::Servers = typedef std::vector<Server>
```

Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).
Definition at line 181 of file mmkp.hpp.

6.3 data Namespace Reference

Namespaces

- namespace generate

  Generates or updates synthetic datasets for services, IoT devices, and EC/CC servers.
- namespace load

  Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.

6.4 data::generate Namespace Reference

Generates or updates synthetic datasets for services, IoT devices, and EC/CC servers.

Functions

- std::vector< std::vector< std::string > > services (const int num_services=10, bool fixed_penalty=true)

  Generates or loads network service requirement templates.
- std::vector< std::vector< std::string > > devices (const int length, const int num_services=10, bool fixed_↩ penalty=true)

  Generates or loads device request data representing IoT network demands.
- std::vector< std::vector< std::string > > servers (const int ec_length=99, const int cc_length=1)

  Generates or loads server infrastructure data for Edge and Cloud nodes.

Variables

- const std::filesystem::path DATA_PATH = std::filesystem::current_path() / "data"

  Path pointing to the root `data/` directory.

6.4.1 Detailed Description

Generates or updates synthetic datasets for services, IoT devices, and EC/CC servers.

6.4.2 Function Documentation

devices()

```
std::vector< std::vector< std::string > > data::generate::devices (
            const int length,
            const int num_services = 10,
            bool fixed_penalty = true )  [inline]
```

Generates or loads device request data representing IoT network demands.
Uniformly samples service requirement profiles from the services dataset and assigns them to each device instance in `data/devices/Devices_<length>.txt`. If the file already exists, synchronizes each device's penalty cost (`COST`) with its corresponding service template.

Parameters

length	The total number of devices to generate.
num_services	The number of underlying service templates to sample from.
fixed_penalty	Whether the penalty (cost) of each device is fixed (`1.0`) or variable.

Returns

A 2D string vector representing the devices dataset.

Definition at line 267 of file mmkp.hpp.
References DATA_PATH, devices(), randomness::random_number(), file_manager::csv::read(), services(), convert::to_string(), and file_manager::csv::write().
Referenced by devices(), and data::load::devices().

servers()

```
std::vector< std::vector< std::string > > data::generate::servers (
            const int ec_length = 99,
            const int cc_length = 1 )  [inline]
```

Generates or loads server infrastructure data for Edge and Cloud nodes.
Creates capacity profiles for `ec_length` Edge servers ('E') and`cc_length` Cloud servers ('C'), saving the table to`data/servers/servers_<ec>_<cc>.txt`.

Parameters

ec_length	The number of Edge Computing servers to generate.
cc_length	The number of Cloud Computing servers to generate.

Returns

A 2D string vector representing the servers dataset.

Definition at line 319 of file mmkp.hpp.
References DATA_PATH, DECIMAL_PLACES, randomness::random_number(), file_manager::csv::read(), servers(), convert::to_string(), and file_manager::csv::write().
Referenced by data::load::servers(), and servers().

services()

```
std::vector< std::vector< std::string > > data::generate::services (
            const int num_services = 10,
            bool fixed_penalty = true )  [inline]
```

Generates or loads network service requirement templates.
Creates random service profiles with varying task counts, memory, storage, and processing demands, persisting them to `data/Services.txt`. If the file already exists, reloads the profiles and updates their penalty costs (`COST`) according to `fixed_penalty`.

Parameters

num_services	The total number of distinct service profiles to generate.
fixed_penalty	If `true`, assigns a unit penalty cost (`1.0`); if `false`, generates task-proportional random penalty costs.

Returns

A 2D string vector containing the generated or updated services dataset.

Definition at line 205 of file mmkp.hpp.
References DATA_PATH, DECIMAL_PLACES, randomness::random_number(), file_manager::csv::read(), services(), convert::to_string(), file_manager::csv::write(), and ZERO.
Referenced by devices(), and services().

6.4.3 Variable Documentation

DATA_PATH

```
const std::filesystem::path data::generate::DATA_PATH = std::filesystem::current_path() /
"data"
```

Path pointing to the root `data/` directory.
Definition at line 193 of file mmkp.hpp.
Referenced by devices(), servers(), and services().

6.5 data::load Namespace Reference

Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.

Functions

- core::Devices devices (const int length, const int num_services=10, bool fixed_penalty=true)

  Loads device data from disk (generating it if absent) into a vector of `core::Device` objects.
- core::Servers servers (const int ec=99, const int cc=1)

  Loads Edge and Cloud server data from disk (generating it if absent).

6.5.1 Detailed Description

Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.

6.5.2 Function Documentation

devices()

```
core::Devices data::load::devices (
            const int length,
            const int num_services = 10,
            bool fixed_penalty = true )  [inline]
```

Loads device data from disk (generating it if absent) into a vector of `core::Device` objects.

Parameters

length	The total number of devices to load.
num_services	The number of distinct services to generate if absent. Defaults to 10.
fixed_penalty	Whether the penalty (cost) of each device is fixed or can vary. Defaults to true.

Returns

A 1-indexed `core::Devices` vector containing the parsed device instances.

Definition at line 371 of file mmkp.hpp.
References data::generate::devices(), and devices().
Referenced by devices().

servers()

```
core::Servers data::load::servers (
            const int ec = 99,
            const int cc = 1 )  [inline]
```

Loads Edge and Cloud server data from disk (generating it if absent).

Parameters

ec	The number of Edge Computing (EC) servers to load.
cc	The number of Cloud Computing (CC) servers to load.

Returns

A 1-indexed `core::Servers` vector containing both EC and CC server instances.

Definition at line 409 of file mmkp.hpp.
References servers(), and data::generate::servers().
Referenced by servers().

6.6 display Namespace Reference

Utility namespace for formatting and printing entities and ASCII metric tables to console or files.

Namespaces

- namespace details

  Internal helper routines for rendering ASCII box-drawing tables.

Functions

- void show_device (const core::Device &device)

  Displays detailed information of a single device to standard output.
- void show_devices (const core::Devices &devices)

  Displays detailed information for all devices in the vector.
- void show_server (const core::Server &server)

  Displays capacity and current demand of a single server.
- void show_servers (const core::Servers &servers)

  Displays capacity and demand information for all servers.
- void show_metrics_console_or_file (const metrics::Metrics &metrics, std::ostream &out)

  Formats and writes the complete simulation metrics table to an output stream.
- void show_metrics (const std::shared_ptr< metrics::Metrics > &ptr)

  Displays the metrics table on the console and appends it to a drawing file.
- void show_servers_usage (const core::Servers &servers)

  Prints a summary table of server resource utilization percentages.

6.6.1 Detailed Description

Utility namespace for formatting and printing entities and ASCII metric tables to console or files.

6.6.2 Function Documentation

show_device()

```
void display::show_device (
            const core::Device & device )  [inline]
```

Displays detailed information of a single device to standard output.

Parameters

device	The device instance to display.

Definition at line 1061 of file mmkp.hpp.
References core::Device::cost, core::Device::cpu, core::Device::id, core::Device::mem, core::Device::served, core::Device::server_id, core::Device::service, core::Device::sto, and core::Device::tasks.
Referenced by show_devices().

show_devices()

```
void display::show_devices (
            const core::Devices & devices )  [inline]
```

Displays detailed information for all devices in the vector.

Parameters

devices	The vector of devices to display.

Definition at line 1083 of file mmkp.hpp.
References show_device().

show_metrics()

```
void display::show_metrics (
            const std::shared_ptr< metrics::Metrics > & ptr )  [inline]
```

Displays the metrics table on the console and appends it to a drawing file.

Parameters

ptr	Shared pointer to the simulation metrics instance.

Definition at line 1217 of file mmkp.hpp.
References show_metrics_console_or_file().
Referenced by metrics::save_results().

show_metrics_console_or_file()

```
void display::show_metrics_console_or_file (
            const metrics::Metrics & metrics,
            std::ostream & out )  [inline]
```

Formats and writes the complete simulation metrics table to an output stream.

Parameters

metrics	The metrics instance to format.
out	The target output stream.

Definition at line 1184 of file mmkp.hpp.
References display::details::print_header(), display::details::print_middle(), display::details::print_row(), and

display::details::print_title().
Referenced by show_metrics().

show_server()

```
void display::show_server (
            const core::Server & server )  [inline]
```

Displays capacity and current demand of a single server.

Parameters

server	The server instance to display.

Definition at line 1095 of file mmkp.hpp.
References core::Supply::cpu, core::Server::cpu, core::Supply::devices, core::Server::id, core::Supply::mem, core::Server::mem, convert::set_to_string(), core::Supply::sto, core::Server::sto, core::Server::supply, and core::Server::type.
Referenced by show_servers().

show_servers()

```
void display::show_servers (
            const core::Servers & servers )  [inline]
```

Displays capacity and demand information for all servers.

Parameters

servers	The vector of servers to display.

Definition at line 1120 of file mmkp.hpp.
References show_server().

show_servers_usage()

```
void display::show_servers_usage (
            const core::Servers & servers )  [inline]
```

Prints a summary table of server resource utilization percentages.

Parameters

servers	The vector of servers to summarize.

Definition at line 1242 of file mmkp.hpp.
References convert::to_percentage_string().
Referenced by metrics::save_results().

6.7 display::details Namespace Reference

Internal helper routines for rendering ASCII box-drawing tables.

Functions

- void print_row (std::ostream &out, const std::string &label, const std::string &value)

  Prints a formatted key-value row within the ASCII table.
- void print_header (std::ostream &out)

  Prints the top or bottom border of the ASCII table.
- void print_middle (std::ostream &out)

Prints a section separator line within the ASCII table.

- void print_title (std::ostream &out, const std::string &title)

  Prints a centered title row within the ASCII table.

Variables

- const int total_width = 72

  Total character width of the formatted ASCII table.

- const int label_width = 26

  Character width allocated to the left-hand label column.

6.7.1 Detailed Description

Internal helper routines for rendering ASCII box-drawing tables.

6.7.2 Function Documentation

print_header()

```
void display::details::print_header (
            std::ostream & out )  [inline]
```

Prints the top or bottom border of the ASCII table.

Parameters

out	The target output stream.

Definition at line 1152 of file mmkp.hpp.
References total_width.
Referenced by display::show_metrics_console_or_file().

print_middle()

```
void display::details::print_middle (
            std::ostream & out )  [inline]
```

Prints a section separator line within the ASCII table.

Parameters

out	The target output stream.

Definition at line 1160 of file mmkp.hpp.
References label_width, and total_width.
Referenced by display::show_metrics_console_or_file().

print_row()

```
void display::details::print_row (
            std::ostream & out,
            const std::string & label,
            const std::string & value )  [inline]
```

Prints a formatted key-value row within the ASCII table.

Parameters

out	The target output stream.
label	The row label string.
value	The row value string.

Definition at line 1142 of file mmkp.hpp.
References label_width, and total_width.
Referenced by display::show_metrics_console_or_file().

print_title()

```
void display::details::print_title (
            std::ostream & out,
            const std::string & title )  [inline]
```

Prints a centered title row within the ASCII table.

Parameters

out	The target output stream.
title	The title text to center.

Definition at line 1170 of file mmkp.hpp.
References total_width.
Referenced by display::show_metrics_console_or_file().

6.7.3 Variable Documentation

label_width

```
const int display::details::label_width = 26
```

Character width allocated to the left-hand label column.
Definition at line 1134 of file mmkp.hpp.
Referenced by print_middle(), and print_row().

total_width

```
const int display::details::total_width = 72
```

Total character width of the formatted ASCII table.
Definition at line 1133 of file mmkp.hpp.
Referenced by print_header(), print_middle(), print_row(), and print_title().

6.8 exact Namespace Reference

Contains mathematical programming models for exact optimization.

Namespaces

- namespace anonymous_namespace{exact.hpp}

Typedefs

- using NumVar2D = IloArray< IloNumVarArray >

  Type alias for a 2D matrix of CPLEX decision variables.

Functions

- void boot (metrics::Result &state, int time_limit_sec=3600)

  Bootstraps and executes the configured exact mathematical model.

6.8.1 Detailed Description

Contains mathematical programming models for exact optimization.
Utilizes the IBM ILOG CPLEX optimizer to solve the network resource allocation problem to optimality or within a specified time limit, providing an exact baseline by maximizing avoided non-service penalties.

6.8.2 Typedef Documentation

NumVar2D

```
using exact::NumVar2D = typedef IloArray<IloNumVarArray>
```

Type alias for a 2D matrix of CPLEX decision variables.
Definition at line 27 of file exact.hpp.

6.8.3 Function Documentation

boot()

```
void exact::boot (
            metrics::Result & state,
            int time_limit_sec = 3600 )  [inline]
```

Bootstraps and executes the configured exact mathematical model.
Reads the specified algorithm identifier from `state.stats` and dispatches execution to the corresponding exact solver implementation.

Parameters

state	The unified simulation state to be processed.
time_limit_sec	The maximum execution time allowed for the exact solver in seconds.

Definition at line 172 of file exact.hpp.
References metrics::solver_maximum_cost, and metrics::Result::stats.
Referenced by run_simulation().

6.9 exact::anonymous_namespace{exact.hpp} Namespace Reference

Functions

- void solver_maximum_cost (metrics::Result &state, int time_limit_sec=3600)

  Formulates and solves the exact ILP resource allocation model using CPLEX.

6.9.1 Function Documentation

solver_maximum_cost()

```
void exact::anonymous_namespace{exact.hpp}::solver_maximum_cost (
            metrics::Result & state,
            int time_limit_sec = 3600 )  [inline]
```

Formulates and solves the exact ILP resource allocation model using CPLEX.
Defines binary decision variables $x_d^i \in \{0, 1\}$ indicating whether device d is assigned to server i. Maximizes the total utility of served devices (avoided SLA penalties) subject to single-server assignment and multidimensional resource capacity constraints (CPU, MEM, STO).

Parameters

state	The unified simulation state containing devices, servers, and metrics.
time_limit_sec	The maximum execution time allowed for the solver in seconds.

Definition at line 41 of file exact.hpp.
References metrics::Result::devices, metrics::Result::servers, solver_maximum_cost(), and metrics::Result::stats.
Referenced by solver_maximum_cost().

6.10 file_manager Namespace Reference

Namespaces

- namespace csv

  Provides file I/O operations for reading, writing, and appending delimited CSV/text files.

6.11 file_manager::csv Namespace Reference

Provides file I/O operations for reading, writing, and appending delimited CSV/text files.

Functions

- std::optional< std::vector< std::vector< std::string > > > read (const std::string &file_path, char delimiter=',')

  Reads a delimited CSV file into a 2D vector of strings.
- bool write (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Writes a 2D vector of strings to a CSV file, creating parent directories if needed.
- bool append (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Appends a 2D vector of strings to an existing or new CSV file.
- int count_lines (const std::string &file_path)

  Counts the total number of lines in a file.

6.11.1 Detailed Description

Provides file I/O operations for reading, writing, and appending delimited CSV/text files.

6.11.2 Function Documentation

append()

```
bool file_manager::csv::append (
            const std::string & file_path,
            const std::vector< std::vector< std::string > > & data,
            char delimiter = ',' )  [inline]
```

Appends a 2D vector of strings to an existing or new CSV file.

Parameters

file_path	The full path to the output file.
data	The 2D vector of strings to append.
delimiter	The character used for separating columns (defaults to ','). @returntrueif the data was appended successfully;false` otherwise.

Definition at line 118 of file tools.hpp.
Referenced by metrics::Metrics::save_to_file().

count_lines()

```
int file_manager::csv::count_lines (
            const std::string & file_path )  [inline]
```

Counts the total number of lines in a file.

Parameters

file_path	The full path to the file.

Returns

The total number of lines, or `0` if the file does not exist or cannot be opened.

Definition at line 153 of file tools.hpp.

read()

```
std::optional< std::vector< std::vector< std::string > > > file_manager::csv::read (
            const std::string & file_path,
            char delimiter = ',' )  [inline]
```

Reads a delimited CSV file into a 2D vector of strings.

Parameters

file_path	The full path to the file to be read.
delimiter	The character that separates columns (defaults to ‘,') . @return Anstd::optionalcontaining the 2D vector of strings upon success, orstd::nullopt` on failure.

Definition at line 52 of file tools.hpp.
Referenced by data::generate::devices(), data::generate::servers(), and data::generate::services().

write()

```
bool file_manager::csv::write (
            const std::string & file_path,
            const std::vector< std::vector< std::string > > & data,
            char delimiter = ',' )  [inline]
```

Writes a 2D vector of strings to a CSV file, creating parent directories if needed.

Parameters

file_path	The full path to the output file.
data	The 2D vector of strings to write.
delimiter	The character used for separating columns (defaults to ‘,') . @returntrueif the file was written successfully;false` otherwise.

Definition at line 85 of file tools.hpp.
Referenced by data::generate::devices(), data::generate::servers(), and data::generate::services().

6.12 heuristic Namespace Reference

Contains constructive heuristic algorithms for network resource allocation.

Namespaces

- namespace anonymous_namespace{heuristic.hpp}

Functions

- void boot (metrics::Result &state)

  Bootstraps and executes the configured constructive heuristic.

6.12.1 Detailed Description

Contains constructive heuristic algorithms for network resource allocation.
Implements fast, polynomial-time approximation strategies (Greedy Cost-Benefit, Greedy Random, and Greedy Random Fallback) prioritizing low execution times while minimizing unserved device penalty costs.

6.12.2 Function Documentation

boot()

```
void heuristic::boot (
            metrics::Result & state )  [inline]
```

Bootstraps and executes the configured constructive heuristic.
Reads the specified algorithm identifier from `state.stats` and dispatches execution to the corresponding heuristic solver.

Parameters

state	The unified simulation state to be processed.

Definition at line 196 of file heuristic.hpp.
References metrics::greedy_cost_benefit, metrics::greedy_random, metrics::greedy_random_fallback, and metrics::Result::stats.
Referenced by metaheuristic::boot(), and run_simulation().

6.13 heuristic::anonymous_namespace{heuristic.hpp} Namespace Reference

Functions

- void greedy_cost_benefit (metrics::Result &state)

  Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.
- void greedy_random (metrics::Result &state)

  Executes a fast random baseline resource allocation.
- void greedy_random_fallback (metrics::Result &state)

  Executes a two-stage random fallback resource allocation.

6.13.1 Function Documentation

greedy_cost_benefit()

```
void heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit (
            metrics::Result & state )  [inline]
```

Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.
Normalizes device resource demands against total system capacity to compute a multidimensional cost-benefit pseudo-utility and sorts candidates once globally. Allocates devices greedily per server using prefix-skipping and minimum-demand early termination.

Parameters

state	The unified simulation result state containing devices, servers, and metrics.

Definition at line 32 of file heuristic.hpp.
References core::Server::add_service(), core::Server::can_serve(), core::Device::cost, core::Device::cpu, core::Supply::cpu, core::Server::cpu, metrics::Result::devices, greedy_cost_benefit(), core::Device::id, core::Device::mem, core::Supply::mem, core::Server::mem, core::Device::served, metrics::Result::servers, metrics::Result::stats, core::Device::sto, core::Supply::sto, core::Server::sto, and core::Server::supply.
Referenced by greedy_cost_benefit().

greedy_random()

```
void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random (
            metrics::Result & state )  [inline]
```

Executes a fast random baseline resource allocation.
Iterates through all devices and attempts to assign each one to a single uniformly selected server. If the selected server lacks capacity, the device remains unserved. Metric calculation is deferred.

Parameters

state	The unified simulation result state containing devices, servers, and metrics.

Definition at line 114 of file heuristic.hpp.
References metrics::Result::devices, greedy_random(), randomness::random_number(), metrics::Result::servers, and metrics::Result::stats.
Referenced by greedy_random().

greedy_random_fallback()

```
void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random_fallback (
            metrics::Result & state )  [inline]
```

Executes a two-stage random fallback resource allocation.
Attempts to assign each device to a randomly selected Edge (EC) server first. If the selected Edge server lacks capacity, falls back to a randomly selected Cloud (CC) server.

Parameters

state	The unified simulation result state containing devices, servers, and metrics.

Definition at line 150 of file heuristic.hpp.
References metrics::Result::devices, greedy_random_fallback(), randomness::random_number(), metrics::Result::servers, and metrics::Result::stats.
Referenced by greedy_random_fallback().

6.14 metaheuristic Namespace Reference

Encapsulates meta-heuristic algorithms for network resource allocation.

Namespaces

- namespace anonymous_namespace{metaheuristic.hpp}

Functions

- void boot (metrics::Result &state)

  Initializes and launches the selected meta-heuristic algorithm.

6.14.1 Detailed Description

Encapsulates meta-heuristic algorithms for network resource allocation.
Provides trajectory-based optimization techniques, specifically Simulated Annealing, to iteratively improve initial heuristic solutions by exploring shift, insertion, and 1-for-1 swap neighborhood structures.

6.14.2 Function Documentation

boot()

```
void metaheuristic::boot (
            metrics::Result & state )  [inline]
```

Initializes and launches the selected meta-heuristic algorithm.
Generates an initial solution using the configured constructive base heuristic, records its standalone metrics in the heuristic results directory, and then executes the meta-heuristic optimization over the resulting state.

Parameters

state	The unified simulation state to be optimized.

Definition at line 221 of file metaheuristic.hpp.
References heuristic::boot(), metrics::calculate_metrics(), metrics::Result::devices, metrics::greedy_cost_benefit, metrics::save_results(), metrics::Result::servers, metrics::simulated_annealing, and metrics::Result::stats.
Referenced by run_simulation().

6.15 metaheuristic::anonymous_namespace{metaheuristic.hpp} Namespace Reference

Classes

- struct Move

  Represents a reversible state transition within the neighborhood search.

Functions

- Move apply_neighbor (core::Devices &devices, core::Servers &servers, int tries=10)

  Applies a shift, insert, or swap neighborhood move to the current state.
- void revert_neighbor (core::Devices &devices, core::Servers &servers, const Move &move)

  Reverts a previously applied neighborhood move in $O(\log N)$ time.
- void simulated_annealing (metrics::Result &state)

  Executes the Simulated Annealing (SA) optimization algorithm.

Variables

- constexpr int LOOP = 10

  Number of inner-loop iterations (Markov chain length) executed per temperature level.

6.15.1 Function Documentation

apply_neighbor()

```
Move metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor (
            core::Devices & devices,
            core::Servers & servers,
            int tries = 10 )  [inline]
```

Applies a shift, insert, or swap neighborhood move to the current state.
Selects a random device and attempts to allocate or shift it across random servers (including a fallback check on the last server for unserved devices). If direct allocation fails for an unserved device, attempts a 1-for-1 swap with a currently served device.

Parameters

devices	The global state vector of devices.
servers	The global state vector of servers.
tries	The maximum number of random candidate attempts.

Returns

Move A structure containing the applied transition details and cost variation.

Definition at line 54 of file metaheuristic.hpp.
References core::Server::add_service(), apply_neighbor(), core::Server::can_serve(), core::Device::cost, metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::delta_cost, metaheuristic::anonymous_namespace{metaheuristic. metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::new_server_id, metaheuristic::anonymous_namespace{metaheur randomness::random_number(), core::Server::rmv_service(), core::Device::served, core::Device::server_id, metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::swapped_id, and metaheuristic::anonymous_namespace{metaheu
Referenced by apply_neighbor(), and simulated_annealing().

revert_neighbor()

```
void metaheuristic::anonymous_namespace{metaheuristic.hpp}::revert_neighbor (
            core::Devices & devices,
            core::Servers & servers,
            const Move & move )  [inline]
```

Reverts a previously applied neighborhood move in $O(\log N)$ time.
Restores the exact server allocations and resource supplies prior to `apply_neighbor`.

Parameters

devices	The global state vector of devices.
servers	The global state vector of servers.
move	The `Move` structure containing the transition details to undo.

Definition at line 122 of file metaheuristic.hpp.
References metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::device_id, metaheuristic::anonymous_namespace{me
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::old_server_id, revert_neighbor(), and metaheuristic::anonymous_n
Referenced by revert_neighbor(), and simulated_annealing().

simulated_annealing()

```
void metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing (
            metrics::Result & state )  [inline]
```

Executes the Simulated Annealing (SA) optimization algorithm.
Iteratively explores the solution space by applying random neighborhood moves. Accepts worsening moves according to the Metropolis criterion $\exp(-\Delta/T)$ to escape local optima, cooling down geometrically via factor α until $T \leq 10^{-3}$. Uses lazy synchronization for tracking the best-found state.

Parameters

state	The unified simulation state containing devices, servers, and metrics.

Definition at line 145 of file metaheuristic.hpp.
References apply_neighbor(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::delta_cost,
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::device_id, metrics::Result::devices, LOOP,
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::new_server_id, metaheuristic::anonymous_namespace{metaheur
randomness::random_number(), revert_neighbor(), metrics::Result::servers, simulated_annealing(), metrics::Result::stats,
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::swapped_id, metaheuristic::anonymous_namespace{metaheuristi
and ZERO.
Referenced by simulated_annealing().

6.15.2 Variable Documentation

LOOP

```
constexpr int metaheuristic::anonymous_namespace{metaheuristic.hpp}::LOOP = 10  [constexpr]
```

Number of inner-loop iterations (Markov chain length) executed per temperature level.
Definition at line 28 of file metaheuristic.hpp.
Referenced by simulated_annealing().

6.16 metrics Namespace Reference

Provides structures and routines for tracking, formatting, and persisting simulation metrics.

Classes

- struct ExactMetrics

Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.

- struct HeuristicMetrics

  Specialization of `Metrics` for constructive heuristic runs.
- struct MetaHeuristicMetrics

  Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
- struct MetricEntry

  Represents a formatted key-value entry for displaying or exporting simulation metrics.
- struct Metrics

  Polymorphic base structure for tracking, calculating, and saving simulation metrics.
- struct Result

  Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.

Enumerations

- enum class Paradigm { NONE , EXACT , HEURISTIC , METAHEURISTIC }

  Enumerates the optimization paradigms supported by the simulator.
- enum class Algorithm {
  NONE , solver_maximum_cost , greedy_cost_benefit , greedy_random ,
  greedy_random_fallback , simulated_annealing }

  Identifies the specific optimization algorithm used in a simulation run.

Functions

- std::string to_string (Paradigm paradigm)

  Converts a `Paradigm` enum value to its uppercase string representation.
- std::string to_string (Algorithm algorithm)

  Converts an `Algorithm` enum value to its standardized string identifier.
- void calculate_metrics (Result &state)

  Computes the final simulation metrics based on the current device-to-server allocations.
- void save_results (const Result &state, bool show_servers_usage=false)

  Saves the final calculated metrics and ASCII summary tables to the file system.

Variables

- const std::filesystem::path RESULTS_PATH = std::filesystem::current_path() / "results"

  Path pointing to the root `results/` directory.

6.16.1 Detailed Description

Provides structures and routines for tracking, formatting, and persisting simulation metrics.

6.16.2 Enumeration Type Documentation

Algorithm

`enum class metrics::Algorithm [strong]`

Identifies the specific optimization algorithm used in a simulation run.

Enumerator

NONE	Uninitialized algorithm.
solver_maximum_cost	Exact ILP solver maximizing avoided penalty cost.
greedy_cost_benefit	Deterministic surrogate cost-benefit greedy heuristic.
greedy_random	Single-stage uniform random greedy heuristic.
greedy_random_fallback	Two-stage Edge-first random greedy heuristic with Cloud fallback.
simulated_annealing	Simulated Annealing meta-heuristic.

Definition at line 480 of file mmkp.hpp.

Paradigm

```
enum class metrics::Paradigm  [strong]
```

Enumerates the optimization paradigms supported by the simulator.

Enumerator

NONE	Uninitialized paradigm.
EXACT	Exact mathematical programming (ILP via CPLEX).
HEURISTIC	Constructive heuristic algorithms.
METAHEURISTIC	Meta-heuristic optimization algorithms.

Definition at line 455 of file mmkp.hpp.

6.16.3 Function Documentation

calculate_metrics()

```
void metrics::calculate_metrics (
            Result & state )  [inline]
```

Computes the final simulation metrics based on the current device-to-server allocations.

Parameters

state	The simulation result structure holding `devices`, `servers`, and `stats`.

Definition at line 1001 of file mmkp.hpp.
References metrics::Result::devices, metrics::Result::servers, and metrics::Result::stats.
Referenced by metaheuristic::boot(), and run_simulation().

save_results()

```
void metrics::save_results (
            const Result & state,
            bool show_servers_usage = false )  [inline]
```

Saves the final calculated metrics and ASCII summary tables to the file system.

Parameters

state	The simulation result structure to be saved.
show_servers_usage	Flag to indicate if individual server resource usage should be printed.

Definition at line 1263 of file mmkp.hpp.
References metrics::Result::servers, display::show_metrics(), display::show_servers_usage(), and metrics::Result::stats.
Referenced by metaheuristic::boot(), and run_simulation().

to_string() [1/2]

```
std::string metrics::to_string (
            Algorithm algorithm )  [inline]
```

Converts an `Algorithm` enum value to its standardized string identifier.

Parameters

algorithm	The `Algorithm` value to convert.

Returns

String representation of the algorithm.

Definition at line 494 of file mmkp.hpp.
References greedy_cost_benefit, greedy_random, greedy_random_fallback, simulated_annealing, and solver_maximum_cost.

to_string() [2/2]

```
std::string metrics::to_string (
            Paradigm paradigm ) [inline]
```

Converts a `Paradigm` enum value to its uppercase string representation.

Parameters

paradigm	The `Paradigm` value to convert.

Returns

String representation of the paradigm.

Definition at line 467 of file mmkp.hpp.
References EXACT, HEURISTIC, and METAHEURISTIC.
Referenced by metrics::MetaHeuristicMetrics::build_metrics(), metrics::Metrics::data(), metrics::MetaHeuristicMetrics::data(), metrics::Metrics::get_base_directory_path(), and metrics::Metrics::save_to_file().

6.16.4 Variable Documentation

RESULTS_PATH

```
const std::filesystem::path metrics::RESULTS_PATH = std::filesystem::current_path() / "results"
```

Path pointing to the root `results/` directory.
Definition at line 449 of file mmkp.hpp.
Referenced by metrics::Metrics::get_base_directory_path().

6.17 randomness Namespace Reference

Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.

Functions

- std::mt19937 & get_engine ()

  Provides a singleton instance of a pseudo-random number engine.

- template<typename T >
  T random_number (T min, T max)

  Generates a uniform random number within a specified inclusive interval `[min, max]`.

- template<typename T >
  T random_number (T min, T max, std::vector< T > &exclude)

  Generates a random number within `[min, max]`, excluding values present in `exclude`.

- template<typename T >
  std::vector< T > shuffled_range (T min, T max)

  Creates a randomly shuffled vector containing all integers in the inclusive range `[min, max]`.

6.17.1 Detailed Description

Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.

6.17.2 Function Documentation

get_engine()

```
std::mt19937 & randomness::get_engine ( )  [inline]
```

Provides a singleton instance of a pseudo-random number engine.
Returns a reference to a static `std::mt19937` engine seeded once via `std::random_device`.

Returns

A reference to the static `std::mt19937` random engine.

Definition at line 268 of file tools.hpp.
References random_number().
Referenced by random_number(), random_number(), and shuffled_range().

random_number() [1/2]

```
template<typename T >
T randomness::random_number (
            T min,
            T max )  [inline]
```

Generates a uniform random number within a specified inclusive interval `[min, max]`.

Template Parameters

T	The numeric type (must satisfy `std::is_arithmetic_v<T>`).

Parameters

min	The lower bound of the interval (inclusive).
max	The upper bound of the interval (inclusive).

Returns

A uniformly distributed random number within `[min, max]`.

Exceptions

std::invalid_argument	If `min > max`.

Definition at line 283 of file tools.hpp.
References get_engine(), and random_number().
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), data::generate::devices(), get_engine(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_random(), heuristic::anonymous_namespace{heuristic.hpp}::
random_number(), random_number(), data::generate::servers(), data::generate::services(), shuffled_range(), and metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing().

random_number() [2/2]

```
template<typename T >
T randomness::random_number (
            T min,
            T max,
            std::vector< T > & exclude )  [inline]
```

Generates a random number within `[min, max]`, excluding values present in `exclude`.
The generated number is automatically appended to `exclude` to prevent future duplicates.

Template Parameters

T	The numeric type (must satisfy `std::is_arithmetic_v<T>`).

Parameters

min	The lower bound of the interval (inclusive).
max	The upper bound of the interval (inclusive).
exclude	A reference to a vector containing values to be excluded.

Returns

A random number within `[min, max]` not previously in `exclude`.

Exceptions

std::invalid_argument	If `min > max`, or if all possible integral values in the range are already excluded.

Definition at line 309 of file tools.hpp.
References get_engine(), and random_number().

shuffled_range()

```
template<typename T >
std::vector< T > randomness::shuffled_range (
            T min,
            T max )  [inline]
```

Creates a randomly shuffled vector containing all integers in the inclusive range `[min, max]`.

Template Parameters

T	The integral type for the range.

Parameters

min	The lower bound of the range (inclusive).
max	The upper bound of the range (inclusive).

Returns

A randomly shuffled `std::vector<T>` of unique integers from `min` to `max`.

Exceptions

std::invalid_argument	If `min > max`.

Definition at line 348 of file tools.hpp.
References get_engine(), and random_number().

6.18 sorting Namespace Reference

Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes.

Namespaces

- namespace details

  Internal implementation helpers for pointer-to-member index sorting.

Functions

- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sort_entities (const std::vector< T > &entities)

  Sorts the indices of all entities (from 1 to N-1) by a specific struct/class attribute.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sort_entities (const std::vector< T > &entities, const std::vector< int > &indices_to_sort)

  Sorts a specific subset of entity indices by a struct/class member attribute.

6.18.1 Detailed Description

Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes.

6.18.2 Function Documentation

sort_entities() [1/2]

```
template<bool Ascending, typename T , typename AttributeType , AttributeType T::* Attribute↩
Ptr>
std::vector< int > sorting::sort_entities (
            const std::vector< T > & entities ) [inline]
```

Sorts the indices of all entities (from 1 to N-1) by a specific struct/class attribute.
Does not mutate the original entities vector. Assumes 1-based indexing where index 0 is a placeholder.

Template Parameters

Ascending	If true, sorts in ascending order; otherwise, descending.
T	The type of the entity.
AttributeType	The data type of the member attribute.
AttributePtr	Pointer-to-member specifying the attribute for comparison.

Parameters

entities	The constant 1-indexed vector of entities.

Returns

A new std::vector<int> containing the sorted 1-based indices.

Definition at line 410 of file tools.hpp.
References sort_entities().
Referenced by sort_entities(), and sort_entities().

sort_entities() [2/2]

```
template<bool Ascending, typename T , typename AttributeType , AttributeType T::* Attribute↩
Ptr>
std::vector< int > sorting::sort_entities (
            const std::vector< T > & entities,
            const std::vector< int > & indices_to_sort ) [inline]
```

Sorts a specific subset of entity indices by a struct/class member attribute.

Template Parameters

Ascending	If `true`, sorts in ascending order; otherwise, descending.
T	The type of the entity.
AttributeType	The data type of the member attribute.
AttributePtr	Pointer-to-member specifying the attribute for comparison.

Parameters

entities	The constant 1-indexed vector of entities.
indices_to_sort	A vector of integers representing the subset of 1-based indices to sort.

Returns

A new `std::vector<int>` containing the sorted subset indices.

Definition at line 429 of file tools.hpp.
References sort_entities().

6.19 sorting::details Namespace Reference

Internal implementation helpers for pointer-to-member index sorting.

Functions

- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  void sort_indices_impl (const std::vector< T > &entities, std::vector< int > &idxs)

  Sorts a vector of indices based on a specified struct/class member attribute.

6.19.1 Detailed Description

Internal implementation helpers for pointer-to-member index sorting.

6.19.2 Function Documentation

sort_indices_impl()

```
template<bool Ascending, typename T , typename AttributeType , AttributeType T::* Attribute↩
Ptr>
void sorting::details::sort_indices_impl (
            const std::vector< T > & entities,
            std::vector< int > & idxs )
```

Sorts a vector of indices based on a specified struct/class member attribute.

Template Parameters

Ascending	Sort direction flag (`true` for ascending, `false` for descending).
T	Entity struct/class type.
AttributeType	Data type of the sorted member attribute.
AttributePtr	Pointer-to-member specifying the comparison attribute.

Parameters

entities	Constant reference to the entity vector.
idxs	Vector of 1-based indices to be sorted in-place.

Definition at line 382 of file tools.hpp.

7 Class Documentation

7.1 core::Device Struct Reference

Represents an IoT device requesting computational resources for a specific service.
`#include <mmkp.hpp>`

Public Member Functions

- Device ()

  Default constructor initializing an empty/placeholder device.
- Device (int id_, double cost_, double cpu_, double mem_, double sto_, int tasks_, int service_)

  Parameterized constructor initializing a device with specific resource demands.

Public Attributes

- int id

  Unique 1-based identifier of the device.
- double cost

  SLA penalty cost incurred if the device is not served.
- double cpu

  Required processing capacity (CPU).
- double mem

  Required memory capacity (MEM).
- double sto

  Required storage capacity (STO).
- int tasks

  Number of atomic tasks composing the requested service.
- int service

  Identifier of the service template assigned to this device.
- bool served = false

  Flag indicating whether the device is currently allocated to a server.
- int server_id = -1

  ID of the server hosting this device (`-1` if unserved).

7.1.1 Detailed Description

Represents an IoT device requesting computational resources for a specific service.
Definition at line 34 of file mmkp.hpp.

7.1.2 Constructor & Destructor Documentation

Device() [1/2]

`core::Device::Device ( ) [inline]`
Default constructor initializing an empty/placeholder device.
Definition at line 48 of file mmkp.hpp.

Device() [2/2]

```
core::Device::Device (
            int id_,
            double cost_,
            double cpu_,
            double mem_,
            double sto_,
            int tasks_,
            int service_ )  [inline]
```

Parameterized constructor initializing a device with specific resource demands.

Parameters

*id_*	Unique identifier of the device.
*cost_*	Penalty cost if unserved.
*cpu_*	CPU demand.
*mem_*	Memory demand.
*sto_*	Storage demand.
*tasks_*	Number of service tasks.
*service_*	Service profile ID.

Definition at line 60 of file mmkp.hpp.

7.1.3 Member Data Documentation

cost

```
double core::Device::cost
```

SLA penalty cost incurred if the device is not served.
Definition at line 36 of file mmkp.hpp.
Referenced by core::Server::add_service(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), core::Server::rmv_service(), and display::show_device().

cpu

```
double core::Device::cpu
```

Required processing capacity (CPU).
Definition at line 37 of file mmkp.hpp.
Referenced by core::Server::add_service(), core::Server::can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost core::Server::rmv_service(), and display::show_device().

id

```
int core::Device::id
```

Unique 1-based identifier of the device.
Definition at line 35 of file mmkp.hpp.
Referenced by core::Server::add_service(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), core::Server::rmv_service(), and display::show_device().

mem

```
double core::Device::mem
```

Required memory capacity (MEM).

Definition at line 38 of file mmkp.hpp.
Referenced by core::Server::add_service(), core::Server::can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost
core::Server::rmv_service(), and display::show_device().

served

`bool core::Device::served = false`
Flag indicating whether the device is currently allocated to a server.
Definition at line 42 of file mmkp.hpp.
Referenced by core::Server::add_service(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), core::Server::rmv_service(), and display::show_device().

server_id

`int core::Device::server_id = -1`
ID of the server hosting this device (`-1` if unserved).
Definition at line 43 of file mmkp.hpp.
Referenced by core::Server::add_service(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), core::Server::rmv_service(), and display::show_device().

service

`int core::Device::service`
Identifier of the service template assigned to this device.
Definition at line 41 of file mmkp.hpp.
Referenced by display::show_device().

sto

`double core::Device::sto`
Required storage capacity (STO).
Definition at line 39 of file mmkp.hpp.
Referenced by core::Server::add_service(), core::Server::can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost
core::Server::rmv_service(), and display::show_device().

tasks

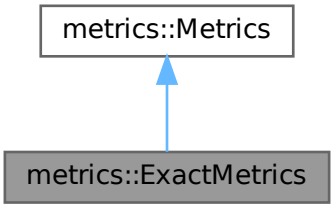
`int core::Device::tasks`
Number of atomic tasks composing the requested service.
Definition at line 40 of file mmkp.hpp.
Referenced by display::show_device().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

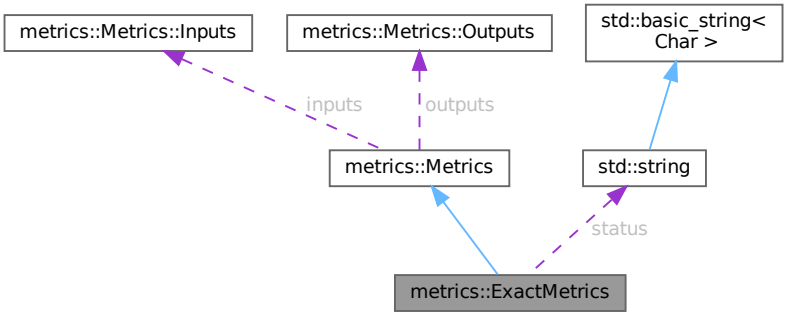
7.2 metrics::ExactMetrics Struct Reference

Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.
`#include <mmkp.hpp>`

Inheritance diagram for metrics::ExactMetrics:



Collaboration diagram for metrics::ExactMetrics:



Public Member Functions

- ExactMetrics (Algorithm algorithm_name_, int run_id_, const std::shared_ptr< Metrics > &base)

  Constructs an `ExactMetrics` object copying input dimensions from a base instance.
- ExactMetrics (Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs an `ExactMetrics` object with explicit instance dimensions.
- ExactMetrics (const std::shared_ptr< Metrics > &base, int run_id)

  Constructs an `ExactMetrics` object from a base pointer and run ID.
- std::shared_ptr< Metrics > clone () const override

  Clones this `ExactMetrics` instance polymorphically.
- std::vector< MetricEntry > build_metrics () const override

  Appends CPLEX solver statistics (`Status`, `OF`, `GAP`) to the base metric entries.
- std::vector< std::string > get_header () const override

  Appends exact solver column headers (`Status`, `Obj_Function`, `Gap`) to the CSV header.
- std::vector< std::string > data () const override

  Appends exact solver values to the CSV row data.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- std::string status = "Unknown"

  Termination status reported by CPLEX (e.g., "Optimal").
- double objective_function = 0.0

  Objective function value achieved by the solver.
- double gap = 1.0

  Relative MIP optimality gap.
- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.
- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

7.2.1 Detailed Description

Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.
Definition at line 709 of file mmkp.hpp.

7.2.2 Constructor & Destructor Documentation

ExactMetrics() [1/3]

```
metrics::ExactMetrics::ExactMetrics (
            Algorithm algorithm_name_,
            int run_id_,
            const std::shared_ptr< Metrics > & base )  [inline]
```

Constructs an `ExactMetrics` object copying input dimensions from a base instance.

Parameters

*algorithm_name_*	Exact algorithm identifier.
*run_id_*	Execution run number.
base	Base metrics pointer containing input parameters.

Definition at line 720 of file mmkp.hpp.
References metrics::Metrics::inputs.

ExactMetrics() [2/3]

```
metrics::ExactMetrics::ExactMetrics (
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc )  [inline]
```

Constructs an `ExactMetrics` object with explicit instance dimensions.

Parameters

*algorithm_↩ name_*	Exact algorithm identifier.
run_id	Execution run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.

Definition at line 733 of file mmkp.hpp.

ExactMetrics() [3/3]

```
metrics::ExactMetrics::ExactMetrics (
            const std::shared_ptr< Metrics > & base,
            int run_id )  [inline]
```

Constructs an `ExactMetrics` object from a base pointer and run ID.

Parameters

base	Base metrics pointer.
run↩ _id	Execution run number.

Definition at line 741 of file mmkp.hpp.

7.2.3 Member Function Documentation

build_metrics()

```
std::vector< MetricEntry > metrics::ExactMetrics::build_metrics ( ) const  [inline], [override],
[virtual]
```

Appends CPLEX solver statistics (`Status`, `OF`, `GAP`) to the base metric entries.

Returns

Vector of `MetricEntry` records.

Reimplemented from metrics::Metrics.
Definition at line 756 of file mmkp.hpp.
References metrics::Metrics::build_metrics(), DECIMAL_PLACES, gap, objective_function, status, convert::to_percentage_string(), and convert::to_string().

clone()

```
std::shared_ptr< Metrics > metrics::ExactMetrics::clone ( ) const  [inline], [override], [virtual]
```

Clones this `ExactMetrics` instance polymorphically.

Returns

Shared pointer to the cloned `ExactMetrics` object.

Reimplemented from metrics::Metrics.
Definition at line 748 of file mmkp.hpp.

data()

```
std::vector< std::string > metrics::ExactMetrics::data ( ) const  [inline], [override], [virtual]
```

Appends exact solver values to the CSV row data.

Returns

Vector of formatted string values.

Reimplemented from metrics::Metrics.
Definition at line 780 of file mmkp.hpp.
References metrics::Metrics::data(), DECIMAL_PLACES, gap, objective_function, status, and convert::to_string().

get_base_directory_path()

`virtual std::filesystem::path metrics::Metrics::get_base_directory_path ( ) const [inline], [virtual], [inherited]`
Returns the output directory path corresponding to the current paradigm.

Returns

Filesystem path `results/<PARADIGM>`.

Definition at line 621 of file mmkp.hpp.
References metrics::RESULTS_PATH, and metrics::to_string().
Referenced by metrics::Metrics::save_to_file().

get_header()

`std::vector< std::string > metrics::ExactMetrics::get_header ( ) const [inline], [override], [virtual]`
Appends exact solver column headers (`Status`, `Obj_Function`, `Gap`) to the CSV header.

Returns

Vector of column header strings.

Reimplemented from metrics::Metrics.
Definition at line 768 of file mmkp.hpp.
References metrics::Metrics::get_header().

save_to_file()

`void metrics::Metrics::save_to_file ( ) const [inline], [inherited]`
Appends the current simulation metrics row to the corresponding algorithm CSV file.
Definition at line 692 of file mmkp.hpp.
References file_manager::csv::append(), metrics::Metrics::data(), metrics::Metrics::get_base_directory_path(), metrics::Metrics::get_header(), and metrics::to_string().

7.2.4 Member Data Documentation

algorithm

`Algorithm metrics::Metrics::algorithm = Algorithm::NONE [inherited]`
Algorithm executed in the current run.
Definition at line 523 of file mmkp.hpp.
Referenced by metrics::MetaHeuristicMetrics::build_metrics(), and metrics::Metrics::data().

gap

`double metrics::ExactMetrics::gap = 1.0`
Relative MIP optimality gap.
Definition at line 712 of file mmkp.hpp.
Referenced by build_metrics(), and data().

inputs

```
struct metrics::Metrics::Inputs metrics::Metrics::inputs  [inherited]
```
Instance input parameters.
Referenced by metrics::Metrics::build_metrics(), metrics::Metrics::data(), ExactMetrics(), metrics::HeuristicMetrics::HeuristicMetrics(), metrics::MetaHeuristicMetrics::MetaHeuristicMetrics(), metrics::Metrics::Metrics(), and metrics::Metrics::Metrics().

objective_function

```
double metrics::ExactMetrics::objective_function = 0.0
```
Objective function value achieved by the solver.
Definition at line 711 of file mmkp.hpp.
Referenced by build_metrics(), and data().

outputs

```
struct metrics::Metrics::Outputs metrics::Metrics::outputs  [inherited]
```
Simulation output metrics.
Referenced by metrics::Metrics::build_metrics(), metrics::Metrics::data(), and metrics::Metrics::Metrics().

paradigm

```
Paradigm metrics::Metrics::paradigm = Paradigm::NONE  [inherited]
```
Optimization paradigm of the current run.
Definition at line 522 of file mmkp.hpp.
Referenced by metrics::Metrics::data().

run_id

```
int metrics::Metrics::run_id = 0  [inherited]
```
Execution run identifier (1-indexed).
Definition at line 524 of file mmkp.hpp.
Referenced by metrics::Metrics::data().

status

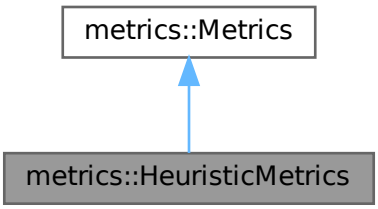
```
std::string metrics::ExactMetrics::status = "Unknown"
```
Termination status reported by CPLEX (e.g., "`Optimal`").
Definition at line 710 of file mmkp.hpp.
Referenced by build_metrics(), and data().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

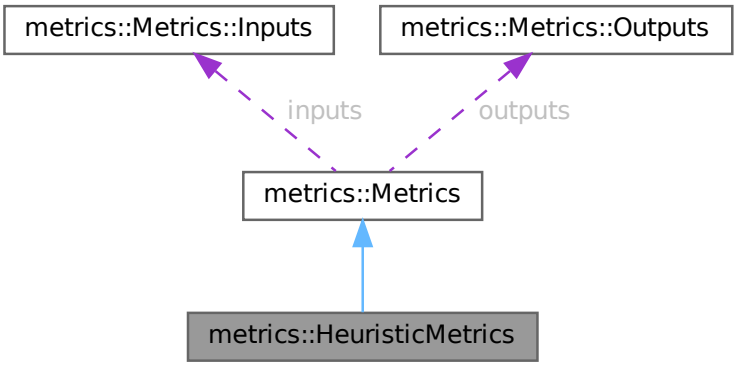
7.3 metrics::HeuristicMetrics Struct Reference

Specialization of `Metrics` for constructive heuristic runs.
`#include <mmkp.hpp>`

Inheritance diagram for metrics::HeuristicMetrics:



Collaboration diagram for metrics::HeuristicMetrics:



Public Member Functions

- HeuristicMetrics (Algorithm algorithm_name_, int run_id_, const std::shared_ptr< Metrics > &base)

  Constructs a `HeuristicMetrics` object copying input dimensions from a base instance.
- HeuristicMetrics (Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `HeuristicMetrics` object with explicit instance dimensions.
- HeuristicMetrics (const std::shared_ptr< Metrics > &base, int run_id)

  Constructs a `HeuristicMetrics` object from a base pointer and run ID.
- std::shared_ptr< Metrics > clone () const override

  Clones this `HeuristicMetrics` instance polymorphically.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- virtual std::vector< MetricEntry > build_metrics () const

  Builds the list of formatted metric entries for console and file reporting.
- virtual std::vector< std::string > get_header () const

  Returns the CSV header columns for the current metric type.
- virtual std::vector< std::string > data () const

  Collects the metric values matching the columns of `get_header()`.

- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.
- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

7.3.1 Detailed Description

Specialization of `Metrics` for constructive heuristic runs.
Definition at line 793 of file mmkp.hpp.

7.3.2 Constructor & Destructor Documentation

HeuristicMetrics() [1/3]

```
metrics::HeuristicMetrics::HeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id_,
            const std::shared_ptr< Metrics > & base )  [inline]
```

Constructs a `HeuristicMetrics` object copying input dimensions from a base instance.

Parameters

*algorithm_↩ name_*	Heuristic algorithm identifier.
*run_id_*	Execution run number.
base	Base metrics pointer containing input parameters.

Definition at line 800 of file mmkp.hpp.
References metrics::Metrics::inputs.

HeuristicMetrics() [2/3]

```
metrics::HeuristicMetrics::HeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc )  [inline]
```

Constructs a `HeuristicMetrics` object with explicit instance dimensions.

Parameters

*algorithm_↩ name_*	Heuristic algorithm identifier.
run_id	Execution run number.
d	Total device count.

Parameters

s_ec	Edge server count.
s_cc	Cloud server count.

Definition at line 813 of file mmkp.hpp.

HeuristicMetrics() [3/3]

```
metrics::HeuristicMetrics::HeuristicMetrics (
            const std::shared_ptr< Metrics > & base,
            int run_id )  [inline]
```

Constructs a `HeuristicMetrics` object from a base pointer and run ID.

Parameters

base	Base metrics pointer.
run↩ _id	Execution run number.

Definition at line 821 of file mmkp.hpp.

7.3.3 Member Function Documentation

build_metrics()

```
virtual std::vector< MetricEntry > metrics::Metrics::build_metrics ( ) const  [inline], [virtual],
[inherited]
```

Builds the list of formatted metric entries for console and file reporting.

Returns

Vector of `MetricEntry` records.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 629 of file mmkp.hpp.
References DECIMAL_PLACES, metrics::Metrics::Inputs::devices_count, metrics::Metrics::Outputs::devices_served_cc_count, metrics::Metrics::Outputs::devices_served_count, metrics::Metrics::Outputs::devices_served_ec_count, metrics::Metrics::Outputs::ex metrics::Metrics::inputs, metrics::Metrics::Inputs::max_penalty_cost, metrics::Metrics::outputs, metrics::Metrics::Inputs::servers_cc_c metrics::Metrics::Inputs::servers_ec_count, metrics::Metrics::Outputs::servers_used_cc_count, metrics::Metrics::Outputs::servers_us metrics::Metrics::Outputs::servers_used_ec_count, convert::to_percentage_string(), convert::to_string(), and metrics::Metrics::Outputs::total_penalty_cost.
Referenced by metrics::ExactMetrics::build_metrics(), and metrics::MetaHeuristicMetrics::build_metrics().

clone()

```
std::shared_ptr< Metrics > metrics::HeuristicMetrics::clone ( ) const  [inline], [override],
[virtual]
```

Clones this `HeuristicMetrics` instance polymorphically.

Returns

Shared pointer to the cloned `HeuristicMetrics` object.

Reimplemented from metrics::Metrics.
Definition at line 828 of file mmkp.hpp.

data()

```
virtual std::vector< std::string > metrics::Metrics::data ( ) const  [inline], [virtual],
[inherited]
```

Collects the metric values matching the columns of `get_header()`.

Returns

Vector of formatted string values for a single CSV row.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 669 of file mmkp.hpp.
References metrics::Metrics::algorithm, DECIMAL_PLACES, metrics::Metrics::Inputs::devices_count, metrics::Metrics::Outputs::devic
metrics::Metrics::Outputs::devices_served_count, metrics::Metrics::Outputs::devices_served_ec_count, metrics::Metrics::Outputs::ex
metrics::Metrics::inputs, metrics::Metrics::Inputs::max_penalty_cost, metrics::Metrics::outputs, metrics::Metrics::paradigm,
metrics::Metrics::run_id, metrics::Metrics::Inputs::servers_cc_count, metrics::Metrics::Inputs::servers_ec_count,
metrics::Metrics::Outputs::servers_used_cc_count, metrics::Metrics::Outputs::servers_used_count, metrics::Metrics::Outputs::servers
convert::to_string(), metrics::to_string(), and metrics::Metrics::Outputs::total_penalty_cost.
Referenced by metrics::ExactMetrics::data(), metrics::MetaHeuristicMetrics::data(), and metrics::Metrics::save_to_file().

get_base_directory_path()

```
virtual std::filesystem::path metrics::Metrics::get_base_directory_path ( ) const  [inline],
[virtual], [inherited]
```

Returns the output directory path corresponding to the current paradigm.

Returns

Filesystem path `results/<PARADIGM>`.

Definition at line 621 of file mmkp.hpp.
References metrics::RESULTS_PATH, and metrics::to_string().
Referenced by metrics::Metrics::save_to_file().

get_header()

```
virtual std::vector< std::string > metrics::Metrics::get_header ( ) const  [inline], [virtual],
[inherited]
```

Returns the CSV header columns for the current metric type.

Returns

Vector of column header strings.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 657 of file mmkp.hpp.
Referenced by metrics::ExactMetrics::get_header(), metrics::MetaHeuristicMetrics::get_header(), and metrics::Metrics::save_to_file().

save_to_file()

```
void metrics::Metrics::save_to_file ( ) const  [inline], [inherited]
```

Appends the current simulation metrics row to the corresponding algorithm CSV file.
Definition at line 692 of file mmkp.hpp.
References file_manager::csv::append(), metrics::Metrics::data(), metrics::Metrics::get_base_directory_path(),
metrics::Metrics::get_header(), and metrics::to_string().

7.3.4 Member Data Documentation

algorithm

```
Algorithm metrics::Metrics::algorithm = Algorithm::NONE  [inherited]
```

Algorithm executed in the current run.
Definition at line 523 of file mmkp.hpp.
Referenced by metrics::MetaHeuristicMetrics::build_metrics(), and metrics::Metrics::data().

inputs

```
struct metrics::Metrics::Inputs metrics::Metrics::inputs  [inherited]
```

Instance input parameters.

Referenced by metrics::Metrics::build_metrics(), metrics::Metrics::data(), metrics::ExactMetrics::ExactMetrics(), HeuristicMetrics(), metrics::MetaHeuristicMetrics::MetaHeuristicMetrics(), metrics::Metrics::Metrics(), and metrics::Metrics::Metrics().

outputs

```
struct metrics::Metrics::Outputs metrics::Metrics::outputs  [inherited]
```

Simulation output metrics.
Referenced by metrics::Metrics::build_metrics(), metrics::Metrics::data(), and metrics::Metrics::Metrics().

paradigm

```
Paradigm metrics::Metrics::paradigm = Paradigm::NONE  [inherited]
```

Optimization paradigm of the current run.
Definition at line 522 of file mmkp.hpp.
Referenced by metrics::Metrics::data().

run_id

```
int metrics::Metrics::run_id = 0  [inherited]
```

Execution run identifier (1-indexed).
Definition at line 524 of file mmkp.hpp.
Referenced by metrics::Metrics::data().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

7.4 metrics::Metrics::Inputs Struct Reference

Captures the static input instance dimensions and maximum possible penalty cost.
`#include <mmkp.hpp>`

Public Member Functions

- Inputs & operator= (const Inputs &other)=default

Public Attributes

- int devices_count = 0

  Total number of IoT devices in the instance.
- int servers_ec_count = 0

  Total number of Edge Computing (EC) servers.
- int servers_cc_count = 0

  Total number of Cloud Computing (CC) servers.
- double max_penalty_cost = 0

  Sum of penalty costs across all devices if none are served.

7.4.1 Detailed Description

Captures the static input instance dimensions and maximum possible penalty cost.
Definition at line 530 of file mmkp.hpp.

7.4.2 Member Function Documentation

operator=()

```
Inputs & metrics::Metrics::Inputs::operator= (
            const Inputs & other )  [default]
```

7.4.3 Member Data Documentation

devices_count

```
int metrics::Metrics::Inputs::devices_count = 0
```

Total number of IoT devices in the instance.
Definition at line 531 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

max_penalty_cost

```
double metrics::Metrics::Inputs::max_penalty_cost = 0
```

Sum of penalty costs across all devices if none are served.
Definition at line 534 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

servers_cc_count

```
int metrics::Metrics::Inputs::servers_cc_count = 0
```

Total number of Cloud Computing (CC) servers.
Definition at line 533 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

servers_ec_count

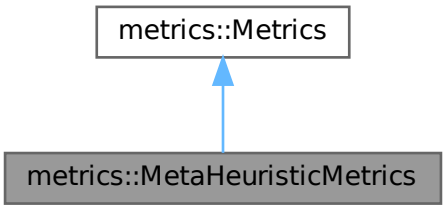
```
int metrics::Metrics::Inputs::servers_ec_count = 0
```

Total number of Edge Computing (EC) servers.
Definition at line 532 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().
The documentation for this struct was generated from the following file:

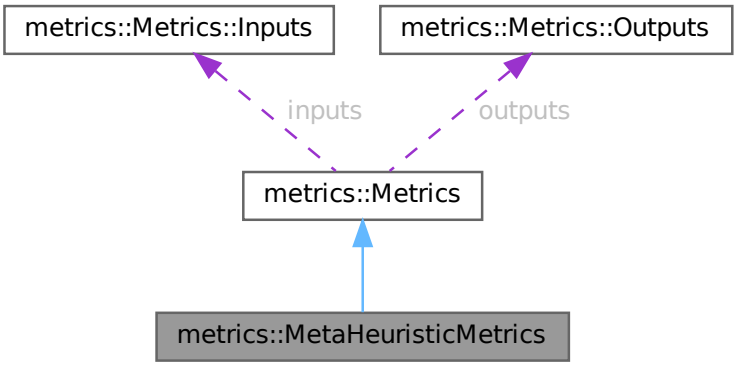
- include/mmkp.hpp

7.5 metrics::MetaHeuristicMetrics Struct Reference

Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
`#include <mmkp.hpp>`
Inheritance diagram for metrics::MetaHeuristicMetrics:



Collaboration diagram for metrics::MetaHeuristicMetrics:



Public Member Functions

- MetaHeuristicMetrics (Algorithm algorithm_name_, int run_id_, const std::shared_ptr< Metrics > &base, double temperature_, double alpha_, Algorithm heuristic_)

  Constructs a `MetaHeuristicMetrics` object copying inputs from a base instance.
- MetaHeuristicMetrics (Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc, double temperature_, double alpha_, Algorithm heuristic_)

  Constructs a `MetaHeuristicMetrics` object with explicit instance dimensions and hyperparameters.
- MetaHeuristicMetrics (const std::shared_ptr< Metrics > &base, int run_id)

  Constructs a `MetaHeuristicMetrics` object from a base pointer and run ID.
- std::shared_ptr< Metrics > clone () const override

  Clones this `MetaHeuristicMetrics` instance polymorphically.
- std::vector< MetricEntry > build_metrics () const override

  Appends meta-heuristic parameters (`Heuristic`, `Temperature`, `Alpha`) to the metric entries.
- std::vector< std::string > get_header () const override

  Appends meta-heuristic column headers (`Initial_Heuristic`, `Temperature`, `Alpha`) to the CSV header.
- std::vector< std::string > data () const override

  Appends meta-heuristic hyperparameter values to the CSV row data.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- Algorithm heuristic = Algorithm::NONE

  Constructive heuristic used to generate the initial solution.
- double temperature = 0.0

  Initial temperature T_0 for Simulated Annealing.
- double alpha = 0.0

  Cooling rate factor α for Simulated Annealing.
- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.

- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

7.5.1 Detailed Description

Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
Definition at line 837 of file mmkp.hpp.

7.5.2 Constructor & Destructor Documentation

MetaHeuristicMetrics() [1/3]

```
metrics::MetaHeuristicMetrics::MetaHeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id_,
            const std::shared_ptr< Metrics > & base,
            double temperature_,
            double alpha_,
            Algorithm heuristic_ )  [inline]
```

Constructs a `MetaHeuristicMetrics` object copying inputs from a base instance.

Parameters

*algorithm_↩ name_*	Meta-heuristic algorithm identifier.
*run_id_*	Execution run number.
base	Base metrics pointer containing input parameters.
*temperature_*	Initial temperature T_0.
*alpha_*	Geometric cooling factor α.
*heuristic_*	Initial constructive heuristic algorithm.

Definition at line 851 of file mmkp.hpp.
References metrics::Metrics::inputs.

MetaHeuristicMetrics() [2/3]

```
metrics::MetaHeuristicMetrics::MetaHeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc,
            double temperature_,
            double alpha_,
            Algorithm heuristic_ )  [inline]
```

Constructs a `MetaHeuristicMetrics` object with explicit instance dimensions and hyperparameters.

Parameters

*algorithm_↩ name_*	Meta-heuristic algorithm identifier.

Parameters

run_id	Execution run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.
*temperature_*	Initial temperature T_0.
*alpha_*	Geometric cooling factor α.
*heuristic_*	Initial constructive heuristic algorithm.

Definition at line 867 of file mmkp.hpp.

MetaHeuristicMetrics() [3/3]

```
metrics::MetaHeuristicMetrics::MetaHeuristicMetrics (
            const std::shared_ptr< Metrics > & base,
            int run_id ) [inline]
```

Constructs a MetaHeuristicMetrics object from a base pointer and run ID.

Parameters

base	Base metrics pointer.
run↩ _id	Execution run number.

Definition at line 875 of file mmkp.hpp.

7.5.3 Member Function Documentation

build_metrics()

```
std::vector< MetricEntry > metrics::MetaHeuristicMetrics::build_metrics ( ) const [inline],
[override], [virtual]
```

Appends meta-heuristic parameters (`Heuristic`, `Temperature`, `Alpha`) to the metric entries.

Returns

Vector of MetricEntry records.

Reimplemented from metrics::Metrics.
Definition at line 890 of file mmkp.hpp.
References metrics::Metrics::algorithm, alpha, metrics::Metrics::build_metrics(), DECIMAL_PLACES, temperature, convert::to_string(), and metrics::to_string().

clone()

```
std::shared_ptr< Metrics > metrics::MetaHeuristicMetrics::clone ( ) const [inline], [override],
[virtual]
```

Clones this MetaHeuristicMetrics instance polymorphically.

Returns

Shared pointer to the cloned MetaHeuristicMetrics object.

Reimplemented from metrics::Metrics.
Definition at line 882 of file mmkp.hpp.

data()

```
std::vector< std::string > metrics::MetaHeuristicMetrics::data ( ) const [inline], [override],
[virtual]
```

Appends meta-heuristic hyperparameter values to the CSV row data.

Returns

Vector of formatted string values.

Reimplemented from metrics::Metrics.
Definition at line 915 of file mmkp.hpp.
References alpha, metrics::Metrics::data(), DECIMAL_PLACES, temperature, convert::to_string(), and metrics::to_string().

get_base_directory_path()

`virtual std::filesystem::path metrics::Metrics::get_base_directory_path ( ) const [inline], [virtual], [inherited]`
Returns the output directory path corresponding to the current paradigm.

Returns

Filesystem path `results/<PARADIGM>`.

Definition at line 621 of file mmkp.hpp.
References metrics::RESULTS_PATH, and metrics::to_string().
Referenced by metrics::Metrics::save_to_file().

get_header()

`std::vector< std::string > metrics::MetaHeuristicMetrics::get_header ( ) const [inline], [override], [virtual]`
Appends meta-heuristic column headers (`Initial_Heuristic`, `Temperature`, `Alpha`) to the CSV header.

Returns

Vector of column header strings.

Reimplemented from metrics::Metrics.
Definition at line 903 of file mmkp.hpp.
References metrics::Metrics::get_header().

save_to_file()

`void metrics::Metrics::save_to_file ( ) const [inline], [inherited]`
Appends the current simulation metrics row to the corresponding algorithm CSV file.
Definition at line 692 of file mmkp.hpp.
References file_manager::csv::append(), metrics::Metrics::data(), metrics::Metrics::get_base_directory_path(), metrics::Metrics::get_header(), and metrics::to_string().

7.5.4 Member Data Documentation

algorithm

`Algorithm metrics::Metrics::algorithm = Algorithm::NONE [inherited]`
Algorithm executed in the current run.
Definition at line 523 of file mmkp.hpp.
Referenced by build_metrics(), and metrics::Metrics::data().

alpha

`double metrics::MetaHeuristicMetrics::alpha = 0.0`
Cooling rate factor α for Simulated Annealing.
Definition at line 840 of file mmkp.hpp.
Referenced by build_metrics(), and data().

heuristic

```
Algorithm metrics::MetaHeuristicMetrics::heuristic = Algorithm::NONE
```
Constructive heuristic used to generate the initial solution.
Definition at line 838 of file mmkp.hpp.

inputs

```
struct metrics::Metrics::Inputs metrics::Metrics::inputs  [inherited]
```
Instance input parameters.
Referenced by metrics::Metrics::build_metrics(), metrics::Metrics::data(), metrics::ExactMetrics::ExactMetrics(), metrics::HeuristicMetrics::HeuristicMetrics(), MetaHeuristicMetrics(), metrics::Metrics::Metrics(), and metrics::Metrics::Metrics().

outputs

```
struct metrics::Metrics::Outputs metrics::Metrics::outputs  [inherited]
```
Simulation output metrics.
Referenced by metrics::Metrics::build_metrics(), metrics::Metrics::data(), and metrics::Metrics::Metrics().

paradigm

```
Paradigm metrics::Metrics::paradigm = Paradigm::NONE  [inherited]
```
Optimization paradigm of the current run.
Definition at line 522 of file mmkp.hpp.
Referenced by metrics::Metrics::data().

run_id

```
int metrics::Metrics::run_id = 0  [inherited]
```
Execution run identifier (1-indexed).
Definition at line 524 of file mmkp.hpp.
Referenced by metrics::Metrics::data().

temperature

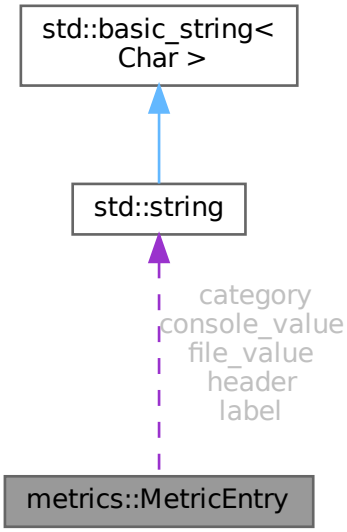
```
double metrics::MetaHeuristicMetrics::temperature = 0.0
```
Initial temperature T_0 for Simulated Annealing.
Definition at line 839 of file mmkp.hpp.
Referenced by build_metrics(), and data().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

7.6 metrics::MetricEntry Struct Reference

Represents a formatted key-value entry for displaying or exporting simulation metrics.
`#include <mmkp.hpp>`

Collaboration diagram for metrics::MetricEntry:



Public Attributes

- std::string category

  Section category header (e.g., "GENERAL", "DEVICES", "COSTS").
- std::string header

  Short column header name for CSV output.
- std::string label

  Human-readable row label for console/ASCII table output.
- std::string file_value

  Raw formatted string value for CSV persistence.
- std::string console_value

  Decorated string value (with units/percentages) for ASCII tables.

7.6.1 Detailed Description

Represents a formatted key-value entry for displaying or exporting simulation metrics.
Definition at line 509 of file mmkp.hpp.

7.6.2 Member Data Documentation

category

`std::string metrics::MetricEntry::category`
Section category header (e.g., "GENERAL", "DEVICES", "COSTS").
Definition at line 510 of file mmkp.hpp.

console_value

`std::string metrics::MetricEntry::console_value`
Decorated string value (with units/percentages) for ASCII tables.
Definition at line 514 of file mmkp.hpp.

file_value

`std::string metrics::MetricEntry::file_value`
Raw formatted string value for CSV persistence.
Definition at line 513 of file mmkp.hpp.

header

`std::string metrics::MetricEntry::header`
Short column header name for CSV output.
Definition at line 511 of file mmkp.hpp.

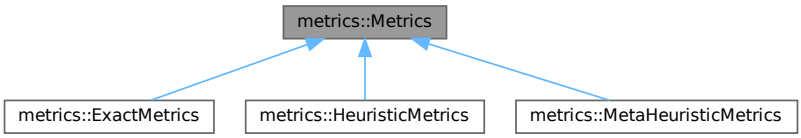
label

`std::string metrics::MetricEntry::label`
Human-readable row label for console/ASCII table output.
Definition at line 512 of file mmkp.hpp.
The documentation for this struct was generated from the following file:

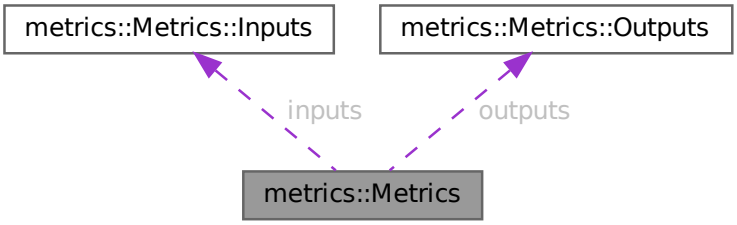
- include/mmkp.hpp

7.7 metrics::Metrics Struct Reference

Polymorphic base structure for tracking, calculating, and saving simulation metrics.
`#include <mmkp.hpp>`
Inheritance diagram for metrics::Metrics:



Collaboration diagram for metrics::Metrics:



Classes

- struct Inputs

  Captures the static input instance dimensions and maximum possible penalty cost.
- struct Outputs

  Captures the resulting performance and resource utilization metrics after allocation.

Public Member Functions

- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id)

  Constructs a `Metrics` object with paradigm, algorithm, and run ID.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const std::shared_ptr< Metrics > &base)

  Constructs a `Metrics` object copying input dimensions from a base instance.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `Metrics` object with explicit instance counts.
- Metrics (const std::shared_ptr< Metrics > &base, int run_id)

  Copy-like constructor initializing from an existing `Metrics` pointer and updating `run_id`.
- virtual ~Metrics ()=default

  Virtual destructor for safe polymorphic cleanup.
- virtual std::shared_ptr< Metrics > clone () const

  Creates a polymorphic deep copy of this `Metrics` instance.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- virtual std::vector< MetricEntry > build_metrics () const

  Builds the list of formatted metric entries for console and file reporting.
- virtual std::vector< std::string > get_header () const

  Returns the CSV header columns for the current metric type.
- virtual std::vector< std::string > data () const

  Collects the metric values matching the columns of `get_header()`.
- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.
- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

7.7.1 Detailed Description

Polymorphic base structure for tracking, calculating, and saving simulation metrics.
Definition at line 521 of file mmkp.hpp.

7.7.2 Constructor & Destructor Documentation

Metrics() [1/4]

```
metrics::Metrics::Metrics (
            Paradigm paradigm_,
            Algorithm algorithm_name_,
            int run_id )  [inline]
```

Constructs a `Metrics` object with paradigm, algorithm, and run ID.

Parameters

*paradigm_*	Optimization paradigm.
*algorithm_name_*	Executed algorithm.
run_id	Current run number.

Definition at line 562 of file mmkp.hpp.

Metrics() [2/4]

```
metrics::Metrics::Metrics (
            Paradigm paradigm_,
            Algorithm algorithm_name_,
            int run_id,
            const std::shared_ptr< Metrics > & base )  [inline]
```

Constructs a `Metrics` object copying input dimensions from a base instance.

Parameters

*paradigm_*	Optimization paradigm.
*algorithm_name_*	Executed algorithm.
run_id	Current run number.
base	Shared pointer to a base `Metrics` instance containing input parameters.

Definition at line 572 of file mmkp.hpp.
References inputs.

Metrics() [3/4]

```
metrics::Metrics::Metrics (
            Paradigm paradigm_,
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc )  [inline]
```

Constructs a `Metrics` object with explicit instance counts.

Parameters

*paradigm_*	Optimization paradigm.
*algorithm_name_*	Executed algorithm.
run_id	Current run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.

Definition at line 586 of file mmkp.hpp.

Metrics() [4/4]

```
metrics::Metrics::Metrics (
            const std::shared_ptr< Metrics > & base,
```

```
            int run_id )  [inline]
```

Copy-like constructor initializing from an existing `Metrics` pointer and updating `run_id`.

Parameters

base	Shared pointer to an existing `Metrics` object.
run↩ _id	New run identifier.

Definition at line 594 of file mmkp.hpp.
References inputs, and outputs.

~Metrics()

```
virtual metrics::Metrics::~Metrics ( )  [virtual], [default]
```

Virtual destructor for safe polymorphic cleanup.

7.7.3 Member Function Documentation

build_metrics()

```
virtual std::vector< MetricEntry > metrics::Metrics::build_metrics ( ) const  [inline], [virtual]
```

Builds the list of formatted metric entries for console and file reporting.

Returns

Vector of `MetricEntry` records.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 629 of file mmkp.hpp.
References DECIMAL_PLACES, metrics::Metrics::Inputs::devices_count, metrics::Metrics::Outputs::devices_served_cc_count, metrics::Metrics::Outputs::devices_served_count, metrics::Metrics::Outputs::devices_served_ec_count, metrics::Metrics::Outputs::ex inputs, metrics::Metrics::Inputs::max_penalty_cost, outputs, metrics::Metrics::Inputs::servers_cc_count, metrics::Metrics::Inputs::serv metrics::Metrics::Outputs::servers_used_cc_count, metrics::Metrics::Outputs::servers_used_count, metrics::Metrics::Outputs::server convert::to_percentage_string(), convert::to_string(), and metrics::Metrics::Outputs::total_penalty_cost.
Referenced by metrics::ExactMetrics::build_metrics(), and metrics::MetaHeuristicMetrics::build_metrics().

clone()

```
virtual std::shared_ptr< Metrics > metrics::Metrics::clone ( ) const  [inline], [virtual]
```

Creates a polymorphic deep copy of this `Metrics` instance.

Returns

A `std::shared_ptr<Metrics>` pointing to the cloned instance.

Reimplemented in metrics::ExactMetrics, metrics::HeuristicMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 613 of file mmkp.hpp.

data()

```
virtual std::vector< std::string > metrics::Metrics::data ( ) const  [inline], [virtual]
```

Collects the metric values matching the columns of `get_header()`.

Returns

Vector of formatted string values for a single CSV row.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 669 of file mmkp.hpp.
References algorithm, DECIMAL_PLACES, metrics::Metrics::Inputs::devices_count, metrics::Metrics::Outputs::devices_served_cc_c metrics::Metrics::Outputs::devices_served_count, metrics::Metrics::Outputs::devices_served_ec_count, metrics::Metrics::Outputs::ex

inputs, metrics::Metrics::Inputs::max_penalty_cost, outputs, paradigm, run_id, metrics::Metrics::Inputs::servers_cc_count, metrics::Metrics::Inputs::servers_ec_count, metrics::Metrics::Outputs::servers_used_cc_count, metrics::Metrics::Outputs::servers_us metrics::Metrics::Outputs::servers_used_ec_count, convert::to_string(), metrics::to_string(), and metrics::Metrics::Outputs::total_pena
Referenced by metrics::ExactMetrics::data(), metrics::MetaHeuristicMetrics::data(), and save_to_file().

get_base_directory_path()

```
virtual std::filesystem::path metrics::Metrics::get_base_directory_path ( ) const  [inline], [virtual]
```

Returns the output directory path corresponding to the current paradigm.

Returns

Filesystem path `results/<PARADIGM>`.

Definition at line 621 of file mmkp.hpp.
References metrics::RESULTS_PATH, and metrics::to_string().
Referenced by save_to_file().

get_header()

```
virtual std::vector< std::string > metrics::Metrics::get_header ( ) const  [inline], [virtual]
```

Returns the CSV header columns for the current metric type.

Returns

Vector of column header strings.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.
Definition at line 657 of file mmkp.hpp.
Referenced by metrics::ExactMetrics::get_header(), metrics::MetaHeuristicMetrics::get_header(), and save_to_file().

save_to_file()

```
void metrics::Metrics::save_to_file ( ) const  [inline]
```

Appends the current simulation metrics row to the corresponding algorithm CSV file.
Definition at line 692 of file mmkp.hpp.
References file_manager::csv::append(), data(), get_base_directory_path(), get_header(), and metrics::to_string().

7.7.4 Member Data Documentation

algorithm

```
Algorithm metrics::Metrics::algorithm = Algorithm::NONE
```

Algorithm executed in the current run.
Definition at line 523 of file mmkp.hpp.
Referenced by metrics::MetaHeuristicMetrics::build_metrics(), and data().

inputs

```
struct metrics::Metrics::Inputs metrics::Metrics::inputs
```

Instance input parameters.
Referenced by build_metrics(), data(), metrics::ExactMetrics::ExactMetrics(), metrics::HeuristicMetrics::HeuristicMetrics(), metrics::MetaHeuristicMetrics::MetaHeuristicMetrics(), Metrics(), and Metrics().

outputs

```
struct metrics::Metrics::Outputs metrics::Metrics::outputs
```

Simulation output metrics.
Referenced by build_metrics(), data(), and Metrics().

paradigm

```
Paradigm metrics::Metrics::paradigm = Paradigm::NONE
```
Optimization paradigm of the current run.
Definition at line 522 of file mmkp.hpp.
Referenced by data().

run_id

```
int metrics::Metrics::run_id = 0
```
Execution run identifier (1-indexed).
Definition at line 524 of file mmkp.hpp.
Referenced by data().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

7.8 metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move Struct Reference

Represents a reversible state transition within the neighborhood search.
`#include <metaheuristic.hpp>`

Public Attributes

- bool valid = false

  Indicates if a feasible move was found and applied.
- int device_id = -1

  ID of the primary device involved in the move.
- int old_server_id = -1

  Original server ID of the primary device (`-1` if previously unserved).
- int new_server_id = -1

  Target server ID where the primary device was allocated.
- int swapped_id = -1

  ID of the secondary device evicted in a swap (`-1` if not a swap).
- double delta_cost = 0.0

  Net change in total penalty cost resulting from the move.

7.8.1 Detailed Description

Represents a reversible state transition within the neighborhood search.
Definition at line 34 of file metaheuristic.hpp.

7.8.2 Member Data Documentation

delta_cost

```
double metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::delta_cost = 0.0
```
Net change in total penalty cost resulting from the move.
Definition at line 40 of file metaheuristic.hpp.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), and metaheuristic::anonymous_namesp

device_id

```
int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::device_id = -1
```
ID of the primary device involved in the move.
Definition at line 36 of file metaheuristic.hpp.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), metaheuristic::anonymous_namespace
and metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing().

new_server_id

`int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::new_server_id = -1`
Target server ID where the primary device was allocated.
Definition at line 38 of file metaheuristic.hpp.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), metaheuristic::anonymous_namespace
and metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing().

old_server_id

`int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::old_server_id = -1`
Original server ID of the primary device (`-1` if previously unserved).
Definition at line 37 of file metaheuristic.hpp.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), metaheuristic::anonymous_namespace
and metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing().

swapped_id

`int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::swapped_id = -1`
ID of the secondary device evicted in a swap (`-1` if not a swap).
Definition at line 39 of file metaheuristic.hpp.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), metaheuristic::anonymous_namespace
and metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing().

valid

`bool metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::valid = false`
Indicates if a feasible move was found and applied.
Definition at line 35 of file metaheuristic.hpp.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), and metaheuristic::anonymous_namesp
The documentation for this struct was generated from the following file:

- include/metaheuristic.hpp

7.9 metrics::Metrics::Outputs Struct Reference

Captures the resulting performance and resource utilization metrics after allocation.
`#include <mmkp.hpp>`

Public Member Functions

- Outputs & operator= (const Outputs &other)=default

Public Attributes

- double execution_time = 0.0

  Total algorithm runtime in seconds.
- int devices_served_count = 0

  Total number of served devices.
- int devices_served_ec_count = 0

  Number of devices served by Edge (EC) servers.
- int devices_served_cc_count = 0

  Number of devices served by Cloud (CC) servers.
- int servers_used_count = 0

  Total number of active servers hosting at least one device.
- int servers_used_ec_count = 0

  Number of active Edge (EC) servers.
- int servers_used_cc_count = 0

Number of active Cloud (CC) servers.

- double total_penalty_cost = 0.0

  Total penalty cost of all unserved devices.

7.9.1 Detailed Description

Captures the resulting performance and resource utilization metrics after allocation.
Definition at line 543 of file mmkp.hpp.

7.9.2 Member Function Documentation

operator=()

```
Outputs & metrics::Metrics::Outputs::operator= (
            const Outputs & other )  [default]
```

7.9.3 Member Data Documentation

devices_served_cc_count

```
int metrics::Metrics::Outputs::devices_served_cc_count = 0
```

Number of devices served by Cloud (CC) servers.
Definition at line 547 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

devices_served_count

```
int metrics::Metrics::Outputs::devices_served_count = 0
```

Total number of served devices.
Definition at line 545 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

devices_served_ec_count

```
int metrics::Metrics::Outputs::devices_served_ec_count = 0
```

Number of devices served by Edge (EC) servers.
Definition at line 546 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

execution_time

```
double metrics::Metrics::Outputs::execution_time = 0.0
```

Total algorithm runtime in seconds.
Definition at line 544 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

servers_used_cc_count

```
int metrics::Metrics::Outputs::servers_used_cc_count = 0
```

Number of active Cloud (CC) servers.
Definition at line 550 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

servers_used_count

```
int metrics::Metrics::Outputs::servers_used_count = 0
```

Total number of active servers hosting at least one device.
Definition at line 548 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

servers_used_ec_count

```
int metrics::Metrics::Outputs::servers_used_ec_count = 0
```

Number of active Edge (EC) servers.
Definition at line 549 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().

total_penalty_cost

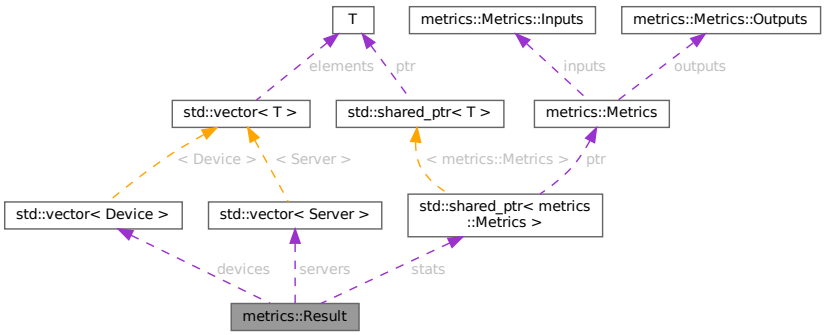
```
double metrics::Metrics::Outputs::total_penalty_cost = 0.0
```

Total penalty cost of all unserved devices.
Definition at line 551 of file mmkp.hpp.
Referenced by metrics::Metrics::build_metrics(), and metrics::Metrics::data().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

7.10 metrics::Result Struct Reference

Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.
`#include <mmkp.hpp>`
Collaboration diagram for metrics::Result:



Public Member Functions

- Result (core::Devices d, core::Servers s)

  Constructs a `Result` state from pre-loaded devices and servers vectors.
- Result (const Result &other)

  Deep-copy constructor cloning both entities and the polymorphic `stats` pointer.
- Result (int num_devices, int num_services, int num_ec, int num_cc, bool fixed_penalty=true)

  Loads (or generates) an instance with the specified dimensions and initializes base metrics.
- Result & operator= (const Result &other)

  Deep-copy assignment operator cloning the polymorphic `stats` object.

Public Attributes

- core::Devices devices

  Vector of all IoT devices in the simulation.
- core::Servers servers

  Vector of all Edge and Cloud servers in the simulation.
- std::shared_ptr< Metrics > stats

  Polymorphic pointer to the run's metrics tracker.

7.10.1 Detailed Description

Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.
Definition at line 928 of file mmkp.hpp.

7.10.2 Constructor & Destructor Documentation

Result() [1/3]

```
metrics::Result::Result (
            core::Devices d,
            core::Servers s )  [inline]
```

Constructs a `Result` state from pre-loaded devices and servers vectors.

Parameters

d	Vector of devices.
s	Vector of servers.

Definition at line 938 of file mmkp.hpp.

Result() [2/3]

```
metrics::Result::Result (
            const Result & other )  [inline]
```

Deep-copy constructor cloning both entities and the polymorphic `stats` pointer.

Parameters

other	Source `Result` instance to copy.

Definition at line 945 of file mmkp.hpp.
References stats.

Result() [3/3]

```
metrics::Result::Result (
            int num_devices,
            int num_services,
            int num_ec,
            int num_cc,
            bool fixed_penalty = true )  [inline]
```

Loads (or generates) an instance with the specified dimensions and initializes base metrics.

Parameters

num_devices	Number of IoT devices to load.
num_services	Number of service templates.
num_ec	Number of Edge Computing servers.
num_cc	Number of Cloud Computing servers.
fixed_penalty	Whether the penalty (cost) of each device is fixed or can vary.

Definition at line 960 of file mmkp.hpp.
References devices, metrics::NONE, servers, and stats.

7.10.3 Member Function Documentation

operator=()

```
Result & metrics::Result::operator= (
            const Result & other )  [inline]
```

Deep-copy assignment operator cloning the polymorphic `stats` object.

Parameters

other	Source `Result` instance.

Returns

Reference to `*this`.

Definition at line 986 of file mmkp.hpp.
References devices, servers, and stats.

7.10.4 Member Data Documentation

devices

```
core::Devices metrics::Result::devices
```

Vector of all IoT devices in the simulation.
Definition at line 929 of file mmkp.hpp.
Referenced by metaheuristic::boot(), metrics::calculate_metrics(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_ber
heuristic::anonymous_namespace{heuristic.hpp}::greedy_random(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_rando
operator=(), Result(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing(), and
exact::anonymous_namespace{exact.hpp}::solver_maximum_cost().

servers

```
core::Servers metrics::Result::servers
```

Vector of all Edge and Cloud servers in the simulation.
Definition at line 930 of file mmkp.hpp.
Referenced by metaheuristic::boot(), metrics::calculate_metrics(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_ber
heuristic::anonymous_namespace{heuristic.hpp}::greedy_random(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_rando
operator=(), Result(), metrics::save_results(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing(),
and exact::anonymous_namespace{exact.hpp}::solver_maximum_cost().

stats

```
std::shared_ptr<Metrics> metrics::Result::stats
```

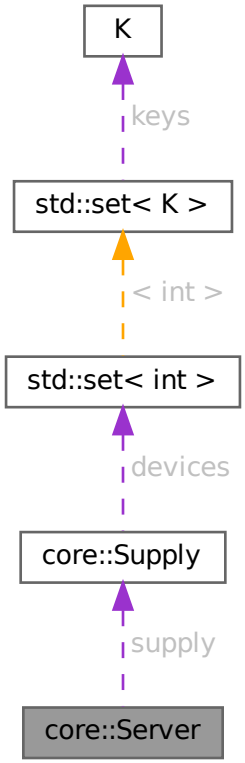
Polymorphic pointer to the run's metrics tracker.
Definition at line 931 of file mmkp.hpp.
Referenced by heuristic::boot(), metaheuristic::boot(), exact::boot(), metrics::calculate_metrics(), heuristic::anonymous_namespace{h
heuristic::anonymous_namespace{heuristic.hpp}::greedy_random(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_rando
operator=(), Result(), Result(), run_simulation(), metrics::save_results(), metaheuristic::anonymous_namespace{metaheuristic.hpp}::s
and exact::anonymous_namespace{exact.hpp}::solver_maximum_cost().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

7.11 core::Server Struct Reference

Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
`#include <mmkp.hpp>`

Collaboration diagram for core::Server:



Public Member Functions

- Server ()

  Default constructor initializing an empty/placeholder server.
- Server (int id_, char type_, double cpu_, double mem_, double sto_)

  Parameterized constructor initializing a server with given capacities.
- bool can_serve (const Device &device) const

  Checks if the server has enough residual capacity to serve a given device.
- bool add_service (Device &device)

  Allocates a device to this server and updates consumed resources.
- bool rmv_service (Device &device)

  Removes a device from this server and releases its consumed resources.

Public Attributes

- int id

  Unique 1-based identifier of the server.
- char type

  Server tier type ('E' for Edge, 'C' for Cloud).
- double cpu

  Maximum total CPU capacity of the server.
- double mem

Maximum total memory capacity of the server.

- double sto

  Maximum total storage capacity of the server.

- Supply supply

  Current resource utilization and set of allocated devices.

7.11.1 Detailed Description

Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
Definition at line 101 of file mmkp.hpp.

7.11.2 Constructor & Destructor Documentation

Server() [1/2]

```
core::Server::Server ( )  [inline]
```

Default constructor initializing an empty/placeholder server.
Definition at line 112 of file mmkp.hpp.

Server() [2/2]

```
core::Server::Server (
            int id_,
            char type_,
            double cpu_,
            double mem_,
            double sto_ )  [inline]
```

Parameterized constructor initializing a server with given capacities.

Parameters

*id_*	Unique 1-based server ID.
*type_*	Server tier character ('E' or 'C').
*cpu_*	Total CPU capacity.
*mem_*	Total memory capacity.
*sto_*	Total storage capacity.

Definition at line 122 of file mmkp.hpp.

7.11.3 Member Function Documentation

add_service()

```
bool core::Server::add_service (
            Device & device )  [inline]
```

Allocates a device to this server and updates consumed resources.

Parameters

device	The device to allocate (modified in-place to mark `served = true` and `server_id`).

Returns

`true` if the device was successfully added; `false` if it was already present.

Definition at line 143 of file mmkp.hpp.
References core::Device::cost, core::Supply::cost, core::Device::cpu, core::Supply::cpu, core::Supply::devices, core::Device::id, id, core::Device::mem, core::Supply::mem, core::Device::served, core::Device::server_id, core::Device::sto, core::Supply::sto, and supply.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), and heuristic::anonymous_namespace{

can_serve()

```
bool core::Server::can_serve (
            const Device & device ) const  [inline]
```

Checks if the server has enough residual capacity to serve a given device.

Parameters

device	The candidate device to be checked.

Returns

`true` if residual CPU, MEM, and STO capacities are sufficient; `false` otherwise.

Definition at line 130 of file mmkp.hpp.
References core::Device::cpu, core::Supply::cpu, cpu, core::Device::mem, core::Supply::mem, mem, core::Device::sto, core::Supply::sto, sto, and supply.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor(), and heuristic::anonymous_namespace{

rmv_service()

```
bool core::Server::rmv_service (
            Device & device )  [inline]
```

Removes a device from this server and releases its consumed resources.

Parameters

device	The device to remove (modified in-place to reset `served = false` and `server_id = -1`).

Returns

`true` if the device was successfully removed; `false` if it was not allocated to this server.

Definition at line 163 of file mmkp.hpp.
References core::Device::cost, core::Supply::cost, core::Device::cpu, core::Supply::cpu, core::Supply::devices, core::Device::id, core::Device::mem, core::Supply::mem, core::Device::served, core::Device::server_id, core::Device::sto, core::Supply::sto, and supply.
Referenced by metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor().

7.11.4 Member Data Documentation

cpu

```
double core::Server::cpu
```

Maximum total CPU capacity of the server.
Definition at line 104 of file mmkp.hpp.
Referenced by can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), and display::show_server().

id

`int core::Server::id`
Unique 1-based identifier of the server.
Definition at line 102 of file mmkp.hpp.
Referenced by add_service(), and display::show_server().

mem

`double core::Server::mem`
Maximum total memory capacity of the server.
Definition at line 105 of file mmkp.hpp.
Referenced by can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), and display::show_server().

sto

`double core::Server::sto`
Maximum total storage capacity of the server.
Definition at line 106 of file mmkp.hpp.
Referenced by can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), and display::show_server().

supply

`Supply core::Server::supply`
Current resource utilization and set of allocated devices.
Definition at line 107 of file mmkp.hpp.
Referenced by add_service(), can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit(), rmv_service(), and display::show_server().

type

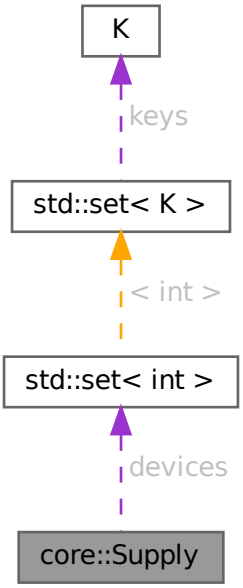
`char core::Server::type`
Server tier type (‘E’`for Edge`,'C'` for Cloud).
Definition at line 103 of file mmkp.hpp.
Referenced by display::show_server().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

7.12 core::Supply Struct Reference

Tracks the cumulative resource consumption and allocated device IDs on a single server.
`#include <mmkp.hpp>`

Collaboration diagram for core::Supply:



Public Member Functions

- Supply ()

  Default constructor initializing zero resource usage.
- Supply (double cost_, double cpu_, double mem_, double sto_)

  Parameterized constructor initializing specific resource usage values.

Public Attributes

- double cost = 0.0

  Total avoided penalty cost of devices currently allocated to the server.
- double cpu = 0.0

  Currently consumed CPU capacity on the server.
- double mem = 0.0

  Currently consumed memory capacity on the server.
- double sto = 0.0

  Currently consumed storage capacity on the server.
- i_set devices

  Set of device IDs currently served by the server.

7.12.1 Detailed Description

Tracks the cumulative resource consumption and allocated device IDs on a single server.
Definition at line 73 of file mmkp.hpp.

7.12.2 Constructor & Destructor Documentation

Supply() [1/2]

```
core::Supply::Supply ( )  [inline]
```

Default constructor initializing zero resource usage.
Definition at line 84 of file mmkp.hpp.

Supply() [2/2]

```
core::Supply::Supply (
            double cost_,
            double cpu_,
            double mem_,
            double sto_ )  [inline]
```

Parameterized constructor initializing specific resource usage values.

Parameters

*cost_*	Initial accumulated cost.
*cpu_*	Initial consumed CPU.
*mem_*	Initial consumed memory.
*sto_*	Initial consumed storage.

Definition at line 93 of file mmkp.hpp.

7.12.3 Member Data Documentation

cost

```
double core::Supply::cost = 0.0
```

Total avoided penalty cost of devices currently allocated to the server.
Definition at line 74 of file mmkp.hpp.
Referenced by core::Server::add_service(), and core::Server::rmv_service().

cpu

```
double core::Supply::cpu = 0.0
```

Currently consumed CPU capacity on the server.
Definition at line 75 of file mmkp.hpp.
Referenced by core::Server::add_service(), core::Server::can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost
core::Server::rmv_service(), and display::show_server().

devices

```
i_set core::Supply::devices
```

Set of device IDs currently served by the server.
Definition at line 79 of file mmkp.hpp.
Referenced by core::Server::add_service(), core::Server::rmv_service(), and display::show_server().

mem

```
double core::Supply::mem = 0.0
```

Currently consumed memory capacity on the server.
Definition at line 76 of file mmkp.hpp.

Referenced by core::Server::add_service(), core::Server::can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost
core::Server::rmv_service(), and display::show_server().

sto

```
double core::Supply::sto = 0.0
```

Currently consumed storage capacity on the server.
Definition at line 77 of file mmkp.hpp.
Referenced by core::Server::add_service(), core::Server::can_serve(), heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost
core::Server::rmv_service(), and display::show_server().
The documentation for this struct was generated from the following file:

- include/mmkp.hpp

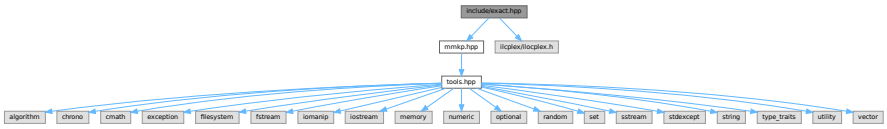
8 File Documentation

8.1 include/exact.hpp File Reference

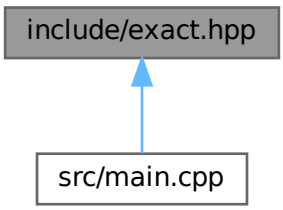
Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX.

```
#include "mmkp.hpp"
#include <ilcplex/ilocplex.h>
```

Include dependency graph for exact.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace exact

  Contains mathematical programming models for exact optimization.
- namespace exact::anonymous_namespace{exact.hpp}

Typedefs

- using exact::NumVar2D = IloArray< IloNumVarArray >

  Type alias for a 2D matrix of CPLEX decision variables.

Functions

- void exact::anonymous_namespace{exact.hpp}::solver_maximum_cost (metrics::Result &state, int time_↩ limit_sec=3600)

  Formulates and solves the exact ILP resource allocation model using CPLEX.
- void exact::boot (metrics::Result &state, int time_limit_sec=3600)

  Bootstraps and executes the configured exact mathematical model.

8.1.1 Detailed Description

Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX.
Provides the mathematical optimization model and solver configuration to solve the Multiple Multidimensional Knapsack Problem (MMKP) for IoT-EC-CC resource allocation.
Definition in file exact.hpp.

8.2 exact.hpp

Go to the documentation of this file.

```
00001 /**
00002  * @file exact.hpp
00003  * @brief Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX.
00004  * @details Provides the mathematical optimization model and solver configuration
00005  * to solve the Multiple Multidimensional Knapsack Problem (MMKP) for IoT-EC-CC
00006  * resource allocation.
00007  */
00008
00009 #pragma once
00010
00011 #include "mmkp.hpp"
00012
00013 #include <ilcplex/ilocplex.h>
00014
00015 /**
00016  * @namespace exact
00017  * @brief Contains mathematical programming models for exact optimization.
00018  * @details Utilizes the IBM ILOG CPLEX optimizer to solve the network resource
00019  * allocation problem to optimality or within a specified time limit, providing
00020  * an exact baseline by maximizing avoided non-service penalties.
00021  */
00022 namespace exact {
00023
00024     /**
00025      * @brief Type alias for a 2D matrix of CPLEX decision variables.
00026      */
00027     using NumVar2D = IloArray<IloNumVarArray>;
00028
00029     namespace {
00030
00031         /**
00032          * @brief Formulates and solves the exact ILP resource allocation model using CPLEX.
00033          * @details Defines binary decision variables \f$x_{d}^{i} \in \{0, 1\}\f$ indicating whether
00034          * device \f$d\f$ is assigned to server \f$i\f$. Maximizes the total utility of served devices
00035          * (avoided SLA penalties) subject to single-server assignment and multidimensional
00036          * resource capacity constraints (CPU, MEM, STO).
00037          *
00038          * @param state The unified simulation state containing devices, servers, and metrics.
00039          * @param time_limit_sec The maximum execution time allowed for the solver in seconds.
00040          */
00041         inline void solver_maximum_cost(metrics::Result& state, int time_limit_sec = 3600) {
00042             core::Devices& devices = state.devices;
00043             core::Servers& servers = state.servers;
00044
00045             auto metrics_ptr = std::dynamic_pointer_cast<metrics::ExactMetrics>(state.stats);
00046             if (!metrics_ptr) {
00047                 std::cerr « "Error: Invalid metrics type for exact solver.\n";
00048                 return;
00049             }
00050
00051             IloEnv env;
00052             try {
00053                 IloModel model(env);
00054                 const size_t num_devices = devices.size();
00055                 const size_t num_servers = servers.size();
00056
00057                 #pragma region VariableDeclaration
00058                 //=========================================================================
00059                 // 1. VARIABLE DECLARATION
00060                 //=========================================================================
```

```
00061
00062                 // x^{i}_{d}: 1 if device d is allocated to server i, 0 otherwise.
00063                 NumVar2D x(env, num_servers);
00064                 for (size_t i = 1; i < num_servers; i++) {
00065                     x[i] = IloNumVarArray(env, num_devices, 0, 1, ILOBOOL);
00066                 }
00067                 #pragma endregion
00068
00069                 #pragma region ObjectiveFunctionAndConstraints
00070                 IloExpr obj(env);
00071
00072                 for (size_t d = 1; d < num_devices; d++) {
00073                     IloExpr c1(env);
00074                     for (size_t i = 1; i < num_servers; i++) {
00075                         c1 += x[i][d];
00076                         obj += devices[d].cost * x[i][d];
00077                     }
00078                     model.add(c1 <= 1);
00079                     c1.end();
00080                 }
00081
00082                 model.add(IloMaximize(env, obj));
00083                 obj.end();
00084
00085                 for (size_t i = 1; i < num_servers; i++) {
00086                     IloExpr c_cpu(env), c_mem(env), c_sto(env);
00087
00088                     for (size_t d = 1; d < num_devices; d++) {
00089                         c_cpu += devices[d].cpu * x[i][d];
00090                         c_mem += devices[d].mem * x[i][d];
00091                         c_sto += devices[d].sto * x[i][d];
00092                     }
00093
00094                     model.add(c_cpu <= servers[i].cpu);
00095                     model.add(c_mem <= servers[i].mem);
00096                     model.add(c_sto <= servers[i].sto);
00097
00098                     c_cpu.end();
00099                     c_mem.end();
00100                     c_sto.end();
00101                 }
00102                 #pragma endregion
00103
00104                 #pragma region SolverConfiguration
00105                 //========================================================================
00106                 // 4. SOLVER CONFIGURATION AND EXECUTION
00107                 //========================================================================
00108
00109                 IloCplex cplex(model);
00110                 cplex.setOut(env.getNullStream());
00111                 cplex.setWarning(env.getNullStream());
00112
00113                 cplex.setParam(IloCplex::Param::Threads, 1);
00114                 cplex.setParam(IloCplex::Param::ClockType, 2);
00115                 cplex.setParam(IloCplex::Param::TimeLimit, time_limit_sec);
00116
00117                 cplex.setParam(IloCplex::Param::WorkMem, 16384);
00118                 cplex.setParam(IloCplex::Param::MIP::Strategy::File, 3);
00119
00120                 // Relative MIP gap tolerance of 1e-3 (0.001)
00121                 cplex.setParam(IloCplex::Param::MIP::Tolerances::MIPGap, 1e-3);
00122                 cplex.setParam(IloCplex::Param::Emphasis::MIP, IloCplex::MIPEmphasisBalanced);
00123
00124                 auto start_chrono = std::chrono::high_resolution_clock::now();
00125                 const IloBool has_solution = cplex.solve();
00126                 auto end_chrono = std::chrono::high_resolution_clock::now();
00127
00128                 metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
      start_chrono).count();
00129
00130                 std::stringstream status;
00131                 status « cplex.getStatus();
00132                 metrics_ptr->status = status.str();
00133                 #pragma endregion
00134
00135                 #pragma region Result
00136                 //========================================================================
00137                 // 5. PARSE RESULTS
00138                 //========================================================================
00139
00140                 if (has_solution) {
00141                     metrics_ptr->objective_function = static_cast<double>(cplex.getObjValue());
00142                     metrics_ptr->gap = static_cast<double>(cplex.getMIPRelativeGap());
00143
00144                     for (size_t i = 1; i < num_servers; i++) {
00145                         for (size_t d = 1; d < num_devices; d++) {
00146                             if (cplex.isExtracted(x[i][d]) && cplex.getValue(x[i][d]) > 0.5) {
```

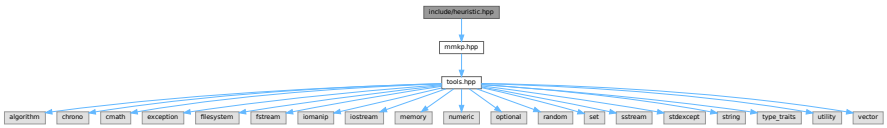
```
00147                                 servers[i].add_service(devices[d]);
00148                             }
00149                         }
00150                     }
00151                 }
00152
00153             } catch (const IloException& e) {
00154                 std::cerr « "CPLEX Error: " « e.getMessage() « "\n";
00155             } catch (const std::exception& e) {
00156                 std::cerr « "Standard Exception: " « e.what() « "\n";
00157             } catch (...) {
00158                 std::cerr « "An unknown error occurred in the CPLEX model.\n";
00159             }
00160             env.end();
00161         }
00162     }
00163
00164     /**
00165      * @brief Bootstraps and executes the configured exact mathematical model.
00166      * @details Reads the specified algorithm identifier from `state.stats` and dispatches
00167      * execution to the corresponding exact solver implementation.
00168      *
00169      * @param state The unified simulation state to be processed.
00170      * @param time_limit_sec The maximum execution time allowed for the exact solver in seconds.
00171      */
00172     inline void boot(metrics::Result& state, int time_limit_sec = 3600) {
00173         if (!state.stats) return;
00174
00175         if (metrics::Algorithm::solver_maximum_cost == state.stats->algorithm) {
00176             solver_maximum_cost(state, time_limit_sec);
00177         } else {
00178             std::cerr « "Error: Unknown mathematical model algorithm type.\n";
00179         }
00180     }
00181 }
```

8.3 include/heuristic.hpp File Reference

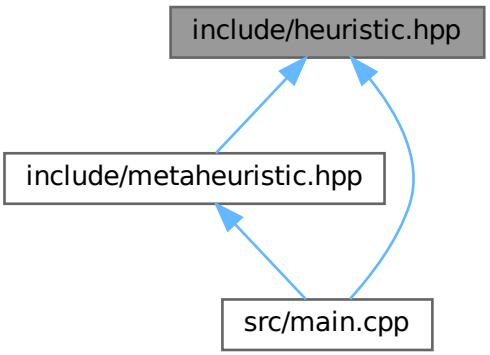
Constructive heuristic algorithms for fast MMKP resource allocation.

`#include "mmkp.hpp"`

Include dependency graph for heuristic.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace heuristic

  Contains constructive heuristic algorithms for network resource allocation.
- namespace heuristic::anonymous_namespace{heuristic.hpp}

Functions

- void heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit (metrics::Result &state)

  Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.
- void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random (metrics::Result &state)

  Executes a fast random baseline resource allocation.
- void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random_fallback (metrics::Result &state)

  Executes a two-stage random fallback resource allocation.
- void heuristic::boot (metrics::Result &state)

  Bootstraps and executes the configured constructive heuristic.

8.3.1 Detailed Description

Constructive heuristic algorithms for fast MMKP resource allocation.
Implements deterministic cost-benefit greedy allocation (`greedy_cost_benefit`) as well as stochastic single-stage (`greedy_random`) and two-stage Edge-first (`greedy_random_fallback`) constructive heuristics.
Definition in file heuristic.hpp.

8.4 heuristic.hpp

Go to the documentation of this file.

```
00001 /**
00002  * @file heuristic.hpp
00003  * @brief Constructive heuristic algorithms for fast MMKP resource allocation.
00004  * @details Implements deterministic cost-benefit greedy allocation (`greedy_cost_benefit`)
00005  * as well as stochastic single-stage (`greedy_random`) and two-stage Edge-first
00006  * (`greedy_random_fallback`) constructive heuristics.
00007  */
00008
00009 #pragma once
00010
00011 #include "mmkp.hpp"
00012
00013 /**
00014  * @namespace heuristic
00015  * @brief Contains constructive heuristic algorithms for network resource allocation.
00016  * @details Implements fast, polynomial-time approximation strategies (Greedy Cost-Benefit,
00017  * Greedy Random, and Greedy Random Fallback) prioritizing low execution times while
00018  * minimizing unserved device penalty costs.
00019  */
00020 namespace heuristic {
00021
00022     namespace {
00023
00024         /**
00025          * @brief Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.
00026          * @details Normalizes device resource demands against total system capacity to compute
00027          * a multidimensional cost-benefit pseudo-utility and sorts candidates once globally.
00028          * Allocates devices greedily per server using prefix-skipping and minimum-demand early
      termination.
00029          *
00030          * @param state The unified simulation result state containing devices, servers, and metrics.
00031          */
00032         inline void greedy_cost_benefit(metrics::Result& state) {
00033             auto metrics_ptr = std::dynamic_pointer_cast<metrics::HeuristicMetrics>(state.stats);
00034             if (!metrics_ptr) {
00035                 std::cerr << "Error: Invalid metrics type for heuristic solver.\n";
00036                 return;
00037             }
00038
00039             core::Devices& devices = state.devices;
00040             core::Servers& servers = state.servers;
00041
00042             auto start_chrono = std::chrono::high_resolution_clock::now();
00043
00044             double total_cpu = 0.0, total_mem = 0.0, total_sto = 0.0;
00045             for (size_t i = 1; i < servers.size(); i++) {
```

```
00046                 total_cpu += servers[i].cpu;
00047                 total_mem += servers[i].mem;
00048                 total_sto += servers[i].sto;
00049             }
00050
00051             id_pair_vector candidates;
00052             candidates.reserve(devices.size());
00053
00054             double min_cpu = std::numeric_limits<double>::max();
00055             double min_mem = std::numeric_limits<double>::max();
00056             double min_sto = std::numeric_limits<double>::max();
00057
00058             for (size_t d = 1; d < devices.size(); d++) {
00059                 core::Device& device = devices[d];
00060                 if (!device.served) {
00061                     double w_cpu = device.cpu / total_cpu;
00062                     double w_mem = device.mem / total_mem;
00063                     double w_sto = device.sto / total_sto;
00064
00065                     double utility = device.cost / (w_cpu + w_mem + w_sto);
00066                     candidates.emplace_back(device.id, utility);
00067
00068                     if (device.cpu < min_cpu) min_cpu = device.cpu;
00069                     if (device.mem < min_mem) min_mem = device.mem;
00070                     if (device.sto < min_sto) min_sto = device.sto;
00071                 }
00072             }
00073
00074             std::sort(candidates.begin(), candidates.end(), [](const id_pair& a, const id_pair& b) {
00075                 return a.second > b.second;
00076             });
00077
00078             size_t start_idx = 0;
00079             const size_t num_candidates = candidates.size();
00080
00081             for (size_t i = 1; i < servers.size() && start_idx < num_candidates; i++) {
00082                 core::Server& server = servers[i];
00083
00084                 for (size_t c = start_idx; c < num_candidates; c++) {
00085                     if ((server.cpu - server.supply.cpu < min_cpu) ||
00086                         (server.mem - server.supply.mem < min_mem) ||
00087                         (server.sto - server.supply.sto < min_sto)) {
00088                         break;
00089                     }
00090
00091                     core::Device& device = devices[candidates[c].first];
00092                     if (!device.served && server.can_serve(device)) {
00093                         server.add_service(device);
00094                     }
00095
00096                     if (c == start_idx && device.served) {
00097                         start_idx++;
00098                     }
00099                 }
00100             }
00101
00102             auto end_chrono = std::chrono::high_resolution_clock::now();
00103             metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
    start_chrono).count();
00104         }
00105
00106         /**
00107          * @brief Executes a fast random baseline resource allocation.
00108          * @details Iterates through all devices and attempts to assign each one to a
00109          * single uniformly selected server. If the selected server lacks capacity,
00110          * the device remains unserved. Metric calculation is deferred.
00111          *
00112          * @param state The unified simulation result state containing devices, servers, and metrics.
00113          */
00114         inline void greedy_random(metrics::Result& state) {
00115             auto metrics_ptr = std::dynamic_pointer_cast<metrics::HeuristicMetrics>(state.stats);
00116             if (!metrics_ptr) {
00117                 std::cerr « "Error: Invalid metrics type for heuristic solver.\n";
00118                 return;
00119             }
00120
00121             core::Devices& devices = state.devices;
00122             core::Servers& servers = state.servers;
00123
00124             int size_servers = static_cast<int>(servers.size()) - 1;
00125
00126             auto start_chrono = std::chrono::high_resolution_clock::now();
00127
00128             for (size_t d = 1; d < devices.size(); d++) {
00129                 core::Device& device = devices[d];
00130
00131                 if (size_servers > 1) {
```

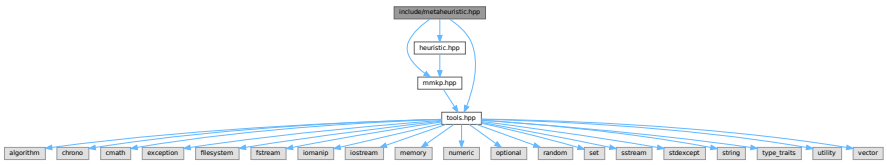
```
00132                     int ec_id = randomness::random_number<int>(1, size_servers);
00133                     if (servers[ec_id].can_serve(device)) {
00134                         servers[ec_id].add_service(device);
00135                     }
00136                 }
00137             }
00138
00139             auto end_chrono = std::chrono::high_resolution_clock::now();
00140             metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
      start_chrono).count();
00141         }
00142
00143         /**
00144          * @brief Executes a two-stage random fallback resource allocation.
00145          * @details Attempts to assign each device to a randomly selected Edge (EC) server first.
00146          * If the selected Edge server lacks capacity, falls back to a randomly selected Cloud (CC)
      server.
00147          *
00148          * @param state The unified simulation result state containing devices, servers, and metrics.
00149          */
00150         inline void greedy_random_fallback(metrics::Result& state) {
00151             auto metrics_ptr = std::dynamic_pointer_cast<metrics::HeuristicMetrics>(state.stats);
00152             if (!metrics_ptr) {
00153                 std::cerr « "Error: Invalid metrics type for heuristic solver.\n";
00154                 return;
00155             }
00156
00157             int ec_count = metrics_ptr->inputs.servers_ec_count;
00158             int cc_count = metrics_ptr->inputs.servers_cc_count;
00159
00160             core::Devices& devices = state.devices;
00161             core::Servers& servers = state.servers;
00162
00163             auto start_chrono = std::chrono::high_resolution_clock::now();
00164
00165             for (size_t d = 1; d < devices.size(); d++) {
00166                 core::Device& device = devices[d];
00167
00168                 if (ec_count > 0) {
00169                     int ec_id = randomness::random_number<int>(1, ec_count);
00170                     if (servers[ec_id].can_serve(device)) {
00171                         servers[ec_id].add_service(device);
00172                         continue;
00173                     }
00174                 }
00175
00176                 if (cc_count > 0) {
00177                     int cc_id = randomness::random_number<int>(ec_count + 1, ec_count + cc_count);
00178                     if (servers[cc_id].can_serve(device)) {
00179                         servers[cc_id].add_service(device);
00180                     }
00181                 }
00182             }
00183
00184             auto end_chrono = std::chrono::high_resolution_clock::now();
00185             metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
      start_chrono).count();
00186         }
00187     }
00188
00189     /**
00190      * @brief Bootstraps and executes the configured constructive heuristic.
00191      * @details Reads the specified algorithm identifier from `state.stats` and dispatches
00192      * execution to the corresponding heuristic solver.
00193      *
00194      * @param state The unified simulation state to be processed.
00195      */
00196     inline void boot(metrics::Result& state) {
00197         if (!state.stats) return;
00198
00199         if (metrics::Algorithm::greedy_cost_benefit == state.stats->algorithm) {
00200             greedy_cost_benefit(state);
00201         } else if (metrics::Algorithm::greedy_random == state.stats->algorithm) {
00202             greedy_random(state);
00203         } else if (metrics::Algorithm::greedy_random_fallback == state.stats->algorithm) {
00204             greedy_random_fallback(state);
00205         } else {
00206             std::cerr « "Error: Unknown heuristic algorithm type.\n";
00207         }
00208     }
00209 }
```

8.5 include/metaheuristic.hpp File Reference

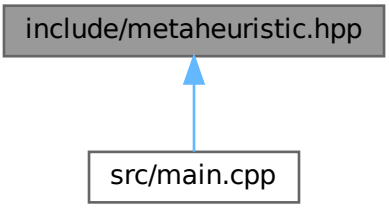
Meta-heuristic optimization algorithms and neighborhood operators for MMKP.

```
#include "mmkp.hpp"
#include "heuristic.hpp"
#include "tools.hpp"
```

Include dependency graph for metaheuristic.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move

  Represents a reversible state transition within the neighborhood search.

Namespaces

- namespace metaheuristic

  Encapsulates meta-heuristic algorithms for network resource allocation.
- namespace metaheuristic::anonymous_namespace{metaheuristic.hpp}

Functions

- Move metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor (core::Devices &devices, core::Servers &servers, int tries=10)

  Applies a shift, insert, or swap neighborhood move to the current state.
- void metaheuristic::anonymous_namespace{metaheuristic.hpp}::revert_neighbor (core::Devices &devices, core::Servers &servers, const Move &move)

  Reverts a previously applied neighborhood move in $O(\log N)$ time.
- void metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing (metrics::Result &state)

  Executes the Simulated Annealing (SA) optimization algorithm.
- void metaheuristic::boot (metrics::Result &state)

  Initializes and launches the selected meta-heuristic algorithm.

Variables

- constexpr int metaheuristic::anonymous_namespace{metaheuristic.hpp}::LOOP = 10

  Number of inner-loop iterations (Markov chain length) executed per temperature level.

8.5.1 Detailed Description

Meta-heuristic optimization algorithms and neighborhood operators for MMKP.
Implements Simulated Annealing (SA) with incremental delta evaluation and shift/insert/swap neighborhood moves to refine initial constructive heuristic solutions.
Definition in file metaheuristic.hpp.

8.6 metaheuristic.hpp

Go to the documentation of this file.

```
00001 /**
00002  * @file metaheuristic.hpp
00003  * @brief Meta-heuristic optimization algorithms and neighborhood operators for MMKP.
00004  * @details Implements Simulated Annealing (SA) with incremental delta evaluation and
00005  * shift/insert/swap neighborhood moves to refine initial constructive heuristic solutions.
00006  */
00007
00008 #pragma once
00009
00010 #include "mmkp.hpp"
00011 #include "heuristic.hpp"
00012 #include "tools.hpp"
00013
00014 /**
00015  * @namespace metaheuristic
00016  * @brief Encapsulates meta-heuristic algorithms for network resource allocation.
00017  * @details Provides trajectory-based optimization techniques, specifically Simulated Annealing,
00018  * to iteratively improve initial heuristic solutions by exploring shift, insertion, and
00019  * 1-for-1 swap neighborhood structures.
00020  */
00021 namespace metaheuristic {
00022
00023     namespace {
00024
00025         /**
00026          * @brief Number of inner-loop iterations (Markov chain length) executed per temperature
      level.
00027          */
00028         constexpr int LOOP = 10;
00029
00030         /**
00031          * @struct Move
00032          * @brief Represents a reversible state transition within the neighborhood search.
00033          */
00034         struct Move {
00035             bool valid = false;      ///< @brief Indicates if a feasible move was found and applied.
00036             int device_id = -1;      ///< @brief ID of the primary device involved in the move.
00037             int old_server_id = -1;  ///< @brief Original server ID of the primary device (`-1` if
      previously unserved).
00038             int new_server_id = -1;  ///< @brief Target server ID where the primary device was
      allocated.
00039             int swapped_id = -1;     ///< @brief ID of the secondary device evicted in a swap (`-1` if
      not a swap).
00040             double delta_cost = 0.0; ///< @brief Net change in total penalty cost resulting from the
      move.
00041         };
00042
00043         /**
00044          * @brief Applies a shift, insert, or swap neighborhood move to the current state.
00045          * @details Selects a random device and attempts to allocate or shift it across random
00046          * servers (including a fallback check on the last server for unserved devices). If direct
00047          * allocation fails for an unserved device, attempts a 1-for-1 swap with a currently served
      device.
00048          *
00049          * @param devices The global state vector of devices.
00050          * @param servers The global state vector of servers.
00051          * @param tries The maximum number of random candidate attempts.
00052          * @return Move A structure containing the applied transition details and cost variation.
00053          */
00054         inline Move apply_neighbor(core::Devices& devices, core::Servers& servers, int tries = 10) {
00055             Move move;
00056             const int max_dev = static_cast<int>(devices.size() - 1);
00057             const int max_srv = static_cast<int>(servers.size() - 1);
00058
00059             move.device_id = randomness::random_number<int>(1, max_dev);
00060             core::Device& device = devices[move.device_id];
```

```
00061             move.old_server_id = device.server_id;
00062
00063             int count = 0;
00064             while (count < tries) {
00065                 int i = (!device.served && count == tries - 1)
00066                         ? max_srv
00067                         : randomness::random_number<int>(1, max_srv);
00068
00069                 if (i == device.server_id) continue;
00070                 count++;
00071
00072                 core::Server& new_server = servers[i];
00073
00074                 if (new_server.can_serve(device)) {
00075                     if (device.served) {
00076                         servers[move.old_server_id].rmv_service(device);
00077                         move.delta_cost = 0.0;
00078                     } else {
00079                         move.delta_cost = -device.cost;
00080                     }
00081
00082                     new_server.add_service(device);
00083                     move.new_server_id = i;
00084                     move.valid = true;
00085                     return move;
00086                 }
00087             }
00088
00089             if (!device.served) {
00090                 for (int t = 0; t < tries; t++) {
00091                     int cand_id = randomness::random_number<int>(1, max_dev);
00092                     core::Device& cand = devices[cand_id];
00093                     if (!cand.served) continue;
00094
00095                     int srv_id = cand.server_id;
00096                     core::Server& target_server = servers[srv_id];
00097
00098                     target_server.rmv_service(cand);
00099                     if (target_server.can_serve(device)) {
00100                         target_server.add_service(device);
00101                         move.new_server_id = srv_id;
00102                         move.swapped_id = cand_id;
00103                         move.delta_cost = cand.cost - device.cost;
00104                         move.valid = true;
00105                         return move;
00106                     }
00107                     target_server.add_service(cand);
00108                 }
00109             }
00110
00111             return move;
00112         }
00113
00114         /**
00115          * @brief Reverts a previously applied neighborhood move in \f$O(\log N)\f$ time.
00116          * @details Restores the exact server allocations and resource supplies prior to
    `apply_neighbor`.
00117          *
00118          * @param devices The global state vector of devices.
00119          * @param servers The global state vector of servers.
00120          * @param move The `Move` structure containing the transition details to undo.
00121          */
00122         inline void revert_neighbor(core::Devices& devices, core::Servers& servers, const Move& move)
    {
00123             core::Device& device = devices[move.device_id];
00124
00125             if (move.new_server_id != -1) {
00126                 servers[move.new_server_id].rmv_service(device);
00127             }
00128             if (move.swapped_id != -1) {
00129                 servers[move.new_server_id].add_service(devices[move.swapped_id]);
00130             }
00131             if (move.old_server_id != -1) {
00132                 servers[move.old_server_id].add_service(device);
00133             }
00134         }
00135
00136         /**
00137          * @brief Executes the Simulated Annealing (SA) optimization algorithm.
00138          * @details Iteratively explores the solution space by applying random neighborhood moves.
00139          * Accepts worsening moves according to the Metropolis criterion \f$\exp(-\Delta / T)\f$
00140          * to escape local optima, cooling down geometrically via factor \f$\alpha\f$ until \f$T \le
    10^{-3}\f$.
00141          * Uses lazy synchronization for tracking the best-found state.
00142          *
00143          * @param state The unified simulation state containing devices, servers, and metrics.
00144          */
```

```
00145         inline void simulated_annealing(metrics::Result& state) {
00146             auto metrics_ptr = std::dynamic_pointer_cast<metrics::MetaHeuristicMetrics>(state.stats);
00147             if (!metrics_ptr) {
00148                 std::cerr << "Error: Invalid metrics type for heuristic solver.\n";
00149                 return;
00150             }
00151 
00152             core::Devices best_devices = state.devices;
00153             core::Servers best_servers = state.servers;
00154 
00155             double best_cost = metrics_ptr->outputs.total_penalty_cost;
00156             double current_cost = best_cost;
00157             bool is_best_synced = true;
00158 
00159             double T = metrics_ptr->temperature;
00160             double alpha = metrics_ptr->alpha;
00161 
00162             auto start_chrono = std::chrono::high_resolution_clock::now();
00163 
00164             while (T > 1e-3) {
00165                 for (int i = 0; i < LOOP; i++) {
00166                     Move move = apply_neighbor(state.devices, state.servers);
00167                     if (!move.valid) continue;
00168 
00169                     double neighbor_cost = current_cost + move.delta_cost;
00170                     double delta = neighbor_cost - current_cost;
00171 
00172                     if (delta < 0.0 || (randomness::random_number(0.0, 1.0) < std::exp(-delta / T))) {
00173                         if (delta > 0.0 && !is_best_synced) {
00174                             revert_neighbor(state.devices, state.servers, move);
00175                             best_devices = state.devices;
00176                             best_servers = state.servers;
00177                             is_best_synced = true;
00178                             if (move.swapped_id != -1) {
00179 
      state.servers[move.new_server_id].rmv_service(state.devices[move.swapped_id]);
00180                             }
00181                             if (move.old_server_id != -1) {
00182 
      state.servers[move.old_server_id].rmv_service(state.devices[move.device_id]);
00183                             }
00184                             if (move.new_server_id != -1) {
00185 
      state.servers[move.new_server_id].add_service(state.devices[move.device_id]);
00186                             }
00187                         }
00188 
00189                         current_cost = neighbor_cost;
00190 
00191                         if (current_cost < best_cost - ZERO) {
00192                             i = 0;
00193                             best_cost = current_cost;
00194                             is_best_synced = false;
00195                         }
00196                     } else {
00197                         revert_neighbor(state.devices, state.servers, move);
00198                     }
00199                 }
00200                 T *= alpha;
00201             }
00202 
00203             if (is_best_synced) {
00204                 state.devices = std::move(best_devices);
00205                 state.servers = std::move(best_servers);
00206             }
00207 
00208             auto end_chrono = std::chrono::high_resolution_clock::now();
00209             metrics_ptr->outputs.execution_time += std::chrono::duration<double>(end_chrono -
      start_chrono).count();
00210         }
00211     }
00212 
00213     /**
00214      * @brief Initializes and launches the selected meta-heuristic algorithm.
00215      * @details Generates an initial solution using the configured constructive base heuristic,
00216      * records its standalone metrics in the heuristic results directory, and then executes
00217      * the meta-heuristic optimization over the resulting state.
00218      *
00219      * @param state The unified simulation state to be optimized.
00220      */
00221     inline void boot(metrics::Result& state) {
00222         if (!state.stats) return;
00223 
00224         auto metrics_ptr = std::dynamic_pointer_cast<metrics::MetaHeuristicMetrics>(state.stats);
00225         if (!metrics_ptr) {
00226             std::cerr << "Error: Invalid metrics type for meta-heuristic solver.\n";
00227             return;
```

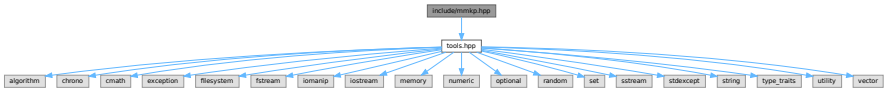
```
00228         }
00229
00230         metrics::Result initial_state = state;
00231         initial_state.stats = std::make_shared<metrics::HeuristicMetrics>(
00232             metrics_ptr->heuristic,
00233             metrics_ptr->run_id,
00234             state.stats
00235         );
00236
00237         heuristic::boot(initial_state);
00238         metrics::calculate_metrics(initial_state);
00239         if (metrics_ptr->heuristic != metrics::Algorithm::greedy_cost_benefit || metrics_ptr->run_id
      == 1) {
00240             metrics::save_results(initial_state, false);
00241         }
00242
00243         state.devices = initial_state.devices;
00244         state.servers = initial_state.servers;
00245         metrics_ptr->inputs = initial_state.stats->inputs;
00246         metrics_ptr->outputs = initial_state.stats->outputs;
00247
00248         if (metrics_ptr->algorithm == metrics::Algorithm::simulated_annealing) {
00249             simulated_annealing(state);
00250         } else {
00251             std::cerr « "Error: Unknown metaheuristic algorithm type.\n";
00252         }
00253     }
00254 }
```

8.7 include/mmkp.hpp File Reference

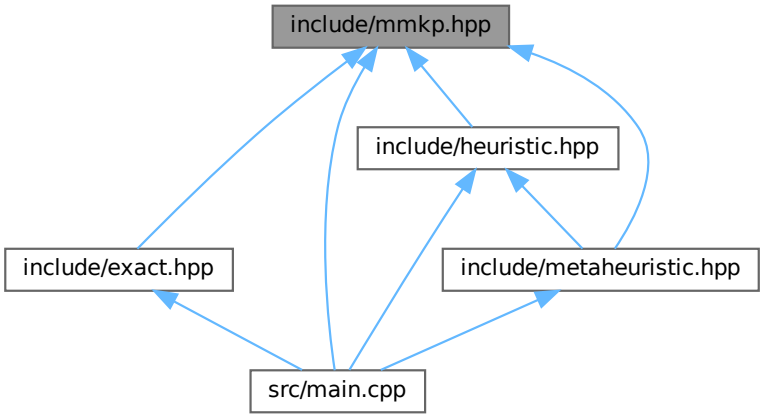
Core data structures, synthetic dataset generators, metric trackers, and console/file display routines.

`#include "tools.hpp"`

Include dependency graph for mmkp.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct core::Device

  Represents an IoT device requesting computational resources for a specific service.

- struct core::Supply

  Tracks the cumulative resource consumption and allocated device IDs on a single server.
- struct core::Server

  Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
- struct metrics::MetricEntry

  Represents a formatted key-value entry for displaying or exporting simulation metrics.
- struct metrics::Metrics

  Polymorphic base structure for tracking, calculating, and saving simulation metrics.
- struct metrics::Metrics::Inputs

  Captures the static input instance dimensions and maximum possible penalty cost.
- struct metrics::Metrics::Outputs

  Captures the resulting performance and resource utilization metrics after allocation.
- struct metrics::ExactMetrics

  Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.
- struct metrics::HeuristicMetrics

  Specialization of `Metrics` for constructive heuristic runs.
- struct metrics::MetaHeuristicMetrics

  Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
- struct metrics::Result

  Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.

Namespaces

- namespace core

  Contains the fundamental domain entities for the MMKP resource allocation problem.
- namespace data::generate

  Generates or updates synthetic datasets for services, IoT devices, and EC/CC servers.
- namespace data
- namespace data::load

  Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.
- namespace metrics

  Provides structures and routines for tracking, formatting, and persisting simulation metrics.
- namespace display

  Utility namespace for formatting and printing entities and ASCII metric tables to console or files.
- namespace display::details

  Internal helper routines for rendering ASCII box-drawing tables.

Typedefs

- using ii_pair = std::pair< int, int >

  Pair of two integers.
- using id_pair = std::pair< int, double >

  Pair of an integer ID and a double value (e.g., utility).
- using i_vector = std::vector< int >

  Vector of integers.
- using s_vector = std::vector< std::string >

  Vector of strings.
- using ii_pair_vector = std::vector< ii_pair >

  Vector of integer-integer pairs.
- using id_pair_vector = std::vector< id_pair >

  Vector of integer-double pairs.
- using i_matrix = std::vector< i_vector >

2D matrix of integers.

- using s_matrix = std::vector< s_vector >

  2D matrix of strings.

- using i_set = std::set< int >

  Ordered set of unique integers.

- using core::Devices = std::vector< Device >

  Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).

- using core::Servers = std::vector< Server >

  Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).

Enumerations

- enum class metrics::Paradigm { metrics::NONE , metrics::EXACT , metrics::HEURISTIC , metrics::METAHEURISTIC }

  Enumerates the optimization paradigms supported by the simulator.

- enum class metrics::Algorithm { metrics::NONE , metrics::solver_maximum_cost , metrics::greedy_cost_benefit , metrics::greedy_random , metrics::greedy_random_fallback , metrics::simulated_annealing }

  Identifies the specific optimization algorithm used in a simulation run.

Functions

- std::vector< std::vector< std::string > > data::generate::services (const int num_services=10, bool fixed_penalty=true)

  Generates or loads network service requirement templates.

- std::vector< std::vector< std::string > > data::generate::devices (const int length, const int num_services=10, bool fixed_penalty=true)

  Generates or loads device request data representing IoT network demands.

- std::vector< std::vector< std::string > > data::generate::servers (const int ec_length=99, const int cc_length=1)

  Generates or loads server infrastructure data for Edge and Cloud nodes.

- core::Devices data::load::devices (const int length, const int num_services=10, bool fixed_penalty=true)

  Loads device data from disk (generating it if absent) into a vector of `core::Device` objects.

- core::Servers data::load::servers (const int ec=99, const int cc=1)

  Loads Edge and Cloud server data from disk (generating it if absent).

- std::string metrics::to_string (Paradigm paradigm)

  Converts a `Paradigm` enum value to its uppercase string representation.

- std::string metrics::to_string (Algorithm algorithm)

  Converts an `Algorithm` enum value to its standardized string identifier.

- void metrics::calculate_metrics (Result &state)

  Computes the final simulation metrics based on the current device-to-server allocations.

- void metrics::save_results (const Result &state, bool show_servers_usage=false)

  Saves the final calculated metrics and ASCII summary tables to the file system.

- void display::show_device (const core::Device &device)

  Displays detailed information of a single device to standard output.

- void display::show_devices (const core::Devices &devices)

  Displays detailed information for all devices in the vector.

- void display::show_server (const core::Server &server)

  Displays capacity and current demand of a single server.

- void display::show_servers (const core::Servers &servers)

  Displays capacity and demand information for all servers.

- void display::details::print_row (std::ostream &out, const std::string &label, const std::string &value)

  Prints a formatted key-value row within the ASCII table.

- void display::details::print_header (std::ostream &out)

  Prints the top or bottom border of the ASCII table.
- void display::details::print_middle (std::ostream &out)

  Prints a section separator line within the ASCII table.
- void display::details::print_title (std::ostream &out, const std::string &title)

  Prints a centered title row within the ASCII table.
- void display::show_metrics_console_or_file (const metrics::Metrics &metrics, std::ostream &out)

  Formats and writes the complete simulation metrics table to an output stream.
- void display::show_metrics (const std::shared_ptr< metrics::Metrics > &ptr)

  Displays the metrics table on the console and appends it to a drawing file.
- void display::show_servers_usage (const core::Servers &servers)

  Prints a summary table of server resource utilization percentages.

Variables

- const std::filesystem::path data::generate::DATA_PATH = std::filesystem::current_path() / "data"

  Path pointing to the root `data/` directory.
- const std::filesystem::path metrics::RESULTS_PATH = std::filesystem::current_path() / "results"

  Path pointing to the root `results/` directory.
- const int display::details::total_width = 72

  Total character width of the formatted ASCII table.
- const int display::details::label_width = 26

  Character width allocated to the left-hand label column.

8.7.1 Detailed Description

Core data structures, synthetic dataset generators, metric trackers, and console/file display routines.
Defines the domain entities (`Device`, `Server`, `Supply`), dataset loading/generation utilities, polymorphic metric collection classes (`Metrics`, `ExactMetrics`, `HeuristicMetrics`, `MetaHeuristicMetrics`), and ASCII table visualization functions for the MMKP IoT-EC-CC simulator.
Definition in file mmkp.hpp.

8.7.2 Typedef Documentation

i_matrix

```
using i_matrix = std::vector<i_vector>
```

2D matrix of integers.
Definition at line 20 of file mmkp.hpp.

i_set

```
using i_set = std::set<int>
```

Ordered set of unique integers.
Definition at line 22 of file mmkp.hpp.

i_vector

```
using i_vector = std::vector<int>
```

Vector of integers.
Definition at line 16 of file mmkp.hpp.

id_pair

```
using id_pair = std::pair<int, double>
```

Pair of an integer ID and a double value (e.g., utility).
Definition at line 15 of file mmkp.hpp.

id_pair_vector

```
using id_pair_vector = std::vector<id_pair>
```
Vector of integer-double pairs.
Definition at line 19 of file mmkp.hpp.

ii_pair

```
using ii_pair = std::pair<int, int>
```
Pair of two integers.
Definition at line 14 of file mmkp.hpp.

ii_pair_vector

```
using ii_pair_vector = std::vector<ii_pair>
```
Vector of integer-integer pairs.
Definition at line 18 of file mmkp.hpp.

s_matrix

```
using s_matrix = std::vector<s_vector>
```
2D matrix of strings.
Definition at line 21 of file mmkp.hpp.

s_vector

```
using s_vector = std::vector<std::string>
```
Vector of strings.
Definition at line 17 of file mmkp.hpp.

8.8 mmkp.hpp

Go to the documentation of this file.

```
00001 /**
00002  * @file mmkp.hpp
00003  * @brief Core data structures, synthetic dataset generators, metric trackers, and console/file
      display routines.
00004  * @details Defines the domain entities (`Device`, `Server`, `Supply`), dataset loading/generation
      utilities,
00005  * polymorphic metric collection classes (`Metrics`, `ExactMetrics`, `HeuristicMetrics`,
      `MetaHeuristicMetrics`),
00006  * and ASCII table visualization functions for the MMKP IoT-EC-CC simulator.
00007  */
00008
00009 #pragma once
00010
00011 #include "tools.hpp"
00012
00013 // Type aliases replacing C-style macros
00014 using ii_pair = std::pair<int, int>;                ///< @brief Pair of two integers.
00015 using id_pair = std::pair<int, double>;             ///< @brief Pair of an integer ID and a double
      value (e.g., utility).
00016 using i_vector = std::vector<int>;                  ///< @brief Vector of integers.
00017 using s_vector = std::vector<std::string>;          ///< @brief Vector of strings.
00018 using ii_pair_vector = std::vector<ii_pair>;        ///< @brief Vector of integer-integer pairs.
00019 using id_pair_vector = std::vector<id_pair>;        ///< @brief Vector of integer-double pairs.
00020 using i_matrix = std::vector<i_vector>;             ///< @brief 2D matrix of integers.
00021 using s_matrix = std::vector<s_vector>;             ///< @brief 2D matrix of strings.
00022 using i_set = std::set<int>;                        ///< @brief Ordered set of unique integers.
00023
00024 /**
00025  * @namespace core
00026  * @brief Contains the fundamental domain entities for the MMKP resource allocation problem.
00027  */
00028 namespace core {
00029
00030     /**
00031      * @struct Device
00032      * @brief Represents an IoT device requesting computational resources for a specific service.
00033      */
00034     struct Device {
00035         int id;             ///< @brief Unique 1-based identifier of the device.
00036         double cost;        ///< @brief SLA penalty cost incurred if the device is not served.
```

```
00037         double cpu;          ///< @brief Required processing capacity (CPU).
00038         double mem;          ///< @brief Required memory capacity (MEM).
00039         double sto;          ///< @brief Required storage capacity (STO).
00040         int tasks;           ///< @brief Number of atomic tasks composing the requested service.
00041         int service;         ///< @brief Identifier of the service template assigned to this device.
00042         bool served = false; ///< @brief Flag indicating whether the device is currently allocated to
      a server.
00043         int server_id = -1;  ///< @brief ID of the server hosting this device (`-1` if unserved).
00044 
00045         /**
00046          * @brief Default constructor initializing an empty/placeholder device.
00047          */
00048         Device() : id(0), cost(0.0), cpu(0.0), mem(0.0), sto(0.0), tasks(0), service(0) {}
00049 
00050         /**
00051          * @brief Parameterized constructor initializing a device with specific resource demands.
00052          * @param id_ Unique identifier of the device.
00053          * @param cost_ Penalty cost if unserved.
00054          * @param cpu_ CPU demand.
00055          * @param mem_ Memory demand.
00056          * @param sto_ Storage demand.
00057          * @param tasks_ Number of service tasks.
00058          * @param service_ Service profile ID.
00059          */
00060         Device(int id_, double cost_, double cpu_, double mem_, double sto_, int tasks_, int service_)
00061             : id(id_), cost(cost_), cpu(cpu_), mem(mem_), sto(sto_), tasks(tasks_), service(service_)
      {}
00062     };
00063 
00064     /**
00065      * @brief Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).
00066      */
00067     using Devices = std::vector<Device>;
00068 
00069     /**
00070      * @struct Supply
00071      * @brief Tracks the cumulative resource consumption and allocated device IDs on a single server.
00072      */
00073     struct Supply {
00074         double cost = 0.0; ///< @brief Total avoided penalty cost of devices currently allocated to
      the server.
00075         double cpu = 0.0;  ///< @brief Currently consumed CPU capacity on the server.
00076         double mem = 0.0;  ///< @brief Currently consumed memory capacity on the server.
00077         double sto = 0.0;  ///< @brief Currently consumed storage capacity on the server.
00078         //double free = 0.0;
00079         i_set devices;     ///< @brief Set of device IDs currently served by the server.
00080 
00081         /**
00082          * @brief Default constructor initializing zero resource usage.
00083          */
00084         Supply() : cost(0.0), cpu(0.0), mem(0.0), sto(0.0) {}
00085 
00086         /**
00087          * @brief Parameterized constructor initializing specific resource usage values.
00088          * @param cost_ Initial accumulated cost.
00089          * @param cpu_ Initial consumed CPU.
00090          * @param mem_ Initial consumed memory.
00091          * @param sto_ Initial consumed storage.
00092          */
00093         Supply(double cost_,  double cpu_, double mem_, double sto_)
00094             : cost(cost_), cpu(cpu_), mem(mem_), sto(sto_) {}
00095     };
00096 
00097     /**
00098      * @struct Server
00099      * @brief Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
00100      */
00101     struct Server {
00102         int id;        ///< @brief Unique 1-based identifier of the server.
00103         char type;     ///< @brief Server tier type (`'E'` for Edge, `'C'` for Cloud).
00104         double cpu;    ///< @brief Maximum total CPU capacity of the server.
00105         double mem;    ///< @brief Maximum total memory capacity of the server.
00106         double sto;    ///< @brief Maximum total storage capacity of the server.
00107         Supply supply; ///< @brief Current resource utilization and set of allocated devices.
00108 
00109         /**
00110          * @brief Default constructor initializing an empty/placeholder server.
00111          */
00112         Server() : id(0), type(' '), cpu(0.0), mem(0.0), sto(0.0), supply() {}
00113 
00114         /**
00115          * @brief Parameterized constructor initializing a server with given capacities.
00116          * @param id_ Unique 1-based server ID.
00117          * @param type_ Server tier character (`'E'` or `'C'`).
00118          * @param cpu_ Total CPU capacity.
00119          * @param mem_ Total memory capacity.
00120          * @param sto_ Total storage capacity.
```

```
00121          */
00122         Server(int id_, char type_, double cpu_, double mem_, double sto_)
00123             : id(id_), type(type_), cpu(cpu_), mem(mem_), sto(sto_) {}
00124 
00125         /**
00126          * @brief Checks if the server has enough residual capacity to serve a given device.
00127          * @param device The candidate device to be checked.
00128          * @return `true` if residual CPU, MEM, and STO capacities are sufficient; `false` otherwise.
00129          */
00130         inline bool can_serve(const Device& device) const {
00131             return (
00132                 (device.cpu <= (cpu - supply.cpu))  &&
00133                 (device.mem <= (mem - supply.mem))  &&
00134                 (device.sto <= (sto - supply.sto))
00135             );
00136         }
00137 
00138         /**
00139          * @brief Allocates a device to this server and updates consumed resources.
00140          * @param device The device to allocate (modified in-place to mark `served = true` and
    `server_id`).
00141          * @return `true` if the device was successfully added; `false` if it was already present.
00142          */
00143         inline bool add_service(Device& device) {
00144             auto result = supply.devices.insert(device.id);
00145             if (!result.second) return false;
00146 
00147             device.served = true;
00148             device.server_id = id;
00149 
00150             supply.cost += device.cost;
00151             supply.cpu += device.cpu;
00152             supply.mem += device.mem;
00153             supply.sto += device.sto;
00154 
00155             return true;
00156         }
00157 
00158         /**
00159          * @brief Removes a device from this server and releases its consumed resources.
00160          * @param device The device to remove (modified in-place to reset `served = false` and
    `server_id = -1`).
00161          * @return `true` if the device was successfully removed; `false` if it was not allocated to
    this server.
00162          */
00163         inline bool rmv_service(Device& device) {
00164             if (supply.devices.erase(device.id) == 0) return false;
00165 
00166             device.served = false;
00167             device.server_id = -1;
00168 
00169             supply.cost -= device.cost;
00170             supply.cpu -= device.cpu;
00171             supply.mem -= device.mem;
00172             supply.sto -= device.sto;
00173 
00174             return true;
00175         }
00176     };
00177 
00178     /**
00179      * @brief Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).
00180      */
00181     using Servers = std::vector<Server>;
00182 }
00183 
00184 /**
00185  * @namespace data::generate
00186  * @brief Generates or updates synthetic datasets for services, IoT devices, and EC/CC servers.
00187  */
00188 namespace data::generate {
00189 
00190     /**
00191      * @brief Path pointing to the root `data/` directory.
00192      */
00193     const std::filesystem::path DATA_PATH = std::filesystem::current_path() / "data";
00194 
00195     /**
00196      * @brief Generates or loads network service requirement templates.
00197      * @details Creates random service profiles with varying task counts, memory, storage,
00198      * and processing demands, persisting them to `data/Services.txt`. If the file already exists,
00199      * reloads the profiles and updates their penalty costs (`COST`) according to `fixed_penalty`.
00200      *
00201      * @param num_services The total number of distinct service profiles to generate.
00202      * @param fixed_penalty If `true`, assigns a unit penalty cost (`1.0`); if `false`, generates
    task-proportional random penalty costs.
00203      * @return A 2D string vector containing the generated or updated services dataset.
```

```
00204      */
00205     inline std::vector<std::vector<std::string>> services(const int num_services = 10, bool
fixed_penalty = true) {
00206         std::filesystem::path service_path = DATA_PATH / "Services.txt";
00207 
00208         if (std::filesystem::exists(service_path)) {
00209             auto loaded = csv::read(service_path.string(),
';').value_or(std::vector<std::vector<std::string>{});
00210 
00211             if (loaded.size() > 1) {
00212                 for (size_t i = 1; i < loaded.size(); i++) {
00213                     int tsk = std::stoi(loaded[i][5]);
00214                     double cos = 1.0;
00215                     if (!fixed_penalty) {
00216                         for (int j = 0; j < tsk; j++) {
00217                             cos += randomness::random_number(ZERO, 2.5);
00218                         }
00219                     }
00220                     loaded[i][1] = convert::to_string(cos, DECIMAL_PLACES);
00221                 }
00222                 csv::write(service_path.string(), loaded, ';');
00223             }
00224             return loaded;
00225         }
00226 
00227         std::vector<std::vector<std::string>> services;
00228         services.push_back({"#", "COST", "CPU", "MEM", "STO", "TSK"});
00229 
00230         for (int i = 1; i <= num_services; i++) {
00231             int tsk = randomness::random_number(1, 4);
00232             double cpu = 0.0, mem = 0.0, sto = 0.0, cos = 1.0;
00233 
00234             for (int j = 0; j < tsk; j++) {
00235                 if (!fixed_penalty) {
00236                     cos += randomness::random_number(ZERO, 2.5);
00237                 }
00238                 cpu += randomness::random_number(ZERO, 2.5);
00239                 mem += randomness::random_number(ZERO, 2.5);
00240                 sto += randomness::random_number(ZERO, 15.0);
00241             }
00242 
00243             services.push_back({
00244                 convert::to_string(i), convert::to_string(cos, DECIMAL_PLACES),
00245                 convert::to_string(cpu, DECIMAL_PLACES), convert::to_string(mem, DECIMAL_PLACES),
convert::to_string(sto, DECIMAL_PLACES),
00246                 convert::to_string(tsk)
00247             });
00248         }
00249 
00250         if (!csv::write(service_path.string(), services, ';')) {
00251             return {};
00252         }
00253         return services;
00254     }
00255 
00256     /**
00257      * @brief Generates or loads device request data representing IoT network demands.
00258      * @details Uniformly samples service requirement profiles from the services dataset
00259      * and assigns them to each device instance in `data/devices/Devices_<length>.txt`. If the file
00260      * already exists, synchronizes each device's penalty cost (`COST`) with its corresponding service
template.
00261      *
00262      * @param length The total number of devices to generate.
00263      * @param num_services The number of underlying service templates to sample from.
00264      * @param fixed_penalty Whether the penalty (cost) of each device is fixed (`1.0`) or variable.
00265      * @return A 2D string vector representing the devices dataset.
00266      */
00267     inline std::vector<std::vector<std::string>> devices(const int length, const int num_services = 10,
bool fixed_penalty = true) {
00268         std::filesystem::path device_file_path = DATA_PATH / "devices" / ("Devices_" +
std::to_string(length) + ".txt");
00269 
00270         std::vector<std::vector<std::string>> data_services = services(num_services, fixed_penalty);
00271         if (data_services.size() <= 1) return {};
00272 
00273         if (std::filesystem::exists(device_file_path)) {
00274             auto loaded = csv::read(device_file_path.string(),
';').value_or(std::vector<std::vector<std::string>{});
00275 
00276             if (loaded.size() > 1) {
00277                 for (size_t i = 1; i < loaded.size(); i++) {
00278                 int svc = std::stoi(loaded[i][6]);
00279                 loaded[i][1] = data_services.at(svc).at(1);
00280             }
00281             csv::write(device_file_path.string(), loaded, ';');
00282             }
00283 
```

```
00284             return loaded;
00285         }
00286 
00287         std::vector<std::vector<std::string>> devices;
00288         devices.push_back({"#", "COST", "CPU", "MEM", "STO", "TSK", "SVC"});
00289 
00290         for (int i = 1; i <= length; i++) {
00291             const auto& source_service_row = data_services.at(randomness::random_number(1,
      static_cast<int>(data_services.size()) - 1));
00292 
00293             devices.push_back({
00294                 convert::to_string(i),     // #
00295                 source_service_row.at(1), // COST
00296                 source_service_row.at(2), // CPU
00297                 source_service_row.at(3), // MEM
00298                 source_service_row.at(4), // STO
00299                 source_service_row.at(5), // TASKS
00300                 source_service_row.at(0)  // SERVICE
00301             });
00302         }
00303 
00304         if (!csv::write(device_file_path.string(), devices, ';')) {
00305             return {};
00306         }
00307         return devices;
00308     }
00309 
00310     /**
00311      * @brief Generates or loads server infrastructure data for Edge and Cloud nodes.
00312      * @details Creates capacity profiles for `ec_length` Edge servers ('E') and `cc_length`
00313      * Cloud servers ('C'), saving the table to `data/servers/servers_<ec>_<cc>.txt`.
00314      *
00315      * @param ec_length The number of Edge Computing servers to generate.
00316      * @param cc_length The number of Cloud Computing servers to generate.
00317      * @return A 2D string vector representing the servers dataset.
00318      */
00319     inline std::vector<std::vector<std::string>> servers(const int ec_length = 99, const int cc_length
      = 1) {
00320         std::filesystem::path server_file_path = DATA_PATH / "servers" / ("servers_" +
      std::to_string(ec_length) + "_" + std::to_string(cc_length) + ".txt");
00321         if (std::filesystem::exists(server_file_path)) {
00322             return csv::read(server_file_path.string(),
      ';').value_or(std::vector<std::vector<std::string>>{});
00323         }
00324 
00325         std::vector<std::vector<std::string>> servers;
00326         servers.push_back({"#", "Type", "CPU", "MEM", "STO"});
00327         int size = ec_length + cc_length;
00328         double cpu, mem, sto;
00329 
00330         for (int it = 1; it <= size; it++) {
00331             if (it <= ec_length) {
00332                 cpu = randomness::random_number(2.5, 25.0);
00333                 mem = randomness::random_number(2.5, 125.0);
00334                 sto = randomness::random_number(15.0, 1000.0);
00335             } else {
00336                 double factor = 2.0;
00337                 cpu = randomness::random_number(100000.0/factor, 200000.0/factor);
00338                 mem = randomness::random_number(51200.0/factor, 102800.0/factor);
00339                 sto = randomness::random_number(500000.0/factor, 1000000.0/factor);
00340             }
00341 
00342             servers.push_back({
00343                     convert::to_string(it),        // IDX
00344                     (it <= ec_length) ? "E" : "C", // TYPE
00345                     convert::to_string(cpu, DECIMAL_PLACES),       // CPU
00346                     convert::to_string(mem, DECIMAL_PLACES),       // MEM
00347                     convert::to_string(sto, DECIMAL_PLACES)        // STO
00348                 });
00349         }
00350 
00351         if (!csv::write(server_file_path.string(), servers, ';')) {
00352             return {};
00353         }
00354         return servers;
00355     }
00356 }
00357 
00358 /**
00359  * @namespace data::load
00360  * @brief Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.
00361  */
00362 namespace data::load {
00363 
00364     /**
00365      * @brief Loads device data from disk (generating it if absent) into a vector of `core::Device`
      objects.
```

```
00366      * @param length The total number of devices to load.
00367      * @param num_services The number of distinct services to generate if absent. Defaults to 10.
00368      * @param fixed_penalty Whether the penalty (cost) of each device is fixed or can vary. Defaults
      to true.
00369      * @return A 1-indexed `core::Devices` vector containing the parsed device instances.
00370      */
00371     inline core::Devices devices(const int length, const int num_services = 10, bool fixed_penalty =
      true) {
00372         std::vector<std::vector<std::string>> load_devices = data::generate::devices(length,
      num_services, fixed_penalty);
00373         if (load_devices.size() <= 1) {
00374             std::cerr << "Error: Device data file not found or is empty.\n";
00375             return {};
00376         }
00377
00378         core::Devices devices;
00379         devices.reserve(length + 1);
00380         devices.emplace_back(); // Placeholder for 1-based indexing.
00381
00382         for (const auto& row : load_devices) {
00383             if (row.at(0) == "#") continue;
00384             try {
00385                 devices.emplace_back(
00386                     std::stoi(row.at(0)),  // #
00387                     std::stod(row.at(1)),  // COST
00388                     std::stod(row.at(2)),  // CPU
00389                     std::stod(row.at(3)),  // MEM
00390                     std::stod(row.at(4)),  // STO
00391                     std::stoi(row.at(5)),  // TASKS
00392                     std::stoi(row.at(6))   // SERVICE
00393                 );
00394             } catch (const std::exception& e) {
00395                 std::cerr << "Error parsing device data: " << e.what() << "\n";
00396                 break;
00397             }
00398         }
00399
00400         return devices;
00401     }
00402
00403     /**
00404      * @brief Loads Edge and Cloud server data from disk (generating it if absent).
00405      * @param ec The number of Edge Computing (EC) servers to load.
00406      * @param cc The number of Cloud Computing (CC) servers to load.
00407      * @return A 1-indexed `core::Servers` vector containing both EC and CC server instances.
00408      */
00409     inline core::Servers servers(const int ec = 99, const int cc = 1) {
00410         std::vector<std::vector<std::string>> load_servers = data::generate::servers(ec, cc);
00411         if (load_servers.size() <= 1) {
00412             std::cerr << "Error: Server data file(s) not found or are empty.\n";
00413             return {};
00414         }
00415
00416         core::Servers servers;
00417         servers.reserve(ec + cc + 1);
00418         servers.emplace_back(); // Placeholder for 1-based indexing.
00419
00420         for (const auto& row : load_servers) {
00421             if (row.at(0) == "#") continue;
00422             try {
00423                 servers.emplace_back(
00424                     std::stoi(row.at(0)),  // #
00425                     row.at(1).front(),     // TYPE (extracting char safely)
00426                     std::stod(row.at(2)),  // CPU
00427                     std::stod(row.at(3)),  // MEM
00428                     std::stod(row.at(4))   // STO
00429                 );
00430             } catch (const std::exception& e) {
00431                 std::cerr << "Error parsing server data: " << e.what() << "\n";
00432                 break;
00433             }
00434         }
00435
00436         return servers;
00437     }
00438 }
00439
00440 /**
00441  * @namespace metrics
00442  * @brief Provides structures and routines for tracking, formatting, and persisting simulation
      metrics.
00443  */
00444 namespace metrics {
00445
00446     /**
00447      * @brief Path pointing to the root `results/` directory.
00448      */
```

```
00449     const std::filesystem::path RESULTS_PATH = std::filesystem::current_path() / "results";
00450
00451     /**
00452      * @enum Paradigm
00453      * @brief Enumerates the optimization paradigms supported by the simulator.
00454      */
00455     enum class Paradigm {
00456         NONE,           ///< @brief Uninitialized paradigm.
00457         EXACT,          ///< @brief Exact mathematical programming (ILP via CPLEX).
00458         HEURISTIC,      ///< @brief Constructive heuristic algorithms.
00459         METAHEURISTIC   ///< @brief Meta-heuristic optimization algorithms.
00460     };
00461
00462     /**
00463      * @brief Converts a `Paradigm` enum value to its uppercase string representation.
00464      * @param paradigm The `Paradigm` value to convert.
00465      * @return String representation of the paradigm.
00466      */
00467     inline std::string to_string(Paradigm paradigm) {
00468         switch (paradigm) {
00469             case Paradigm::EXACT:         return "EXACT";
00470             case Paradigm::HEURISTIC:     return "HEURISTIC";
00471             case Paradigm::METAHEURISTIC: return "METAHEURISTIC";
00472             default:                       return "NONE";
00473         }
00474     }
00475
00476     /**
00477      * @enum Algorithm
00478      * @brief Identifies the specific optimization algorithm used in a simulation run.
00479      */
00480     enum class Algorithm {
00481         NONE,                   ///< @brief Uninitialized algorithm.
00482         solver_maximum_cost,    ///< @brief Exact ILP solver maximizing avoided penalty cost.
00483         greedy_cost_benefit,    ///< @brief Deterministic surrogate cost-benefit greedy heuristic.
00484         greedy_random,          ///< @brief Single-stage uniform random greedy heuristic.
00485         greedy_random_fallback, ///< @brief Two-stage Edge-first random greedy heuristic with Cloud
      fallback.
00486         simulated_annealing     ///< @brief Simulated Annealing meta-heuristic.
00487     };
00488
00489     /**
00490      * @brief Converts an `Algorithm` enum value to its standardized string identifier.
00491      * @param algorithm The `Algorithm` value to convert.
00492      * @return String representation of the algorithm.
00493      */
00494     inline std::string to_string(Algorithm algorithm) {
00495         switch (algorithm) {
00496             case Algorithm::solver_maximum_cost:    return "SOLVER_MAXIMUM_COST";
00497             case Algorithm::greedy_cost_benefit:    return "GREEDY_CB";
00498             case Algorithm::greedy_random:          return "GREEDY_RANDOM";
00499             case Algorithm::greedy_random_fallback: return "GREEDY_RANDOM_FALLBACK";
00500             case Algorithm::simulated_annealing:    return "SA";
00501             default:                                return "NONE";
00502         }
00503     }
00504
00505     /**
00506      * @struct MetricEntry
00507      * @brief Represents a formatted key-value entry for displaying or exporting simulation metrics.
00508      */
00509     struct MetricEntry {
00510         std::string category;      ///< @brief Section category header (e.g., `"GENERAL"`,
      `"DEVICES"`, `"COSTS"`).
00511         std::string header;        ///< @brief Short column header name for CSV output.
00512         std::string label;         ///< @brief Human-readable row label for console/ASCII table
      output.
00513         std::string file_value;    ///< @brief Raw formatted string value for CSV persistence.
00514         std::string console_value; ///< @brief Decorated string value (with units/percentages) for
      ASCII tables.
00515     };
00516
00517     /**
00518      * @struct Metrics
00519      * @brief Polymorphic base structure for tracking, calculating, and saving simulation metrics.
00520      */
00521     struct Metrics {
00522         Paradigm paradigm = Paradigm::NONE;    ///< @brief Optimization paradigm of the current run.
00523         Algorithm algorithm = Algorithm::NONE; ///< @brief Algorithm executed in the current run.
00524         int run_id = 0;                        ///< @brief Execution run identifier (1-indexed).
00525
00526         /**
00527          * @struct Inputs
00528          * @brief Captures the static input instance dimensions and maximum possible penalty cost.
00529          */
00530         struct Inputs {
00531             int devices_count = 0;       ///< @brief Total number of IoT devices in the instance.
```

```
00532             int servers_ec_count = 0;     ///< @brief Total number of Edge Computing (EC) servers.
00533             int servers_cc_count = 0;     ///< @brief Total number of Cloud Computing (CC) servers.
00534             double max_penalty_cost = 0; ///< @brief Sum of penalty costs across all devices if none
      are served.
00535
00536             Inputs& operator=(const Inputs& other) = default;
00537         } inputs; ///< @brief Instance input parameters.
00538
00539         /**
00540          * @struct Outputs
00541          * @brief Captures the resulting performance and resource utilization metrics after
      allocation.
00542          */
00543         struct Outputs {
00544             double execution_time = 0.0;      ///< @brief Total algorithm runtime in seconds.
00545             int    devices_served_count = 0;  ///< @brief Total number of served devices.
00546             int    devices_served_ec_count = 0; ///< @brief Number of devices served by Edge (EC)
      servers.
00547             int    devices_served_cc_count = 0; ///< @brief Number of devices served by Cloud (CC)
      servers.
00548             int    servers_used_count = 0;    ///< @brief Total number of active servers hosting at
      least one device.
00549             int    servers_used_ec_count = 0; ///< @brief Number of active Edge (EC) servers.
00550             int    servers_used_cc_count = 0; ///< @brief Number of active Cloud (CC) servers.
00551             double total_penalty_cost = 0.0;  ///< @brief Total penalty cost of all unserved devices.
00552
00553             Outputs& operator=(const Outputs& other) = default;
00554         } outputs; ///< @brief Simulation output metrics.
00555
00556         /**
00557          * @brief Constructs a `Metrics` object with paradigm, algorithm, and run ID.
00558          * @param paradigm_ Optimization paradigm.
00559          * @param algorithm_name_ Executed algorithm.
00560          * @param run_id Current run number.
00561          */
00562         Metrics(Paradigm paradigm_, Algorithm algorithm_name_, int run_id)
00563             : paradigm(paradigm_), algorithm(algorithm_name_), run_id(run_id) {}
00564
00565         /**
00566          * @brief Constructs a `Metrics` object copying input dimensions from a base instance.
00567          * @param paradigm_ Optimization paradigm.
00568          * @param algorithm_name_ Executed algorithm.
00569          * @param run_id Current run number.
00570          * @param base Shared pointer to a base `Metrics` instance containing input parameters.
00571          */
00572         Metrics(Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const
      std::shared_ptr<Metrics>& base)
00573             : paradigm(paradigm_), algorithm(algorithm_name_), run_id(run_id) {
00574                 if (base) inputs = base->inputs;
00575             }
00576
00577         /**
00578          * @brief Constructs a `Metrics` object with explicit instance counts.
00579          * @param paradigm_ Optimization paradigm.
00580          * @param algorithm_name_ Executed algorithm.
00581          * @param run_id Current run number.
00582          * @param d Total device count.
00583          * @param s_ec Edge server count.
00584          * @param s_cc Cloud server count.
00585          */
00586         Metrics(Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)
00587             : paradigm(paradigm_), algorithm(algorithm_name_), run_id(run_id), inputs({d, s_ec, s_cc,
      0}) {}
00588
00589         /**
00590          * @brief Copy-like constructor initializing from an existing `Metrics` pointer and updating
      `run_id`.
00591          * @param base Shared pointer to an existing `Metrics` object.
00592          * @param run_id New run identifier.
00593          */
00594         Metrics(const std::shared_ptr<Metrics>& base, int run_id)
00595             : paradigm(base ? base->paradigm : Paradigm::NONE),
00596               algorithm(base ? base->algorithm : Algorithm::NONE),
00597               run_id(run_id) {
00598             if (base) {
00599                 inputs = base->inputs;
00600                 outputs = base->outputs;
00601             }
00602         }
00603
00604         /**
00605          * @brief Virtual destructor for safe polymorphic cleanup.
00606          */
00607         virtual ~Metrics() = default;
00608
00609         /**
00610          * @brief Creates a polymorphic deep copy of this `Metrics` instance.
```

```
00611          * @return A `std::shared_ptr<Metrics>` pointing to the cloned instance.
00612          */
00613         virtual std::shared_ptr<Metrics> clone() const {
00614             return std::shared_ptr<Metrics>(new Metrics(*this));
00615         }
00616
00617         /**
00618          * @brief Returns the output directory path corresponding to the current paradigm.
00619          * @return Filesystem path `results/<PARADIGM>`.
00620          */
00621         virtual std::filesystem::path get_base_directory_path() const {
00622             return RESULTS_PATH / to_string(this->paradigm);
00623         }
00624
00625         /**
00626          * @brief Builds the list of formatted metric entries for console and file reporting.
00627          * @return Vector of `MetricEntry` records.
00628          */
00629         virtual std::vector<MetricEntry> build_metrics() const {
00630             int current_total = inputs.devices_count;
00631             int total_servers = inputs.servers_ec_count + inputs.servers_cc_count;
00632
00633             std::vector<MetricEntry> m;
00634
00635             m.push_back({"GENERAL", "ExeTime", "Execution Time (s)",
    convert::to_string(outputs.execution_time, DECIMAL_PLACES), convert::to_string(outputs.execution_time,
    DECIMAL_PLACES)});
00636
00637             m.push_back({"DEVICES", "Devices", "Total", convert::to_string(current_total),
    std::to_string(current_total)});
00638             m.push_back({"DEVICES", "DServed", "Served",
    convert::to_percentage_string(outputs.devices_served_count, current_total),
    std::to_string(outputs.devices_served_count) + " (" +
    convert::to_percentage_string(outputs.devices_served_count, current_total) + "%)"});
00639             m.push_back({"DEVICES", "DServedEC", "  - on EC",
    convert::to_percentage_string(outputs.devices_served_ec_count, outputs.devices_served_count),
    std::to_string(outputs.devices_served_ec_count) + " (" +
    convert::to_percentage_string(outputs.devices_served_ec_count, outputs.devices_served_count) + "% of
    served)"});
00640             m.push_back({"DEVICES", "DServedCC", "  - on CC",
    convert::to_percentage_string(outputs.devices_served_cc_count, outputs.devices_served_count),
    std::to_string(outputs.devices_served_cc_count) + " (" +
    convert::to_percentage_string(outputs.devices_served_cc_count, outputs.devices_served_count) + "% of
    served)"});
00641
00642             m.push_back({"SERVERS", "Servers", "Total", convert::to_string(total_servers),
    std::to_string(total_servers) + " (" + std::to_string(inputs.servers_ec_count) + " EC + " +
    std::to_string(inputs.servers_cc_count) + " CC)"});
00643             m.push_back({"SERVERS", "SUsed", "Used",
    convert::to_percentage_string(outputs.servers_used_count, total_servers),
    std::to_string(outputs.servers_used_count) + " (" +
    convert::to_percentage_string(outputs.servers_used_count, total_servers) + "%)"});
00644             m.push_back({"SERVERS", "SUsedEC", "  - Used EC",
    convert::to_percentage_string(outputs.servers_used_ec_count, inputs.servers_ec_count),
    std::to_string(outputs.servers_used_ec_count) + " (" +
    convert::to_percentage_string(outputs.servers_used_ec_count, inputs.servers_ec_count) + "% of EC)"});
00645             m.push_back({"SERVERS", "SUsedCC", "  - Used CC",
    convert::to_percentage_string(outputs.servers_used_cc_count, inputs.servers_cc_count),
    std::to_string(outputs.servers_used_cc_count) + " (" +
    convert::to_percentage_string(outputs.servers_used_cc_count, inputs.servers_cc_count) + "% of CC)"});
00646
00647             m.push_back({"COSTS", "MaxCost", "MAX PENALTY COST",
    convert::to_string(inputs.max_penalty_cost, DECIMAL_PLACES),
    convert::to_string(inputs.max_penalty_cost, DECIMAL_PLACES)});
00648             m.push_back({"COSTS", "PenaltyCost", "PENALTY COST",
    convert::to_string(outputs.total_penalty_cost, DECIMAL_PLACES),
    convert::to_string(outputs.total_penalty_cost, DECIMAL_PLACES)});
00649
00650             return m;
00651         }
00652
00653         /**
00654          * @brief Returns the CSV header columns for the current metric type.
00655          * @return Vector of column header strings.
00656          */
00657         virtual std::vector<std::string> get_header() const {
00658             return {
00659                 "Paradigm", "Algorithm", "Run", "Devices", "Servers_EC", "Servers_CC",
00660                 "Max_cost", "Execution_time", "Devices_served", "Devices_served_ec",
00661                 "Devices_served_cc", "Servers_used", "Servers_used_ec", "Servers_used_cc",
    "Total_cost"
00662             };
00663         }
00664
00665         /**
00666          * @brief Collects the metric values matching the columns of `get_header()`.
00667          * @return Vector of formatted string values for a single CSV row.
```

```
00668          */
00669         virtual std::vector<std::string> data() const {
00670             return {
00671                 to_string(paradigm),
00672                 to_string(algorithm),
00673                 std::to_string(run_id),
00674                 std::to_string(inputs.devices_count),
00675                 std::to_string(inputs.servers_ec_count),
00676                 std::to_string(inputs.servers_cc_count),
00677                 convert::to_string(inputs.max_penalty_cost, DECIMAL_PLACES),
00678                 convert::to_string(outputs.execution_time, DECIMAL_PLACES),
00679                 std::to_string(outputs.devices_served_count),
00680                 std::to_string(outputs.devices_served_ec_count),
00681                 std::to_string(outputs.devices_served_cc_count),
00682                 std::to_string(outputs.servers_used_count),
00683                 std::to_string(outputs.servers_used_ec_count),
00684                 std::to_string(outputs.servers_used_cc_count),
00685                 convert::to_string(outputs.total_penalty_cost, DECIMAL_PLACES)
00686             };
00687         }
00688 
00689         /**
00690          * @brief Appends the current simulation metrics row to the corresponding algorithm CSV file.
00691          */
00692         inline void save_to_file() const {
00693             std::filesystem::path dir_path = this->get_base_directory_path();
00694             std::filesystem::path result_path = dir_path / (to_string(this->algorithm) + ".txt");
00695 
00696             std::vector<std::vector<std::string>> content;
00697             if (!std::filesystem::exists(result_path)) {
00698                 content.push_back(this->get_header());
00699             }
00700             content.push_back(this->data());
00701             file_manager::csv::append(result_path.string(), content, ';');
00702         }
00703     };
00704 
00705     /**
00706      * @struct ExactMetrics
00707      * @brief Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality
      gap.
00708      */
00709     struct ExactMetrics : public Metrics {
00710         std::string status = "Unknown"; ///< @brief Termination status reported by CPLEX (e.g.,
      `"Optimal"`).
00711         double objective_function = 0.0; ///< @brief Objective function value achieved by the solver.
00712         double gap = 1.0;                ///< @brief Relative MIP optimality gap.
00713 
00714         /**
00715          * @brief Constructs an `ExactMetrics` object copying input dimensions from a base instance.
00716          * @param algorithm_name_ Exact algorithm identifier.
00717          * @param run_id_ Execution run number.
00718          * @param base Base metrics pointer containing input parameters.
00719          */
00720         ExactMetrics(Algorithm algorithm_name_, int run_id_, const std::shared_ptr<Metrics>& base)
00721             : Metrics(Paradigm::EXACT, algorithm_name_, run_id_, base) {
00722                 if (base) inputs = base->inputs;
00723             }
00724 
00725         /**
00726          * @brief Constructs an `ExactMetrics` object with explicit instance dimensions.
00727          * @param algorithm_name_ Exact algorithm identifier.
00728          * @param run_id Execution run number.
00729          * @param d Total device count.
00730          * @param s_ec Edge server count.
00731          * @param s_cc Cloud server count.
00732          */
00733         ExactMetrics(Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc) :
00734             Metrics(Paradigm::EXACT, algorithm_name_, run_id, d, s_ec, s_cc) {}
00735 
00736         /**
00737          * @brief Constructs an `ExactMetrics` object from a base pointer and run ID.
00738          * @param base Base metrics pointer.
00739          * @param run_id Execution run number.
00740          */
00741         ExactMetrics(const std::shared_ptr<Metrics>& base, int run_id) :
00742             Metrics(base, run_id) {}
00743 
00744         /**
00745          * @brief Clones this `ExactMetrics` instance polymorphically.
00746          * @return Shared pointer to the cloned `ExactMetrics` object.
00747          */
00748         std::shared_ptr<Metrics> clone() const override {
00749             return std::shared_ptr<ExactMetrics>(new ExactMetrics(*this));
00750         }
00751 
00752         /**
```

```
00753          * @brief Appends CPLEX solver statistics (`Status`, `OF`, `GAP`) to the base metric entries.
00754          * @return Vector of `MetricEntry` records.
00755          */
00756         std::vector<MetricEntry> build_metrics() const override {
00757             auto m = Metrics::build_metrics();
00758             m.push_back({"SOLVER STATS", "Status", "Solver Status", status, status});
00759             m.push_back({"SOLVER STATS", "OF", "Objective Function (OF)",
      convert::to_string(objective_function, DECIMAL_PLACES), convert::to_string(objective_function,
      DECIMAL_PLACES)});
00760             m.push_back({"SOLVER STATS", "GAP", "MIP Gap", convert::to_string(gap, DECIMAL_PLACES),
      convert::to_percentage_string(gap, 1.0) + "%"});
00761             return m;
00762         }
00763
00764         /**
00765          * @brief Appends exact solver column headers (`Status`, `Obj_Function`, `Gap`) to the CSV
      header.
00766          * @return Vector of column header strings.
00767          */
00768         std::vector<std::string> get_header() const override {
00769             auto h = Metrics::get_header();
00770             h.push_back("Status");
00771             h.push_back("Obj_Function");
00772             h.push_back("Gap");
00773             return h;
00774         }
00775
00776         /**
00777          * @brief Appends exact solver values to the CSV row data.
00778          * @return Vector of formatted string values.
00779          */
00780         std::vector<std::string> data() const override {
00781             auto d = Metrics::data();
00782             d.push_back(status);
00783             d.push_back(convert::to_string(objective_function, DECIMAL_PLACES));
00784             d.push_back(convert::to_string(gap, DECIMAL_PLACES));
00785             return d;
00786         }
00787     };
00788
00789     /**
00790      * @struct HeuristicMetrics
00791      * @brief Specialization of `Metrics` for constructive heuristic runs.
00792      */
00793     struct HeuristicMetrics : public Metrics {
00794         /**
00795          * @brief Constructs a `HeuristicMetrics` object copying input dimensions from a base
      instance.
00796          * @param algorithm_name_ Heuristic algorithm identifier.
00797          * @param run_id_ Execution run number.
00798          * @param base Base metrics pointer containing input parameters.
00799          */
00800         HeuristicMetrics(Algorithm algorithm_name_, int run_id_, const std::shared_ptr<Metrics>& base)
      :
00801             Metrics(Paradigm::HEURISTIC, algorithm_name_, run_id_, base) {
00802                 if (base) inputs = base->inputs;
00803             }
00804
00805         /**
00806          * @brief Constructs a `HeuristicMetrics` object with explicit instance dimensions.
00807          * @param algorithm_name_ Heuristic algorithm identifier.
00808          * @param run_id Execution run number.
00809          * @param d Total device count.
00810          * @param s_ec Edge server count.
00811          * @param s_cc Cloud server count.
00812          */
00813         HeuristicMetrics(Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc) :
00814             Metrics(Paradigm::HEURISTIC, algorithm_name_, run_id, d, s_ec, s_cc) {}
00815
00816         /**
00817          * @brief Constructs a `HeuristicMetrics` object from a base pointer and run ID.
00818          * @param base Base metrics pointer.
00819          * @param run_id Execution run number.
00820          */
00821         HeuristicMetrics(const std::shared_ptr<Metrics>& base, int run_id) :
00822             Metrics(base, run_id) {}
00823
00824         /**
00825          * @brief Clones this `HeuristicMetrics` instance polymorphically.
00826          * @return Shared pointer to the cloned `HeuristicMetrics` object.
00827          */
00828         std::shared_ptr<Metrics> clone() const override {
00829             return std::shared_ptr<HeuristicMetrics>(new HeuristicMetrics(*this));
00830         }
00831     };
00832
00833     /**
```

```
00834      * @struct MetaHeuristicMetrics
00835      * @brief Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
00836      */
00837     struct MetaHeuristicMetrics : public Metrics {
00838         Algorithm heuristic = Algorithm::NONE; ///< @brief Constructive heuristic used to generate the initial solution.
00839         double temperature = 0.0;              ///< @brief Initial temperature \f$T_0\f$ for Simulated Annealing.
00840         double alpha = 0.0;                    ///< @brief Cooling rate factor \f$\alpha\f$ for Simulated Annealing.
00841 
00842         /**
00843          * @brief Constructs a `MetaHeuristicMetrics` object copying inputs from a base instance.
00844          * @param algorithm_name_ Meta-heuristic algorithm identifier.
00845          * @param run_id_ Execution run number.
00846          * @param base Base metrics pointer containing input parameters.
00847          * @param temperature_ Initial temperature \f$T_0\f$.
00848          * @param alpha_ Geometric cooling factor \f$\alpha\f$.
00849          * @param heuristic_ Initial constructive heuristic algorithm.
00850          */
00851         MetaHeuristicMetrics(Algorithm algorithm_name_, int run_id_, const std::shared_ptr<Metrics>& base, double temperature_, double alpha_, Algorithm heuristic_)
00852             : Metrics(Paradigm::METAHEURISTIC, algorithm_name_, run_id_, base), temperature(temperature_), alpha(alpha_), heuristic(heuristic_) {
00853                 if (base) inputs = base->inputs;
00854             }
00855 
00856         /**
00857          * @brief Constructs a `MetaHeuristicMetrics` object with explicit instance dimensions and hyperparameters.
00858          * @param algorithm_name_ Meta-heuristic algorithm identifier.
00859          * @param run_id Execution run number.
00860          * @param d Total device count.
00861          * @param s_ec Edge server count.
00862          * @param s_cc Cloud server count.
00863          * @param temperature_ Initial temperature \f$T_0\f$.
00864          * @param alpha_ Geometric cooling factor \f$\alpha\f$.
00865          * @param heuristic_ Initial constructive heuristic algorithm.
00866          */
00867         MetaHeuristicMetrics(Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc, double temperature_, double alpha_, Algorithm heuristic_) :
00868             Metrics(Paradigm::METAHEURISTIC, algorithm_name_, run_id, d, s_ec, s_cc), temperature(temperature_), alpha(alpha_), heuristic(heuristic_) {}
00869 
00870         /**
00871          * @brief Constructs a `MetaHeuristicMetrics` object from a base pointer and run ID.
00872          * @param base Base metrics pointer.
00873          * @param run_id Execution run number.
00874          */
00875         MetaHeuristicMetrics(const std::shared_ptr<Metrics>& base, int run_id) :
00876             Metrics(base, run_id) {}
00877 
00878         /**
00879          * @brief Clones this `MetaHeuristicMetrics` instance polymorphically.
00880          * @return Shared pointer to the cloned `MetaHeuristicMetrics` object.
00881          */
00882         std::shared_ptr<Metrics> clone() const override {
00883             return std::shared_ptr<MetaHeuristicMetrics>(new MetaHeuristicMetrics(*this));
00884         }
00885 
00886         /**
00887          * @brief Appends meta-heuristic parameters (`Heuristic`, `Temperature`, `Alpha`) to the metric entries.
00888          * @return Vector of `MetricEntry` records.
00889          */
00890         std::vector<MetricEntry> build_metrics() const override {
00891             auto m = Metrics::build_metrics();
00892             std::string cat = to_string(algorithm) + " PARAMETERS";
00893             m.push_back({cat, "Heuristic", "Initial Solution", to_string(heuristic), to_string(heuristic)});
00894             m.push_back({cat, "Temperature", "Initial Temperature", convert::to_string(temperature, DECIMAL_PLACES), convert::to_string(temperature, DECIMAL_PLACES)});
00895             m.push_back({cat, "Alpha", "Alpha (Cooling Rate)", convert::to_string(alpha, DECIMAL_PLACES), convert::to_string(alpha, DECIMAL_PLACES)});
00896             return m;
00897         }
00898 
00899         /**
00900          * @brief Appends meta-heuristic column headers (`Initial_Heuristic`, `Temperature`, `Alpha`) to the CSV header.
00901          * @return Vector of column header strings.
00902          */
00903         std::vector<std::string> get_header() const override {
00904             auto h = Metrics::get_header();
00905             h.push_back("Initial_Heuristic");
00906             h.push_back("Temperature");
```

```
00907             h.push_back("Alpha");
00908             return h;
00909         }
00910 
00911         /**
00912          * @brief Appends meta-heuristic hyperparameter values to the CSV row data.
00913          * @return Vector of formatted string values.
00914          */
00915         std::vector<std::string> data() const override {
00916             auto d = Metrics::data();
00917             d.push_back(to_string(heuristic));
00918             d.push_back(convert::to_string(temperature, DECIMAL_PLACES));
00919             d.push_back(convert::to_string(alpha, DECIMAL_PLACES));
00920             return d;
00921         }
00922     };
00923 
00924     /**
00925      * @struct Result
00926      * @brief Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.
00927      */
00928     struct Result {
00929         core::Devices devices;          ///< @brief Vector of all IoT devices in the simulation.
00930         core::Servers servers;          ///< @brief Vector of all Edge and Cloud servers in the simulation.
00931         std::shared_ptr<Metrics> stats; ///< @brief Polymorphic pointer to the run's metrics tracker.
00932 
00933         /**
00934          * @brief Constructs a `Result` state from pre-loaded devices and servers vectors.
00935          * @param d Vector of devices.
00936          * @param s Vector of servers.
00937          */
00938         Result(core::Devices d, core::Servers s)
00939             : devices(std::move(d)), servers(std::move(s)) {}
00940 
00941         /**
00942          * @brief Deep-copy constructor cloning both entities and the polymorphic `stats` pointer.
00943          * @param other Source `Result` instance to copy.
00944          */
00945         Result(const Result& other)
00946             : devices(other.devices), servers(other.servers) {
00947             if (other.stats) {
00948                 stats = other.stats->clone();
00949             }
00950         }
00951 
00952         /**
00953          * @brief Loads (or generates) an instance with the specified dimensions and initializes base metrics.
00954          * @param num_devices Number of IoT devices to load.
00955          * @param num_services Number of service templates.
00956          * @param num_ec Number of Edge Computing servers.
00957          * @param num_cc Number of Cloud Computing servers.
00958          * @param fixed_penalty Whether the penalty (cost) of each device is fixed or can vary.
00959          */
00960         Result(int num_devices, int num_services, int num_ec, int num_cc, bool fixed_penalty = true)
00961             : devices(data::load::devices(num_devices, num_services, fixed_penalty)), servers(data::load::servers(num_ec, num_cc)) {
00962 
00963             if (devices.size() <= 1 || servers.size() <= 1) {
00964                 std::cerr << "Error: Data failed to load.\n";
00965                 exit(0);
00966             }
00967 
00968             stats = std::make_shared<Metrics>(Paradigm::NONE, Algorithm::NONE, 0);
00969 
00970             stats->inputs.devices_count = static_cast<int>(this->devices.size()) - 1;
00971             stats->inputs.servers_ec_count = num_ec;
00972             stats->inputs.servers_cc_count = num_cc;
00973 
00974             double total_cost = 0.0;
00975             for (size_t d = 1; d < this->devices.size(); ++d) {
00976                 total_cost += this->devices[d].cost;
00977             }
00978             stats->inputs.max_penalty_cost = static_cast<double>(total_cost);
00979         }
00980 
00981         /**
00982          * @brief Deep-copy assignment operator cloning the polymorphic `stats` object.
00983          * @param other Source `Result` instance.
00984          * @return Reference to `*this`.
00985          */
00986         Result& operator=(const Result& other) {
00987             if (this != &other) {
00988                 devices = other.devices;
00989                 servers = other.servers;
```

```
00990                 if (other.stats) stats = other.stats->clone();
00991                 else stats.reset();
00992             }
00993             return *this;
00994         }
00995     };
00996
00997     /**
00998      * @brief Computes the final simulation metrics based on the current device-to-server allocations.
00999      * @param state The simulation result structure holding `devices`, `servers`, and `stats`.
01000      */
01001     inline void calculate_metrics(Result& state) {
01002         if (!state.stats) return;
01003
01004         auto& out = state.stats->outputs;
01005
01006         out.devices_served_count = 0;
01007         out.devices_served_ec_count = 0;
01008         out.devices_served_cc_count = 0;
01009         out.servers_used_count = 0;
01010         out.servers_used_ec_count = 0;
01011         out.servers_used_cc_count = 0;
01012         out.total_penalty_cost = 0.0;
01013
01014         for (size_t d = 1; d < state.devices.size(); ++d) {
01015             const auto& device = state.devices[d];
01016             if (device.served) {
01017                 out.devices_served_count++;
01018                 if (device.server_id != -1) {
01019                     if (state.servers[device.server_id].type == 'E') {
01020                         out.devices_served_ec_count++;
01021                     } else {
01022                         out.devices_served_cc_count++;
01023                     }
01024                 }
01025             } else {
01026                 out.total_penalty_cost += device.cost;
01027             }
01028         }
01029
01030         for (size_t s = 1; s < state.servers.size(); ++s) {
01031             const auto& server = state.servers[s];
01032             if (!server.supply.devices.empty()) {
01033                 out.servers_used_count++;
01034                 if (server.type == 'E') {
01035                     out.servers_used_ec_count++;
01036                 } else {
01037                     out.servers_used_cc_count++;
01038                 }
01039             }
01040         }
01041     }
01042
01043     /**
01044      * @brief Saves the final calculated metrics and ASCII summary tables to the file system.
01045      * @param state The simulation result structure to be saved.
01046      * @param show_servers_usage Flag to indicate if individual server resource usage should be
      printed.
01047      */
01048     inline void save_results(const Result& state, bool show_servers_usage = false);
01049 }
01050
01051 /**
01052  * @namespace display
01053  * @brief Utility namespace for formatting and printing entities and ASCII metric tables to console or
      files.
01054  */
01055 namespace display {
01056
01057     /**
01058      * @brief Displays detailed information of a single device to standard output.
01059      * @param device The device instance to display.
01060      */
01061     inline void show_device(const core::Device& device) {
01062         std::cout « "========== Device ID: "  « device.id « " ==========\n"
01063                   « "  - Cost Penalty:       " « device.cost « "\n"
01064                   « "  - Requirements (CPU): " « device.cpu « "\n"
01065                   « "  - Requirements (MEM): " « device.mem « "\n"
01066                   « "  - Requirements (STO): " « device.sto « "\n"
01067                   « "  - Tasks Count:        " « device.tasks « "\n"
01068                   « "  - Service ID:         " « device.service « "\n"
01069                   « "  - State (Served):     " « (device.served ? "Yes" : "No") « "\n";
01070
01071         if (device.served) {
01072             std::cout « "  - Assigned Server ID:   " « device.server_id « "\n";
01073         } else {
01074             std::cout « "  - Assigned Server ID:   None\n";
```

```
01075         }
01076         std::cout « "====================================\n" « std::endl;
01077     }
01078
01079     /**
01080      * @brief Displays detailed information for all devices in the vector.
01081      * @param devices The vector of devices to display.
01082      */
01083     inline void show_devices(const core::Devices& devices) {
01084         std::cout « "\n--- Displaying " « (devices.size() > 0 ? devices.size() - 1 : 0) « " Devices
      ---\n";
01085         for (const auto& device : devices) {
01086             if (device.id == 0) continue;
01087             show_device(device);
01088         }
01089     }
01090
01091     /**
01092      * @brief Displays capacity and current demand of a single server.
01093      * @param server The server instance to display.
01094      */
01095     inline void show_server(const core::Server& server) {
01096         std::cout « "========== Server ID: " « server.id « " (Type: " « server.type « ") ==========\n"
01097                   « "  --- Capacity ---\n"
01098                   « "  - Total CPU:           " « server.cpu « "\n"
01099                   « "  - Total MEM:           " « server.mem « "\n"
01100                   « "  - Total STO:           " « server.sto « "\n\n"
01101                   « "  --- Current Demand ---\n"
01102                   « "  - Used CPU:            " « server.supply.cpu « "\n"
01103                   « "  - Used MEM:            " « server.supply.mem « "\n"
01104                   « "  - Used STO:            " « server.supply.sto « "\n"
01105                   « "  - Devices Served (" « server.supply.devices.size() « "): ";
01106
01107         if (server.supply.devices.empty()) {
01108             std::cout « "None\n";
01109         } else {
01110             std::string device_list = convert::set_to_string(server.supply.devices);
01111             std::cout « device_list « "\n";
01112         }
01113         std::cout « "================================================\n" « std::endl;
01114     }
01115
01116     /**
01117      * @brief Displays capacity and demand information for all servers.
01118      * @param servers The vector of servers to display.
01119      */
01120     inline void show_servers(const core::Servers& servers) {
01121         std::cout « "\n--- Displaying " « (servers.size() > 0 ? servers.size() - 1 : 0) « " Servers
      ---\n";
01122         for (const auto& server : servers) {
01123             if (server.id == 0) continue;
01124             show_server(server);
01125         }
01126     }
01127
01128     /**
01129      * @namespace display::details
01130      * @brief Internal helper routines for rendering ASCII box-drawing tables.
01131      */
01132     namespace details {
01133         const int total_width = 72; ///< @brief Total character width of the formatted ASCII table.
01134         const int label_width = 26; ///< @brief Character width allocated to the left-hand label
      column.
01135
01136         /**
01137          * @brief Prints a formatted key-value row within the ASCII table.
01138          * @param out The target output stream.
01139          * @param label The row label string.
01140          * @param value The row value string.
01141          */
01142         inline void print_row(std::ostream& out, const std::string& label, const std::string& value) {
01143             out « "| " « std::left « std::setw(label_width) « label
01144                 « " | "« std::left « std::setw(total_width - label_width - 7) « value
01145                 « " |\n";
01146         }
01147
01148         /**
01149          * @brief Prints the top or bottom border of the ASCII table.
01150          * @param out The target output stream.
01151          */
01152         inline void print_header(std::ostream& out) {
01153             out « "+" « std::string(total_width - 2, '=') « "+\n";
01154         }
01155
01156         /**
01157          * @brief Prints a section separator line within the ASCII table.
01158          * @param out The target output stream.
```

```
01159          */
01160         inline void print_middle(std::ostream& out) {
01161             out « "+" « std::string(label_width + 2, '-') « "+"
01162                 « std::string(total_width - label_width - 5, '-') « "+\n";
01163         }
01164 
01165         /**
01166          * @brief Prints a centered title row within the ASCII table.
01167          * @param out The target output stream.
01168          * @param title The title text to center.
01169          */
01170         inline void print_title(std::ostream& out, const std::string& title) {
01171             int padding_total = total_width - 3 - static_cast<int>(title.length());
01172             int padding_left = padding_total / 2;
01173             int padding_right = padding_total - padding_left + 1;
01174             out « "|" « std::string(padding_left, ' ') « title
01175                 « std::string(padding_right, ' ') « "|\n";
01176         }
01177     }
01178 
01179     /**
01180      * @brief Formats and writes the complete simulation metrics table to an output stream.
01181      * @param metrics The metrics instance to format.
01182      * @param out The target output stream.
01183      */
01184     inline void show_metrics_console_or_file(const metrics::Metrics& metrics, std::ostream& out) {
01185         out « "\n";
01186         details::print_header(out);
01187 
01188         std::string title = "SIMULATION " + to_string(metrics.algorithm);
01189         details::print_title(out, title);
01190         details::print_middle(out);
01191 
01192         details::print_row(out, "Paradigm", to_string(metrics.paradigm));
01193         details::print_row(out, "Algorithm", to_string(metrics.algorithm));
01194         details::print_row(out, "Simulation Run", std::to_string(metrics.run_id));
01195 
01196         auto entries = metrics.build_metrics();
01197         std::string current_category = "";
01198 
01199         for (const auto& entry : entries) {
01200             if (entry.category != current_category) {
01201                 details::print_middle(out);
01202                 if (entry.category != "GENERAL") {
01203                     details::print_title(out, entry.category);
01204                     details::print_middle(out);
01205                 }
01206                 current_category = entry.category;
01207             }
01208             details::print_row(out, entry.label, entry.console_value);
01209         }
01210         details::print_header(out);
01211     }
01212 
01213     /**
01214      * @brief Displays the metrics table on the console and appends it to a drawing file.
01215      * @param ptr Shared pointer to the simulation metrics instance.
01216      */
01217     inline void show_metrics(const std::shared_ptr<metrics::Metrics>& ptr) {
01218         if (!ptr) return;
01219 
01220         show_metrics_console_or_file(*ptr, std::cout);
01221 
01222         std::filesystem::path base_dir = ptr->get_base_directory_path();
01223         std::filesystem::create_directories(base_dir);
01224 
01225         std::string config_name = "D" + std::to_string(ptr->inputs.devices_count) +
01226                                   "_S" + std::to_string(ptr->inputs.servers_ec_count +
      ptr->inputs.servers_cc_count);
01227 
01228         std::filesystem::path file_path = base_dir / (config_name + "_drawing.txt");
01229 
01230         std::ofstream file(file_path, std::ios::app);
01231         if (file.is_open()) {
01232             show_metrics_console_or_file(*ptr, file);
01233         } else {
01234             std::cerr « "Error: Could not save drawing to " « file_path « "\n";
01235         }
01236     }
01237 
01238     /**
01239      * @brief Prints a summary table of server resource utilization percentages.
01240      * @param servers The vector of servers to summarize.
01241      */
01242     inline void show_servers_usage(const core::Servers& servers) {
01243         std::cout « "\n+------------------------------------------------------------+\n";
01244         std::cout « "| SERVERS RESOURCE USAGE                                     |\n";
```

```
01245         std::cout « "+------------------------------------------------------------+\n";
01246
01247         for (const auto& server : servers) {
01248             if (server.id == 0) continue;
01249
01250             std::string cpu_pct = convert::to_percentage_string(server.supply.cpu, server.cpu, 0);
01251             std::string mem_pct = convert::to_percentage_string(server.supply.mem, server.mem, 0);
01252             std::string sto_pct = convert::to_percentage_string(server.supply.sto, server.sto, 0);
01253
01254             std::cout « "  ID: " « std::left « std::setw(4) « server.id
01255                       « "CPU: "  « std::right « std::setw(3) « cpu_pct « "%    "
01256                       « "MEM: "  « std::right « std::setw(3) « mem_pct « "%    "
01257                       « "STO: "  « std::right « std::setw(3) « sto_pct « "%\n";
01258         }
01259         std::cout « "+------------------------------------------------------------+\n\n";
01260     }
01261 }
01262
01263 inline void metrics::save_results(const Result& state, bool show_servers_usage) {
01264     if (!state.stats) return;
01265     state.stats->save_to_file();
01266     display::show_metrics(state.stats);
01267     if (show_servers_usage) display::show_servers_usage(state.servers);
01268 }
```

8.9 include/tools.hpp File Reference

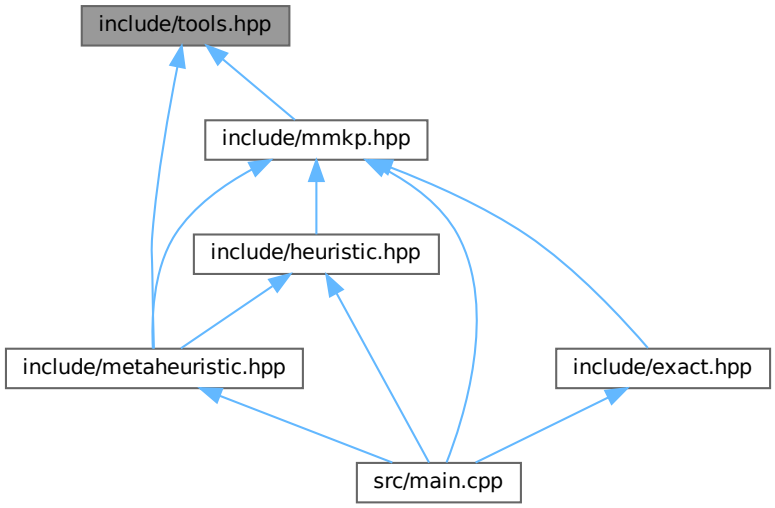
General-purpose utilities for CSV I/O, string formatting, random number generation, and indirect sorting.

```
#include <algorithm>
#include <chrono>
#include <cmath>
#include <exception>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <memory>
#include <numeric>
#include <optional>
#include <random>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <type_traits>
#include <utility>
#include <vector>
```

Include dependency graph for tools.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace file_manager::csv

  Provides file I/O operations for reading, writing, and appending delimited CSV/text files.
- namespace file_manager
- namespace convert

  Short namespace alias for `file_manager::csv`.
- namespace randomness

  Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.
- namespace sorting

  Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes.
- namespace sorting::details

  Internal implementation helpers for pointer-to-member index sorting.

Functions

- std::optional< std::vector< std::vector< std::string > > > file_manager::csv::read (const std::string &file_path, char delimiter=',')

  Reads a delimited CSV file into a 2D vector of strings.
- bool file_manager::csv::write (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Writes a 2D vector of strings to a CSV file, creating parent directories if needed.
- bool file_manager::csv::append (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Appends a 2D vector of strings to an existing or new CSV file.
- int file_manager::csv::count_lines (const std::string &file_path)

  Counts the total number of lines in a file.
- template<typename T >
  std::string convert::to_string (const T &value, int precision=DECIMAL_PLACES)

  Converts a value of any streamable type to its string representation.
- std::string convert::to_percentage_string (double numerator, double denominator, int precision=DECIMAL_PLACES)

Calculates a percentage and converts it to a formatted string.

- template<typename T >
  std::string convert::vector_to_string (const std::vector< T > &vec)

  Converts a vector of primitive types to a bracketed array string representation.
- template<typename T >
  std::string convert::set_to_string (const std::set< T > &s)

  Converts a `std::set` of primitive types to a bracketed array string representation.
- std::mt19937 & randomness::get_engine ()

  Provides a singleton instance of a pseudo-random number engine.
- template<typename T >
  T randomness::random_number (T min, T max)

  Generates a uniform random number within a specified inclusive interval `[min, max]`.
- template<typename T >
  T randomness::random_number (T min, T max, std::vector< T > &exclude)

  Generates a random number within `[min, max]`, excluding values present in `exclude`.
- template<typename T >
  std::vector< T > randomness::shuffled_range (T min, T max)

  Creates a randomly shuffled vector containing all integers in the inclusive range `[min, max]`.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  void sorting::details::sort_indices_impl (const std::vector< T > &entities, std::vector< int > &idxs)

  Sorts a vector of indices based on a specified struct/class member attribute.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sorting::sort_entities (const std::vector< T > &entities)

  Sorts the indices of all entities (from `1` to `N-1`) by a specific struct/class attribute.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sorting::sort_entities (const std::vector< T > &entities, const std::vector< int > &indices_to_sort)

  Sorts a specific subset of entity indices by a struct/class member attribute.

Variables

- constexpr double ZERO = 1e-13

  Small epsilon constant used for floating-point comparisons and non-zero lower bounds.
- constexpr int DECIMAL_PLACES = 10

  Global default precision (number of decimal places) for floating-point string formatting.

8.9.1 Detailed Description

General-purpose utilities for CSV I/O, string formatting, random number generation, and indirect sorting.
Header-only toolkit providing foundational helper functions used across the MMKP simulator.
Definition in file tools.hpp.

8.9.2 Variable Documentation

DECIMAL_PLACES

```
constexpr int DECIMAL_PLACES = 10  [constexpr]
```

Global default precision (number of decimal places) for floating-point string formatting.
Definition at line 38 of file tools.hpp.
Referenced by metrics::Metrics::build_metrics(), metrics::ExactMetrics::build_metrics(), metrics::MetaHeuristicMetrics::build_metrics(), metrics::Metrics::data(), metrics::ExactMetrics::data(), metrics::MetaHeuristicMetrics::data(), data::generate::servers(), and data::generate::services().

ZERO

```
constexpr double ZERO = 1e-13  [constexpr]
```

Small epsilon constant used for floating-point comparisons and non-zero lower bounds.
Definition at line 33 of file tools.hpp.
Referenced by data::generate::services(), and metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing().

8.10 tools.hpp

Go to the documentation of this file.

```
00001 /**
00002  * @file tools.hpp
00003  * @brief General-purpose utilities for CSV I/O, string formatting, random number generation, and
      indirect sorting.
00004  * @details Header-only toolkit providing foundational helper functions used across the MMKP
      simulator.
00005  */
00006
00007 #pragma once
00008
00009 #include <algorithm>
00010 #include <chrono>
00011 #include <cmath>
00012 #include <exception>
00013 #include <filesystem>
00014 #include <fstream>
00015 #include <iomanip>
00016 #include <iostream>
00017 #include <memory>
00018 #include <numeric>
00019 #include <optional>
00020 #include <random>
00021 #include <set>
00022 #include <sstream>
00023 #include <stdexcept>
00024 #include <string>
00025 #include <type_traits>
00026 #include <utility>
00027 #include <vector>
00028
00029
00030 /**
00031  * @brief Small epsilon constant used for floating-point comparisons and non-zero lower bounds.
00032  */
00033 constexpr double ZERO = 1e-13;
00034
00035 /**
00036  * @brief Global default precision (number of decimal places) for floating-point string formatting.
00037  */
00038 constexpr int DECIMAL_PLACES = 10;
00039
00040 /**
00041  * @namespace file_manager::csv
00042  * @brief Provides file I/O operations for reading, writing, and appending delimited CSV/text files.
00043  */
00044 namespace file_manager::csv {
00045
00046     /**
00047      * @brief Reads a delimited CSV file into a 2D vector of strings.
00048      * @param file_path The full path to the file to be read.
00049      * @param delimiter The character that separates columns (defaults to `','`).
00050      * @return An `std::optional` containing the 2D vector of strings upon success, or `std::nullopt`
      on failure.
00051      */
00052     inline std::optional<std::vector<std::vector<std::string>>> read(const std::string& file_path, char
      delimiter = ',') {
00053         std::ifstream file(file_path);
00054         if (!file.is_open()) {
00055             std::cerr << "Error: Could not open file " << file_path << "\n";
00056             return std::nullopt;
00057         }
00058
00059         std::vector<std::vector<std::string>> file_data;
00060         std::string line;
00061
00062         while (std::getline(file, line)) {
00063             std::vector<std::string> row;
00064             std::stringstream ss(line);
00065             std::string cell;
00066
00067             while (std::getline(ss, cell, delimiter)) {
00068                 row.push_back(cell);
00069             }
```

```
00070
00071             if (!row.empty()) {
00072                 file_data.push_back(row);
00073             }
00074         }
00075         return file_data;
00076     }
00077
00078     /**
00079      * @brief Writes a 2D vector of strings to a CSV file, creating parent directories if needed.
00080      * @param file_path The full path to the output file.
00081      * @param data The 2D vector of strings to write.
00082      * @param delimiter The character used for separating columns (defaults to `','`).
00083      * @return `true` if the file was written successfully; `false` otherwise.
00084      */
00085     inline bool write(const std::string& file_path, const std::vector<std::vector<std::string>& data,
      char delimiter = ',') {
00086         std::filesystem::path path(file_path);
00087         if (path.has_parent_path()) {
00088             std::filesystem::create_directories(path.parent_path());
00089         }
00090
00091         std::ofstream file(file_path);
00092         if (!file.is_open()) {
00093             std::cerr << "Error: Could not create or open file " << file_path << "\n";
00094             return false;
00095         }
00096
00097         for (size_t i = 0; i < data.size(); ++i) {
00098             for (size_t j = 0; j < data[i].size(); ++j) {
00099                 file << data[i][j];
00100                 if (j < data[i].size() - 1) {
00101                     file << delimiter;
00102                 }
00103             }
00104             if (i < data.size() - 1) {
00105                 file << "\n";
00106             }
00107         }
00108         return true;
00109     }
00110
00111     /**
00112      * @brief Appends a 2D vector of strings to an existing or new CSV file.
00113      * @param file_path The full path to the output file.
00114      * @param data The 2D vector of strings to append.
00115      * @param delimiter The character used for separating columns (defaults to `','`).
00116      * @return `true` if the data was appended successfully; `false` otherwise.
00117      */
00118     inline bool append(const std::string& file_path, const std::vector<std::vector<std::string>& data,
      char delimiter = ',') {
00119         std::filesystem::path path(file_path);
00120         if (path.has_parent_path()) {
00121             std::filesystem::create_directories(path.parent_path());
00122         }
00123
00124         std::ofstream file(file_path, std::ios::app);
00125         if (!file.is_open()) {
00126             std::cerr << "Error: Could not create or open file for appending " << file_path << "\n";
00127             return false;
00128         }
00129
00130         if (file.tellp() > 0) {
00131             file << "\n";
00132         }
00133
00134         for (size_t i = 0; i < data.size(); ++i) {
00135             for (size_t j = 0; j < data[i].size(); ++j) {
00136                 file << data[i][j];
00137                 if (j < data[i].size() - 1) {
00138                     file << delimiter;
00139                 }
00140             }
00141             if (i < data.size() - 1) {
00142                 file << "\n";
00143             }
00144         }
00145         return true;
00146     }
00147
00148     /**
00149      * @brief Counts the total number of lines in a file.
00150      * @param file_path The full path to the file.
00151      * @return The total number of lines, or `0` if the file does not exist or cannot be opened.
00152      */
00153     inline int count_lines(const std::string& file_path) {
00154         if (!std::filesystem::exists(file_path)) {
```

```
00155             return 0;
00156         }
00157
00158         std::ifstream file(file_path);
00159         if (!file.is_open()) {
00160             return 0;
00161         }
00162
00163         int count = 0;
00164         std::string line;
00165         while (std::getline(file, line)) {
00166             count++;
00167         }
00168         return count;
00169     }
00170
00171 }
00172
00173 /**
00174  * @brief Short namespace alias for `file_manager::csv`.
00175  */
00176 namespace csv = file_manager::csv;
00177
00178 /**
00179  * @namespace convert
00180  * @brief String conversion and formatting utilities for numbers, percentages, vectors, and sets.
00181  */
00182 namespace convert {
00183
00184     /**
00185      * @brief Converts a value of any streamable type to its string representation.
00186      * @details Uses `std::ostringstream` for conversion, applying fixed-point notation with the
      specified precision for floating-point types.
00187      * @tparam T The data type of the value to be converted.
00188      * @param value The value to convert.
00189      * @param precision The number of decimal places for floating-point types.
00190      * @return A formatted string representation of `value`.
00191      */
00192     template <typename T>
00193     inline std::string to_string(const T& value, int precision = DECIMAL_PLACES) {
00194         std::ostringstream out;
00195         if constexpr (std::is_floating_point_v<T>) {
00196             out << std::fixed << std::setprecision(precision);
00197         }
00198         out << value;
00199         return out.str();
00200     }
00201
00202     /**
00203      * @brief Calculates a percentage and converts it to a formatted string.
00204      * @details Computes `(numerator / denominator) * 100.0` and formats it to `precision` decimal
      places. Safely returns `0.0` when `denominator == 0.0`.
00205      * @param numerator The value representing the part of the total.
00206      * @param denominator The value representing the total.
00207      * @param precision The number of decimal places for the resulting percentage string.
00208      * @return A string representation of the calculated percentage.
00209      */
00210     inline std::string to_percentage_string(double numerator, double denominator, int precision =
      DECIMAL_PLACES) {
00211         if (denominator == 0.0) {
00212             return to_string(0.0, precision);
00213         }
00214         double percentage = (numerator / denominator) * 100.0;
00215         return to_string(percentage, precision);
00216     }
00217
00218     /**
00219      * @brief Converts a vector of primitive types to a bracketed array string representation.
00220      * @tparam T The type of elements in the vector.
00221      * @param vec The vector to convert.
00222      * @return A string formatted as `"[e1,e2,...]"`.
00223      */
00224     template <typename T>
00225     inline std::string vector_to_string(const std::vector<T>& vec) {
00226         if (vec.empty()) return "[]";
00227         std::string str = "[";
00228         for (size_t i = 0; i < vec.size(); ++i) {
00229             str += std::to_string(vec[i]);
00230             if (i < vec.size() - 1) str += ",";
00231         }
00232         str += "]";
00233         return str;
00234     }
00235
00236     /**
00237      * @brief Converts a `std::set` of primitive types to a bracketed array string representation.
00238      * @tparam T The type of elements in the set.
```

```
00239      * @param s The set to convert.
00240      * @return A string formatted as `"[e1,e2,...]"`.
00241      */
00242     template <typename T>
00243     inline std::string set_to_string(const std::set<T>& s) {
00244         if (s.empty()) return "[]";
00245         std::string str = "[";
00246         auto it = s.begin();
00247         while (it != s.end()) {
00248             str += std::to_string(*it);
00249             ++it;
00250             if (it != s.end()) str += ",";
00251         }
00252         str += "]";
00253         return str;
00254     }
00255 }
00256
00257 /**
00258  * @namespace randomness
00259  * @brief Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.
00260  */
00261 namespace randomness {
00262
00263     /**
00264      * @brief Provides a singleton instance of a pseudo-random number engine.
00265      * @details Returns a reference to a static `std::mt19937` engine seeded once via
    `std::random_device`.
00266      * @return A reference to the static `std::mt19937` random engine.
00267      */
00268     inline std::mt19937& get_engine() {
00269         static std::random_device rd;
00270         static std::mt19937 engine(rd());
00271         return engine;
00272     }
00273
00274     /**
00275      * @brief Generates a uniform random number within a specified inclusive interval `[min, max]`.
00276      * @tparam T The numeric type (must satisfy `std::is_arithmetic_v<T>`).
00277      * @param min The lower bound of the interval (inclusive).
00278      * @param max The upper bound of the interval (inclusive).
00279      * @return A uniformly distributed random number within `[min, max]`.
00280      * @throws std::invalid_argument If `min > max`.
00281      */
00282     template<typename T>
00283     inline T random_number(T min, T max) {
00284         static_assert(std::is_arithmetic_v<T>, "T must be an arithmetic type.");
00285         if (min > max) {
00286             throw std::invalid_argument("Error in random_number: min cannot be greater than max.");
00287         }
00288
00289         if constexpr (std::is_integral_v<T>) {
00290             std::uniform_int_distribution<T> distribution(min, max);
00291             return distribution(get_engine());
00292         } else {
00293             std::uniform_real_distribution<T> distribution(min, max);
00294             return distribution(get_engine());
00295         }
00296     }
00297
00298     /**
00299      * @brief Generates a random number within `[min, max]`, excluding values present in `exclude`.
00300      * @details The generated number is automatically appended to `exclude` to prevent future
    duplicates.
00301      * @tparam T The numeric type (must satisfy `std::is_arithmetic_v<T>`).
00302      * @param min The lower bound of the interval (inclusive).
00303      * @param max The upper bound of the interval (inclusive).
00304      * @param exclude A reference to a vector containing values to be excluded.
00305      * @return A random number within `[min, max]` not previously in `exclude`.
00306      * @throws std::invalid_argument If `min > max`, or if all possible integral values in the range
    are already excluded.
00307      */
00308     template<typename T>
00309     inline T random_number(T min, T max, std::vector<T>& exclude) {
00310         static_assert(std::is_arithmetic_v<T>, "T must be an arithmetic type.");
00311
00312         if (min > max) {
00313             throw std::invalid_argument("Error in random_number: min cannot be greater than max.");
00314         }
00315
00316         if constexpr (std::is_integral_v<T>) {
00317             if (exclude.size() >= static_cast<size_t>(max - min + 1)) {
00318                 throw std::invalid_argument("Error in random_number: all possible values are excluded
    causing infinite loop.");
00319             }
00320         }
00321
```

```
00322         T result;
00323         if constexpr (std::is_integral_v<T>) {
00324             std::uniform_int_distribution<T> distribution(min, max);
00325             do {
00326                 result = distribution(get_engine());
00327             } while (std::find(exclude.begin(), exclude.end(), result) != exclude.end());
00328         } else {
00329             std::uniform_real_distribution<T> distribution(min, max);
00330             do {
00331                 result = distribution(get_engine());
00332             } while (std::find(exclude.begin(), exclude.end(), result) != exclude.end());
00333         }
00334 
00335         exclude.push_back(result);
00336         return result;
00337     }
00338 
00339     /**
00340      * @brief Creates a randomly shuffled vector containing all integers in the inclusive range `[min,
      max]`.
00341      * @tparam T The integral type for the range.
00342      * @param min The lower bound of the range (inclusive).
00343      * @param max The upper bound of the range (inclusive).
00344      * @return A randomly shuffled `std::vector<T>` of unique integers from `min` to `max`.
00345      * @throws std::invalid_argument If `min > max`.
00346      */
00347     template<typename T>
00348     inline std::vector<T> shuffled_range(T min, T max) {
00349         static_assert(std::is_integral_v<T>, "shuffled_range requires an integral type.");
00350         if (min > max) {
00351             throw std::invalid_argument("Error in shuffled_range: min cannot be greater than max.");
00352         }
00353         std::vector<T> numbers(max - min + 1);
00354         std::iota(numbers.begin(), numbers.end(), min);
00355         std::shuffle(numbers.begin(), numbers.end(), get_engine());
00356         return numbers;
00357     }
00358 }
00359 
00360 /**
00361  * @namespace sorting
00362  * @brief Generic index-based sorting utilities for ordering 1-indexed entity vectors by member
      attributes.
00363  */
00364 namespace sorting {
00365 
00366     /**
00367      * @namespace sorting::details
00368      * @brief Internal implementation helpers for pointer-to-member index sorting.
00369      */
00370     namespace details {
00371 
00372         /**
00373          * @brief Sorts a vector of indices based on a specified struct/class member attribute.
00374          * @tparam Ascending Sort direction flag (`true` for ascending, `false` for descending).
00375          * @tparam T Entity struct/class type.
00376          * @tparam AttributeType Data type of the sorted member attribute.
00377          * @tparam AttributePtr Pointer-to-member specifying the comparison attribute.
00378          * @param entities Constant reference to the entity vector.
00379          * @param idxs Vector of 1-based indices to be sorted in-place.
00380          */
00381         template <bool Ascending, typename T, typename AttributeType, AttributeType T::* AttributePtr>
00382         void sort_indices_impl(const std::vector<T>& entities, std::vector<int>& idxs) {
00383             std::sort(idxs.begin(), idxs.end(), [&](int i, int j) {
00384                 const auto& attr_i = entities.at(i).*AttributePtr;
00385                 const auto& attr_j = entities.at(j).*AttributePtr;
00386 
00387                 if (attr_i != attr_j) {
00388                     if constexpr (Ascending) {
00389                         return attr_i < attr_j; // Ascending order
00390                     } else {
00391                         return attr_i > attr_j; // Descending order
00392                     }
00393                 }
00394                 return i < j; // Tie-breaker
00395             });
00396         }
00397     }
00398 
00399     /**
00400      * @brief Sorts the indices of all entities (from `1` to `N-1`) by a specific struct/class
      attribute.
00401      * @details Does not mutate the original `entities` vector. Assumes 1-based indexing where index
      `0` is a placeholder.
00402      * @tparam Ascending If `true`, sorts in ascending order; otherwise, descending.
00403      * @tparam T The type of the entity.
00404      * @tparam AttributeType The data type of the member attribute.
```

```
00405      * @tparam AttributePtr Pointer-to-member specifying the attribute for comparison.
00406      * @param entities The constant 1-indexed vector of entities.
00407      * @return A new `std::vector<int>` containing the sorted 1-based indices.
00408      */
00409     template <bool Ascending, typename T, typename AttributeType, AttributeType T::* AttributePtr>
00410     inline std::vector<int> sort_entities(const std::vector<T>& entities) {
00411         if (entities.size() <= 1) return {};
00412         std::vector<int> idxs(entities.size() - 1);
00413         std::iota(idxs.begin(), idxs.end(), 1); // Assumes 1-based indexing
00414         details::sort_indices_impl<Ascending, T, AttributeType, AttributePtr>(entities, idxs);
00415         return idxs;
00416     }
00417
00418     /**
00419      * @brief Sorts a specific subset of entity indices by a struct/class member attribute.
00420      * @tparam Ascending If `true`, sorts in ascending order; otherwise, descending.
00421      * @tparam T The type of the entity.
00422      * @tparam AttributeType The data type of the member attribute.
00423      * @tparam AttributePtr Pointer-to-member specifying the attribute for comparison.
00424      * @param entities The constant 1-indexed vector of entities.
00425      * @param indices_to_sort A vector of integers representing the subset of 1-based indices to sort.
00426      * @return A new `std::vector<int>` containing the sorted subset indices.
00427      */
00428     template <bool Ascending, typename T, typename AttributeType, AttributeType T::* AttributePtr>
00429     inline std::vector<int> sort_entities(const std::vector<T>& entities, const std::vector<int>&
    indices_to_sort) {
00430         if (entities.empty() || indices_to_sort.empty()) return {};
00431         std::vector<int> idxs = indices_to_sort; // Create a copy to sort
00432         details::sort_indices_impl<Ascending, T, AttributeType, AttributePtr>(entities, idxs);
00433         return idxs;
00434     }
00435 }
```

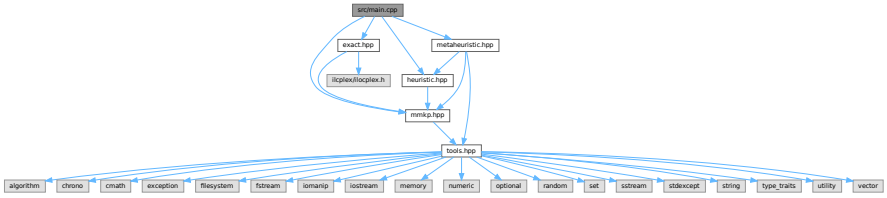
8.11 README.md File Reference

8.12 src/main.cpp File Reference

Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator.

```
#include "mmkp.hpp"
#include "exact.hpp"
#include "heuristic.hpp"
#include "metaheuristic.hpp"
```

Include dependency graph for main.cpp:



Functions

- void run_simulation (const metrics::Result &base, int option, int run_id)

  Executes a specific algorithm configuration for the network resource allocation simulation.
- int main ()

  Main entry point for the network resource allocation simulation benchmark.

Variables

- constexpr int SERVICES = 100

  Total number of distinct service profiles generated.
- constexpr int DEVICES_START = 10000

  Initial number of IoT devices in the scaling benchmark.
- constexpr int DEVICES_STEP = 1000

Increment step for the number of IoT devices per batch.

- constexpr int DEVICES_END = 20000

  Maximum number of IoT devices in the scaling benchmark.

- constexpr int EC_LENGTH = 499

  Total number of Edge Computing (EC) servers available.

- constexpr int CC_LENGTH = 1

  Total number of Cloud Computing (CC) servers available.

- constexpr int MAX_RUNS = 50

  Number of independent runs per instance configuration.

- constexpr double CPLEX_TIME_LIMIT = 3600

  Time limit in seconds for the CPLEX exact solver.

- constexpr double TEMPERATURE = 50.0

  Initial temperature for Simulated Annealing.

- constexpr double ALPHA = 0.9999

  Geometric cooling factor for Simulated Annealing.

- constexpr bool FIXED_PENALTY = false

  Whether the penalty (cost) of each device is fixed or can vary.

8.12.1 Detailed Description

Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator.
Defines the experimental parameters (scaling of IoT devices, server counts, CPLEX time limit, and Simulated Annealing cooling schedule) and orchestrates the execution across Exact, Heuristic, and Meta-heuristic paradigms.
Definition in file main.cpp.

8.12.2 Function Documentation

main()

```
int main ( )
```

Main entry point for the network resource allocation simulation benchmark.
Iterates over the configured range of device counts and independent runs, executing deterministic algorithms once per instance and stochastic algorithms across all runs.

Returns

int Exit status code (`0` on normal termination).

Definition at line 107 of file main.cpp.
References CC_LENGTH, DEVICES_END, DEVICES_START, DEVICES_STEP, EC_LENGTH, FIXED_PENALTY, MAX_RUNS, run_simulation(), and SERVICES.

run_simulation()

```
void run_simulation (
            const metrics::Result & base,
            int option,
            int run_id )
```

Executes a specific algorithm configuration for the network resource allocation simulation.
Instantiates the appropriate metrics tracker based on the selected option, dispatches the simulation state to the corresponding solver paradigm (Exact, Heuristic, or Meta-heuristic), computes final utilization/cost metrics, and saves the results to disk.

Parameters

base	The initial system state containing untouched devices and servers.

Parameters

option	The identifier for the algorithm combination to execute: • `0`: Exact CPLEX solver (`solver_maximum_cost`) • `-1`: Constructive heuristic (`greedy_cost_benefit`) • `-2`: Constructive heuristic (`greedy_random`) • `-3`: Constructive heuristic (`greedy_random_fallback`) • `1`: Simulated Annealing + `greedy_cost_benefit` • `2`: Simulated Annealing + `greedy_random` • `3`: Simulated Annealing + `greedy_random_fallback`
run↩ _id	The current execution run number (1-indexed).

Definition at line 43 of file main.cpp.
References ALPHA, heuristic::boot(), metaheuristic::boot(), exact::boot(), metrics::calculate_metrics(), CPLEX_TIME_LIMIT, metrics::EXACT, metrics::greedy_cost_benefit, metrics::greedy_random, metrics::greedy_random_fallback, metrics::HEURISTIC, metrics::METAHEURISTIC, metrics::NONE, metrics::save_results(), metrics::simulated_annealing, metrics::solver_maximum_cost, metrics::Result::stats, and TEMPERATURE.
Referenced by main().

8.12.3 Variable Documentation

ALPHA

```
constexpr double ALPHA = 0.9999  [constexpr]
```
Geometric cooling factor for Simulated Annealing.
Definition at line 23 of file main.cpp.
Referenced by run_simulation().

CC_LENGTH

```
constexpr int CC_LENGTH = 1  [constexpr]
```
Total number of Cloud Computing (CC) servers available.
Definition at line 19 of file main.cpp.
Referenced by main().

CPLEX_TIME_LIMIT

```
constexpr double CPLEX_TIME_LIMIT = 3600  [constexpr]
```
Time limit in seconds for the CPLEX exact solver.
Definition at line 21 of file main.cpp.
Referenced by run_simulation().

DEVICES_END

```
constexpr int DEVICES_END = 20000  [constexpr]
```
Maximum number of IoT devices in the scaling benchmark.
Definition at line 17 of file main.cpp.
Referenced by main().

DEVICES_START

```
constexpr int DEVICES_START = 10000  [constexpr]
```
Initial number of IoT devices in the scaling benchmark.

Definition at line 15 of file main.cpp.
Referenced by main().

DEVICES_STEP

```
constexpr int DEVICES_STEP = 1000  [constexpr]
```
Increment step for the number of IoT devices per batch.
Definition at line 16 of file main.cpp.
Referenced by main().

EC_LENGTH

```
constexpr int EC_LENGTH = 499  [constexpr]
```
Total number of Edge Computing (EC) servers available.
Definition at line 18 of file main.cpp.
Referenced by main().

FIXED_PENALTY

```
constexpr bool FIXED_PENALTY = false  [constexpr]
```
Whether the penalty (cost) of each device is fixed or can vary.
Definition at line 24 of file main.cpp.
Referenced by main().

MAX_RUNS

```
constexpr int MAX_RUNS = 50  [constexpr]
```
Number of independent runs per instance configuration.
Definition at line 20 of file main.cpp.
Referenced by main().

SERVICES

```
constexpr int SERVICES = 100  [constexpr]
```
Total number of distinct service profiles generated.
Definition at line 14 of file main.cpp.
Referenced by main().

TEMPERATURE

```
constexpr double TEMPERATURE = 50.0  [constexpr]
```
Initial temperature for Simulated Annealing.
Definition at line 22 of file main.cpp.
Referenced by run_simulation().

8.13 main.cpp

Go to the documentation of this file.

```
00001 /**
00002  * @file main.cpp
00003  * @brief Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator.
00004  * @details Defines the experimental parameters (scaling of IoT devices, server counts,
00005  * CPLEX time limit, and Simulated Annealing cooling schedule) and orchestrates
00006  * the execution across Exact, Heuristic, and Meta-heuristic paradigms.
00007  */
00008
00009 #include "mmkp.hpp"
00010 #include "exact.hpp"
00011 #include "heuristic.hpp"
00012 #include "metaheuristic.hpp"
00013
00014 constexpr int SERVICES = 100;             ///< @brief Total number of distinct service profiles
      generated.
00015 constexpr int DEVICES_START = 10000;      ///< @brief Initial number of IoT devices in the scaling
      benchmark.
```

```
00016 constexpr int DEVICES_STEP = 1000;        ///< @brief Increment step for the number of IoT devices per
      batch.
00017 constexpr int DEVICES_END = 20000;        ///< @brief Maximum number of IoT devices in the scaling
      benchmark.
00018 constexpr int EC_LENGTH = 499;            ///< @brief Total number of Edge Computing (EC) servers
      available.
00019 constexpr int CC_LENGTH = 1;              ///< @brief Total number of Cloud Computing (CC) servers
      available.
00020 constexpr int MAX_RUNS = 50;             ///< @brief Number of independent runs per instance
      configuration.
00021 constexpr double CPLEX_TIME_LIMIT = 3600; ///< @brief Time limit in seconds for the CPLEX exact
      solver.
00022 constexpr double TEMPERATURE = 50.0;      ///< @brief Initial temperature for Simulated Annealing.
00023 constexpr double ALPHA = 0.9999;          ///< @brief Geometric cooling factor for Simulated
      Annealing.
00024 constexpr bool FIXED_PENALTY = false;     ///< @brief Whether the penalty (cost) of each device is
      fixed or can vary.
00025
00026 /**
00027  * @brief Executes a specific algorithm configuration for the network resource allocation simulation.
00028  * @details Instantiates the appropriate metrics tracker based on the selected option, dispatches
00029  * the simulation state to the corresponding solver paradigm (Exact, Heuristic, or Meta-heuristic),
00030  * computes final utilization/cost metrics, and saves the results to disk.
00031  *
00032  * @param base The initial system state containing untouched devices and servers.
00033  * @param option The identifier for the algorithm combination to execute:
00034  *               - `0`: Exact CPLEX solver (`solver_maximum_cost`)
00035  *               - `-1`: Constructive heuristic (`greedy_cost_benefit`)
00036  *               - `-2`: Constructive heuristic (`greedy_random`)
00037  *               - `-3`: Constructive heuristic (`greedy_random_fallback`)
00038  *               - `1`: Simulated Annealing + `greedy_cost_benefit`
00039  *               - `2`: Simulated Annealing + `greedy_random`
00040  *               - `3`: Simulated Annealing + `greedy_random_fallback`
00041  * @param run_id The current execution run number (1-indexed).
00042  */
00043 void run_simulation(const metrics::Result& base, int option, int run_id) {
00044     metrics::Paradigm paradigm = metrics::Paradigm::NONE;
00045     metrics::Algorithm algorithm = metrics::Algorithm::NONE;
00046     metrics::Algorithm initial_heuristic = metrics::Algorithm::NONE;
00047
00048     if (option == 0) {
00049         paradigm = metrics::Paradigm::EXACT;
00050         algorithm = metrics::Algorithm::solver_maximum_cost;
00051     } else if (option == -1) {
00052         paradigm = metrics::Paradigm::HEURISTIC;
00053         algorithm = metrics::Algorithm::greedy_cost_benefit;
00054     } else if (option == -2) {
00055         paradigm = metrics::Paradigm::HEURISTIC;
00056         algorithm = metrics::Algorithm::greedy_random;
00057     } else if (option == -3) {
00058         paradigm = metrics::Paradigm::HEURISTIC;
00059         algorithm = metrics::Algorithm::greedy_random_fallback;
00060     } else if (option == 1) {
00061         paradigm = metrics::Paradigm::METAHEURISTIC;
00062         algorithm = metrics::Algorithm::simulated_annealing;
00063         initial_heuristic = metrics::Algorithm::greedy_cost_benefit;
00064     } else if (option == 2) {
00065         paradigm = metrics::Paradigm::METAHEURISTIC;
00066         algorithm = metrics::Algorithm::simulated_annealing;
00067         initial_heuristic = metrics::Algorithm::greedy_random;
00068     } else if (option == 3) {
00069         paradigm = metrics::Paradigm::METAHEURISTIC;
00070         algorithm = metrics::Algorithm::simulated_annealing;
00071         initial_heuristic = metrics::Algorithm::greedy_random_fallback;
00072     } else {
00073         std::cerr << "Error: Invalid option selected for simulation.\n";
00074         return;
00075     }
00076
00077     metrics::Result state = base;
00078
00079     if (paradigm == metrics::Paradigm::EXACT) {
00080         state.stats = std::make_shared<metrics::ExactMetrics>(algorithm, run_id, base.stats);
00081     } else if (paradigm == metrics::Paradigm::HEURISTIC) {
00082         state.stats = std::make_shared<metrics::HeuristicMetrics>(algorithm, run_id, base.stats);
00083     } else if (paradigm == metrics::Paradigm::METAHEURISTIC) {
00084         state.stats = std::make_shared<metrics::MetaHeuristicMetrics>(algorithm, run_id, base.stats,
      TEMPERATURE, ALPHA, initial_heuristic);
00085     }
00086
00087     if (paradigm == metrics::Paradigm::EXACT) {
00088         exact::boot(state, CPLEX_TIME_LIMIT);
00089     } else if (paradigm == metrics::Paradigm::HEURISTIC) {
00090         heuristic::boot(state);
00091     } else if (paradigm == metrics::Paradigm::METAHEURISTIC) {
00092         metaheuristic::boot(state);
00093     }
```

```
00094
00095     metrics::calculate_metrics(state);
00096     metrics::save_results(state, false);
00097 }
00098
00099 /**
00100  * @brief Main entry point for the network resource allocation simulation benchmark.
00101  * @details Iterates over the configured range of device counts and independent runs,
00102  * executing deterministic algorithms once per instance and stochastic algorithms across
00103  * all runs.
00104  *
00105  * @return int Exit status code (`0` on normal termination).
00106  */
00107 int main() {
00108     for (int devices_length = DEVICES_START; devices_length <= DEVICES_END; devices_length +=
      DEVICES_STEP) {
00109         metrics::Result base_state(devices_length, SERVICES, EC_LENGTH, CC_LENGTH, FIXED_PENALTY);
00110
00111         for (int run = 1; run <= MAX_RUNS; run++) {
00112             for (int option = 0; option <= 3; option++) {
00113                 if (run > 1 && (option == 0 || option == -1)) continue;
00114                 run_simulation(base_state, option, run);
00115             }
00116         }
00117     }
00118
00119     return 0;
00120 }
```

Index

convert, 6
    set_to_string, 6

MMKP Resource Allocation in IoT-EC-CC Networks, 1

set_to_string
    convert, 6